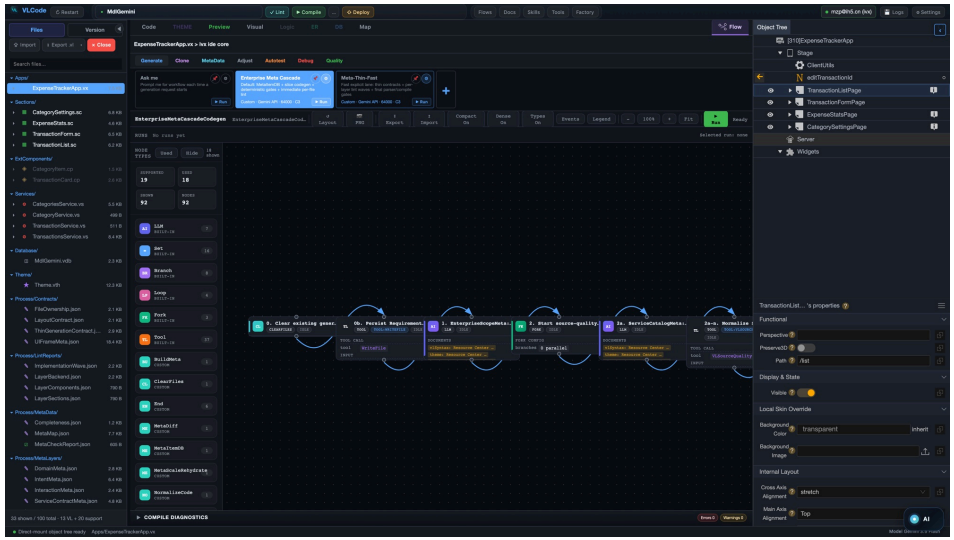
VL AGENT

# VL Agent Complete Tutorial

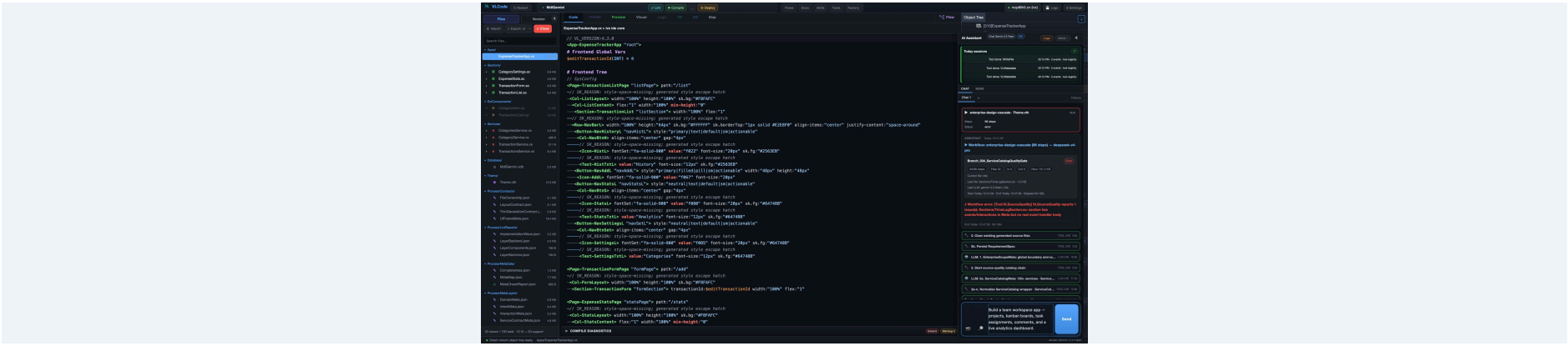
The Local Agent OS, Workflows, Agent Apps, And The .vflow Pipeline

- For workflow authors, Agent App developers, delivery teams, and VLCode collaborators.
- It starts from concepts, then takes you through authoring, running, and debugging workflows on the local DAG canvas, and shipping .vflow bundles to VLCode.



VL Agent shell — the DAG canvas with the floating AI Assistant

# How To Use This Tutorial



A concise project brief in the assistant panel

## LEARNING ORDER

- Read concepts first, follow interface operations next, finish with case studies.
- Do not skip validation chapters; Lint/Compile and the run ledger are central.

## PRACTICE METHOD

- Practice each concept with one small scoped operation.
- After each operation, save errors, screenshots, or logs so the run is reproducible.

## DELIVERY STANDARD

- Explain the project / workflow structure.
- Run a full flow end to end.
- Read failure logs and make a minimal fix.

# Roadmap I

## 1. PART I: WHAT VL AGENT IS

- What VL Agent Is
- What A Local Agent OS Means
- Workflows, DAGs, And Nodes
- What An Agent App Is
- ...and more pages

## 2. PART II: INSTALLATION, STARTUP, AND WORKSPACE

- Choose Desktop, Browser, Or Gateway
- Install The Electron Desktop App
- Start The Web IDE From Source (:3200)
- Gateway / Headless Runtime (:4310)
- ...and more pages

## 3. PART III: INTERFACE TOUR

- Overall Layout
- Left File Tree And Workflow Picker
- Top Header And Workspace Switcher
- Workflow Editor (DAG Canvas)
- ...and more pages

## 4. PART IV: WORKFLOW AND DAG BASICS

- Workflow Spec 4.1 Overview
- Nodes And Directed Edges
- LLM Nodes
- Tool Nodes
- ...and more pages

# Roadmap II

## 5. PART V: AUTHORING WORKFLOWS ON THE CANVAS

- Create A New Workflow
- Place And Connect Nodes
- Multi-Select, Delete, And Multi-Flow Tabs
- Open The Node Detail Drawer
- ...and more pages

## 6. PART VI: GENERATING AGENT APPS (THE CORE CAPABILITY)

- How VL Agent Generates Agent Apps
- One-Line Quick Generate (generate-agent-app-quick)
- Guided, Human-In-The-Loop Generation (generate-agent-app-guided)
- Enterprise Quality-Gated Generation (generate-agent-app-cascade)
- ...and more pages

## 7. PART VII: RUN, DEBUG, AND THE MASTER AGENT

- Run A Workflow With One Click
- Watch Live Events In Detail Log
- Pause, Resume, And Graceful Pause
- Control A Run In Natural Language (DAG Operator)
- ...and more pages

## 8. PART VIII: .VFLOW, RESOURCE CENTER, AND SHIPPING TO VLCODE

- The .vflow Bundle Format
- Save And Save-As A .vflow
- Register To The Local Resource Center
- The Local Registry Layout
- ...and more pages

# Roadmap III

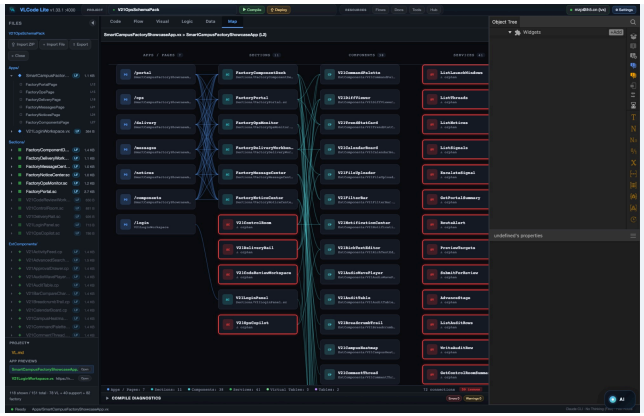
## 9. PART IX: COMPLETE CASE STUDIES

- Case 1: Your First Workflow
- Case 1: Run And Inspect
- Case 2: Generate An Approval Agent App From One Line
- Case 2: From Intent To A Running App
- ...and more pages

## 10. PART X: APPENDICES AND QUICK REFERENCE

- Port Cheat Sheet
- CLI Command Cheat Sheet
- Node Type Cheat Sheet
- Slash Command Cheat Sheet
- ...and more pages

# Core Glossary



Map mode for application structure

## Agent OS

A resident local Agent operating system: author workflows, run Agent Apps, maintain .vflow.

## Workflow / DAG

A Workflow Spec 4.1 directed acyclic graph; nodes are steps, edges are order/dependency.

## Node types

LLM, Tool, Set, WriteFile, Pause, Review, VisualLogic, Formula, Actor, Loop, and more.

## Agent App

One .vflow file is one app: an entry workflow plus an app-block manifest.

## Unit / DAG Map

The Unit Map is the human-readable logic layout; the DAG Map is the compiled execution graph.

## Interaction cards

A Pause node renders a human input form; a Review node renders an approval decision card.

## Master Agent

A Planner/Evaluator/Scheduler/Arbiter loop that scores, prunes, and promotes

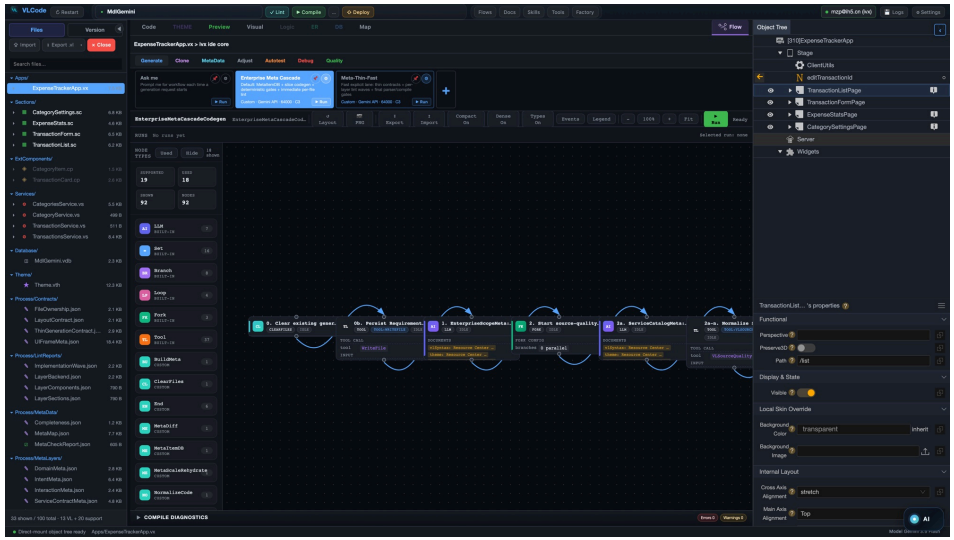
## DAG Operator

Control a run in natural language: query status/node/variable,

CHAPTER

# Part I: What VL Agent Is

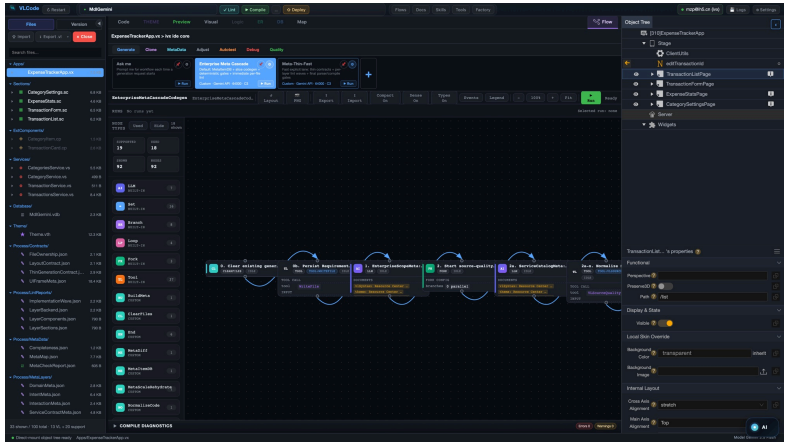
- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



VL Agent shell — the DAG canvas with the floating AI Assistant

PART I: WHAT VL AGENT IS

# What VL Agent Is



VL Agent shell — the DAG canvas with the floating AI Assistant

## CORE IDEA

VL Agent is a local Agent OS plus Agent App Studio (formerly AAS), and the first public local Electron package. Its headline capability is generating a runnable Agent App straight from a natural-language intent — the output is a manifest plus an entry workflow (a .vflow), not a hand-written script. It also generates the .vflow workflows that ship to VLCode, making it the authoring end of VL's two-product ecosystem connected by .vflow.

## OPERATIONS

1. Open the VL Agent desktop app (download the .dmg/.exe/AppImage from releases.visuallogic.ai for one-click launch; for source dev run npm start for the Web IDE on :3200) to reach the main shell
2. In the floating AI Chat panel, describe the app you want in plain language (e.g. a one-line approval app)
3. In Flow Mode pick generate-agent-app-quick or generate-agent-app-guided and run the generation
4. Inspect the generated entry-workflow DAG on the center canvas and the derived manifest in the Manifest Inspector
5. Click Run once to test the newly generated Agent App

## EXAMPLE / EXERCISE

One Chat line is enough to generate a runnable app:

```
Generate an employee leave-approval Agent App: submit request -> manager Review approve/reject -> notify requester
```

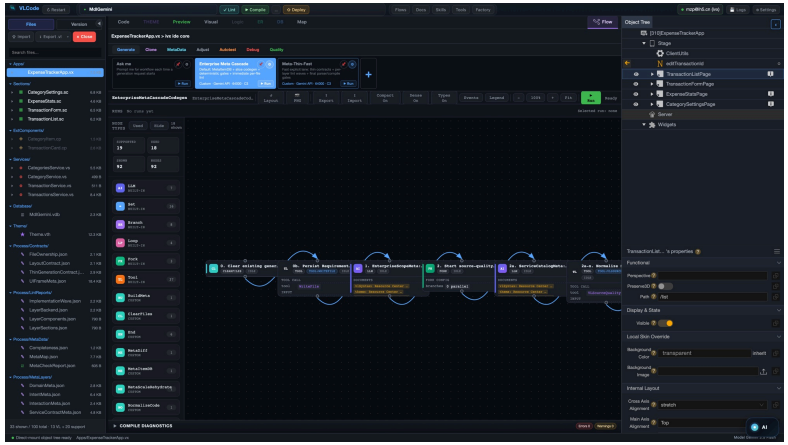
VL Agent runs generate-agent-app-quick, turning that intent in a single LLM pass into a manifest + entry workflow (with a Review decision node), deterministically normalizes the manifest, and then you Run it directly.

## PITFALL

Treating VL Agent as a generic chat assistant that just emits code snippets is the most common misread. Its core is generating runnable Agent Apps and workflows, so phrase requests as a complete app goal (inputs, processing, human gates, outputs) rather than scattered questions.

PART I: WHAT VL AGENT IS

# What A Local Agent OS Means



VL Agent shell — the DAG canvas with the floating AI Assistant

## CORE IDEA

A "local Agent OS" means VL Agent runs the workflow engine, control plane, resource center, and Agent App runtime on your own machine rather than a hosted cloud. Each workspace gets its own .vl-code/control-plane.sqlite holding the run ledger, node results, review decisions, and artifact lineage. The cloud is an optional delivery target, not the only surface, and secrets are never written to committed project files.

## OPERATIONS

1. Launch VL Agent locally via desktop, the Web IDE (:3200), or the Gateway/headless runtime (:4310)
2. Confirm each workspace root contains a .vl-code/control-plane.sqlite control-plane database
3. Configure a local LLM provider and keys in Settings (the SecretRoot paste-in vault keeps keys out of source)
4. When running a workflow, watch the locally-ledgered event stream in Detail Log
5. On switching workspaces, confirm the control-plane DB reloads (the --isolated flag gives a per-port DB)

## EXAMPLE / EXERCISE

Inspect the current workspace's control plane and local registry:

```
ls -la .vl-code/
# control-plane.sqlite        <- run ledger / node results / reviews / artifact lineage
# registry/workflows  apps  viewkits   <- local Resource Center registry
```

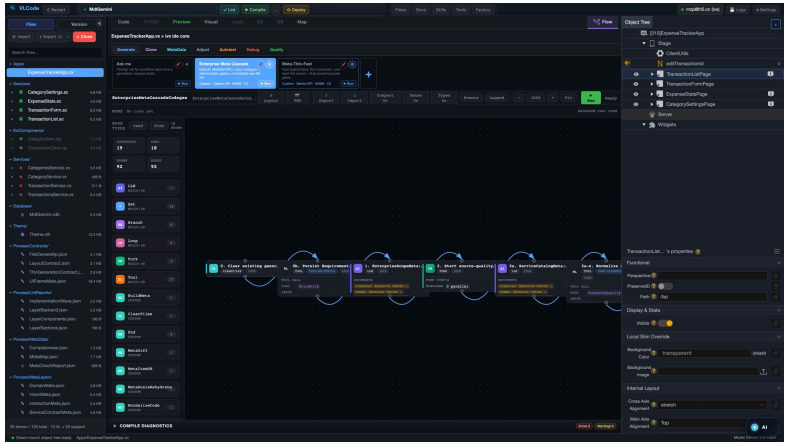
All run history, error stacks, and human decisions land in local SQLite with no cloud dependency.

## PITFALL

Assuming you must be online or signed into the cloud to run anything, then stalling when the Resource Center is slow. VL Agent bundles seed workflows and docs so generation works offline; cloud sync is an optional enhancement, not a prerequisite.

PART I: WHAT VL AGENT IS

# Workflows, DAGs, And Nodes



Workflow Editor — nodes and directed edges on the DAG canvas

## CORE IDEA

In VL Agent a workflow is a directed acyclic graph (DAG): nodes are steps, directed edges are data and execution order. The engine is Workflow-Engine 4.13.1 implementing Workflow Spec 4.1, with event schema vl.workflow.run-event.v4. The node vocabulary is rich — LLM, Tool, Set, Write/Download/Unzip, Pause, Review, Actor, Swarm, Loop/Iteration and more — composed into a runnable, checkpointable, resumable process.

## OPERATIONS

1. Open the Workflow Editor (DAG node canvas) on the center canvas
2. Drag nodes (e.g. LLM, Tool, Pause, Review) from the node drawer onto the canvas
3. Drag directed edges between nodes to define data flow and execution order
4. Fill each node's I/O contract and variable references so upstream outputs feed downstream inputs
5. Click Run and watch each node ledger to Detail Log under vl.workflow.run-event.v4

## EXAMPLE / EXERCISE

A minimal three-node DAG:

```
N1 LLM       -- parse user intent into structured fields
   |
N2 Review    -- human approval node (approve / reject / request_changes)
   |
N3 Write     -- persist the approved result to disk
```

The engine executes node-by-node with native checkpoint / resume / rerun-from-step; graceful pause survives even a closed socket.

## PITFALL

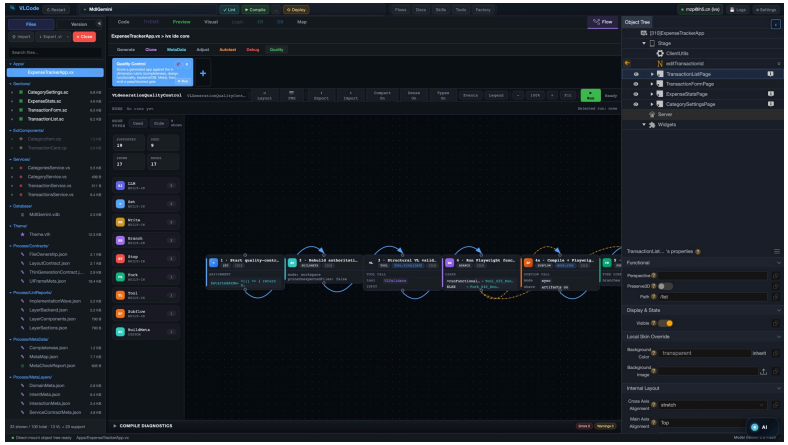
Wiring nodes into a cycle or leaving a dangling edge makes runs hang or behave unpredictably. After wiring, use structural validation to catch dangling edges and confirm the graph is acyclic with both ends of every edge connected.



## CHECKPOINT

PART I: WHAT VL AGENT IS

# What An Agent App Is



Agent App Studio — Unit Map over the compiled DAG Map

## CORE IDEA

An Agent App is the target artifact VL Agent generates and runs: defined by Agent App Studio (AAS, spec v2.2.0), its model is a single .vflow file — the app pack is "entry workflow + a top-level app block", with NO separate .app.json. The manifest is derived at load time from the app block plus Pause/Review steps. It has an install -> release -> runtime lifecycle and an instance-scoped SQLite control plane separate from the IDE workspace.

## OPERATIONS

1. Produce an Agent App from intent with a generation workflow (e.g. generate-agent-app-quick)
2. Inspect the three-layer editor: Unit editor (I/O contracts + manifest assets), Unit Map (human-readable logic), DAG Map (compiled execution graph)
3. Open the Manifest Inspector to see planned vs actual surfaces, lifecycle dots, and resource lineage derived from the app block
4. Drive the instance from the App chat console with /start /pause /resume /stop /status /rerun-step
5. When a Pause/Review node fires, fill the form or make the approval in the interaction card

## EXAMPLE / EXERCISE

An Agent App's .vflow top-level shape (illustrative):

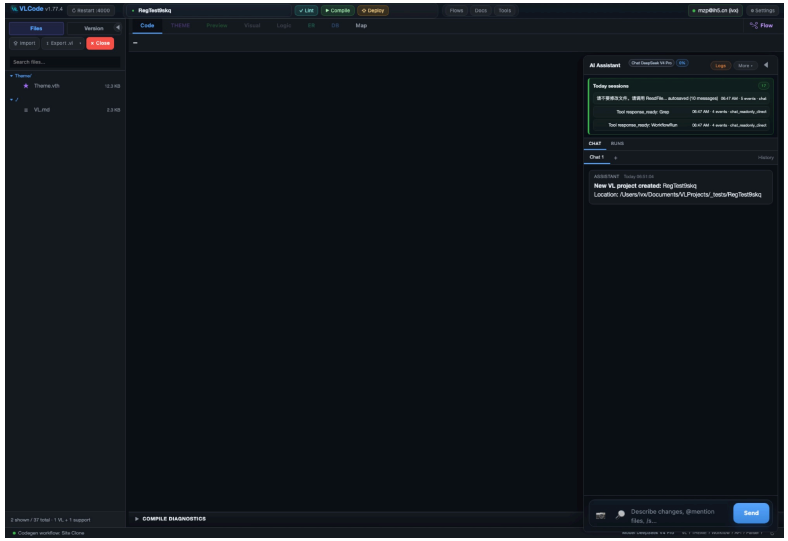
```
app:                    <- top-level app block (manifest derives from here)
  name: LeaveApproval
  entry: approval-flow     <- entry workflow
workflow approval-flow:
  N1 Pause    requester fills a form (text/textarea/select)
  N2 Review   manager approval card (approve/reject/request_changes)
  N3 Write    persist
```

At load time the manifest derives from the app block + Pause/Review, and App Home shows a workflow-first DAG + manifest + core data + artifacts + run ledger.

## PITFALL

Trying to add a separate .app.json manifest file for an Agent App — spec v2.2.0 is a single-.vflow model and the manifest is derived at load time, so a hand-written manifest conflicts

# .vflow: The Single App File



Flow Bundle Mode — editing a .vflow directly

## CORE IDEA

.vflow is the connector of the VL ecosystem and the single app file for an Agent App. A .vflow bundle = workflow JSON + manifest + optional subflows/docs/prompts — a self-contained bundle (with optional embedded VL surfaces and config docs). In VL Agent you can save any workflow as a .vflow, and in Flow Bundle Mode you can open, edit, and save a .vflow directly. It is exactly the file passed between VL Agent and VLCode.

## OPERATIONS

1. Author the workflow and app block in the Workflow Editor or Agent App Studio
2. Enter Flow Bundle Mode to open/edit/save the .vflow directly
3. Or use Save as .vflow to export the current workflow as a self-contained bundle
4. Register the .vflow to the local Resource Center (.vl-code/registry/workflows/<id>.json)
5. Via Resource Center cloud sync, let a VLCode developer browse, import, and customize the same .vflow

## EXAMPLE / EXERCISE

The shape of a .vflow bundle:

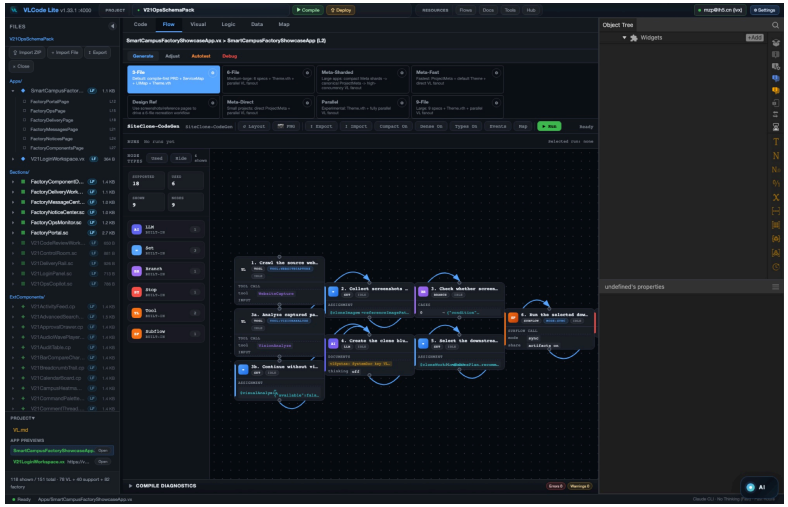
```
myapp.vflow
|- workflow JSON   <- nodes + edges + variables
|- manifest        <- app block / surfaces / lifecycle
|- subflows/       <- optional subflows
|- docs/           <- optional config docs
|- prompts/        <- optional seed prompts
```

On the VLCode side, a developer drags the same .vflow into the Flow panel and picks a category (Generate/Enterprise/Clone/MetaData/Test/Adjust/Autofix/Quality) to reuse it.

## PITFALL

Assuming a .vflow is just bare workflow JSON, then finding the manifest or docs missing after importing into VLCode. Always save in the .vflow bundle format (with the manifest and optional docs/prompts) to keep it self-contained and portable.

# The Two-Product Model With VLCode



Resource Center — registering and syncing a .vflow

## CORE IDEA

VL is two independently-sold products in one ecosystem: VL Agent (local Agent OS + Agent App Studio) authors workflows and Agent Apps, while VLCode (the local-first IDE, the base for building VL enterprise apps) generates and repairs VL enterprise apps fast. The connector is the .vflow file: VL Agent ships .vflow to VLCode through the Resource Center. Both use one unified visuallogic.ai account with same-site download.

## OPERATIONS

1. In VL Agent, generate a new reusable workflow from intent with workflow-factory-main
2. Save the generated workflow as a .vflow and register it to the local Resource Center
3. Trigger Resource Center cloud sync so the workflow becomes visible to VLCode
4. A VLCode developer imports the same .vflow via the Flow panel's + Upload .vflow or drag-drop
5. In VLCode they pick a category, customize, and publish to teams to drive enterprise app generation

## EXAMPLE / EXERCISE

The full chain in one line:

```
Generate in VL Agent -> Resource Center -> import in VLCode
```

For example, a team needs a standardized "add-service" generation flow: in VL Agent, use workflow-factory-main to generate its .vflow, register and cloud-sync it; a VLCode developer imports it and uses that same workflow to add services to enterprise projects.

## PITFALL

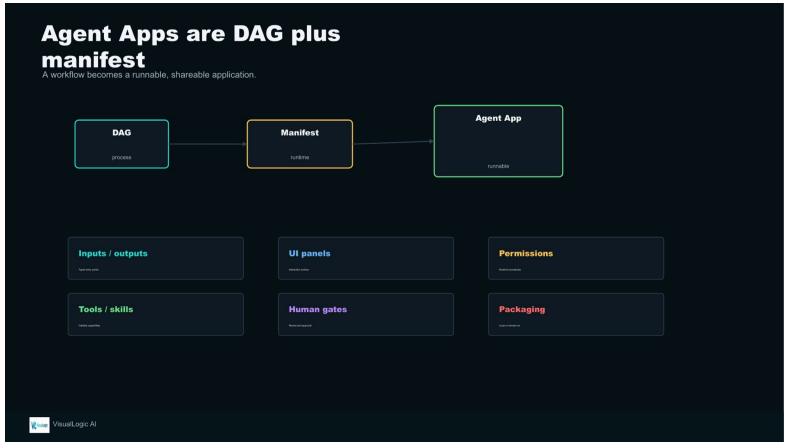
Conflating VL Agent and VLCode as one tool, or trying to deploy an Agent App directly as a VL enterprise app. Their roles differ: VL Agent authors .vflow, VLCode consumes .vflow to generate/repair enterprise apps; the only connection is the .vflow.

## CHECKPOINT

You can draw the "VL Agent generate -> Resource Center -> VLCode" .vflow shipping chain and state each side's role.

PART I: WHAT VL AGENT IS

# Master Agent Runtime Overview




Master Agent — branch scoring, pruning, and promotion

## CORE IDEA

The Master Agent runtime is how VL Agent handles complex, exploration-heavy tasks: a Planner / Evaluator / Scheduler / Arbiter loop. It generates and scores candidate branches, prunes the weak ones, preserves versions, and promotes the best mainline. Pruned/expanded/promoted states are shown directly on the DAG, and goal-convergence demos let you watch it close in on a target.

## OPERATIONS

1. Pick a workflow suited to exploratory solving (with complex nodes like Score/Grow/Loop/Iteration)
2. On running, observe the Master Agent's Planner propose candidate branches and the Evaluator score them
3. On the DAG, watch the Scheduler dispatch execution and the Arbiter decide to prune or promote
4. Inspect the pruned, expanded, and promoted node-state markers
5. Confirm the preserved versions and the finally-promoted mainline in Detail Log

## EXAMPLE / EXERCISE

In a goal-convergence demo, DAG nodes carry state markers:

```
branch A  -> score 0.42 -> pruned
branch B  -> score 0.78 -> expanded
branch B' -> score 0.91 -> promoted (becomes mainline)
```

Planner proposes branches, Evaluator scores, Scheduler dispatches, Arbiter adjudicates, looping until it converges on the goal.

## PITFALL

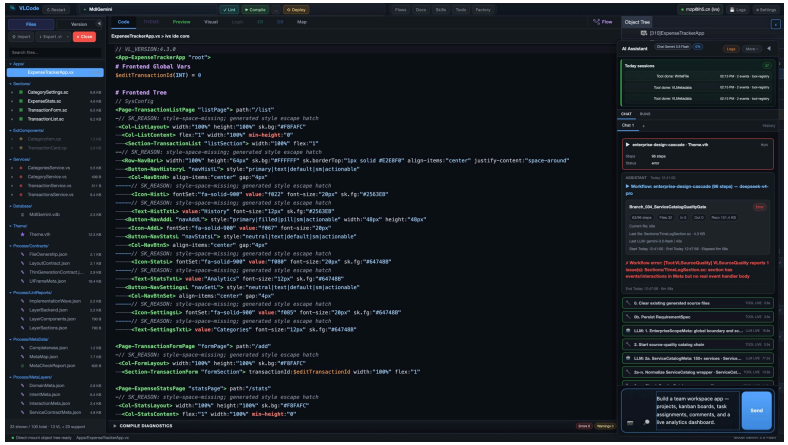
Enabling Master Agent multi-branch exploration on a simple, deterministic linear task wastes tokens and time. Branch scoring only pays off when you need to explore or compare candidate solutions; use a plain linear workflow for simple tasks.

## CHECKPOINT

You can name the Master Agent's four roles (Planner/Evaluator/Scheduler/Arbiter) and recognize pruned/expanded/promoted states on the DAG.

PART I: WHAT VL AGENT IS

# The Control Plane And Traceability



Detail Log — the live run ledger and node results

## CORE IDEA

The control plane is VL Agent's traceability foundation: one .vl-code/control-plane.sqlite per workspace recording the run ledger (history + timestamps + error stacks), node results, review resources (human decisions/audit), artifact lineage, and actor tasks. Together with the registry (.vl-code/registry/workflows|apps|viewkits), any run can be replayed, root-caused, and audited. An Agent App additionally has an instance-scoped control plane separate from the IDE workspace.

## OPERATIONS

1. After running any workflow, view the full run-event ledger in Detail Log
2. Use the CLI bin/vlflow.js subcommands ledger / state / artifacts to query the ledger and artifacts
3. Human decisions made at Review nodes land in the control plane as review resources for audit
4. On failure, read node results and error stacks from the control plane to find the root cause
5. On switching workspaces, confirm the control-plane DB reloads

## EXAMPLE / EXERCISE

Query a run's ledger and state with the vlflow CLI:

```
node bin/vlflow.js list             # list workflows
node bin/vlflow.js state   <runId>  # run state
node bin/vlflow.js ledger  <runId>  # run ledger (history/timestamps/error stacks)
node bin/vlflow.js artifacts <runId>  # output artifacts and lineage
```

Every node's result and every human approval is traceable.

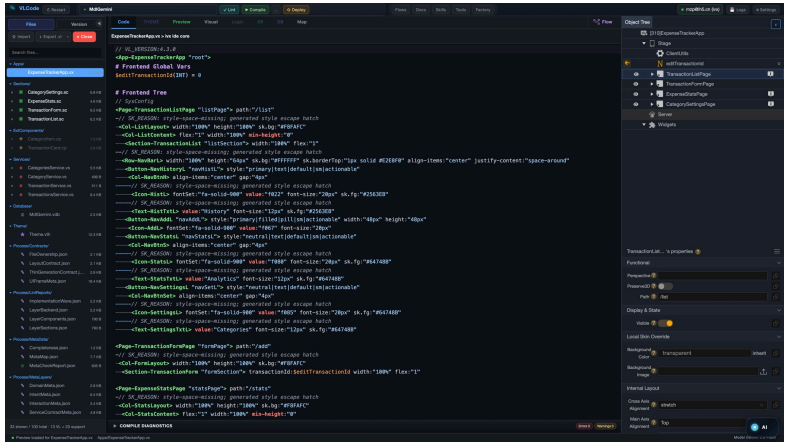
## PITFALL

On failure, reading only the final Chat summary and ignoring node results and error stacks in the control plane, leading to blind broad rewrites. The root cause is in the control-plane ledger — read it to locate the failure first, then apply a minimal fix.

## CHECKPOINT

PART I: WHAT VL AGENT IS

# Why Humans Gate At The Node Layer



A Pause/Review interaction card waiting for a human

## CORE IDEA

VL Agent puts human gating at the node layer rather than reviewing one big blob of output afterwards. A Pause node is a human-in-the-loop input gate (text/textarea/number/select/checkbox form), and a Review node is a structured decision gate (approve/reject/request_changes). This lets humans verify the AI's process at key steps, with decisions landing in the control plane as auditable review resources — making it verified, not hopeful.

## OPERATIONS

1. Place a Pause or Review node wherever the workflow needs a human decision
2. Define the Pause node's input form fields (text/textarea/number/select/checkbox)
3. Configure the Review node's approval options (approve/reject/request_changes)
4. When the run reaches that node, fill the form or make the decision in the Chat interaction card
5. Confirm the decision is recorded in the control plane as a review resource for later audit

## EXAMPLE / EXERCISE

An approval Agent App gates at the node layer:

```
N2 Review  "Approve purchase amount of $48,000?"
   [ Approve ]  [ Reject ]  [ Request changes ]
```

A human clicking Approve continues the flow; Reject branches to a stop per the graph; the decision plus its timestamp enters the control plane's review resources. Graceful pause survives even a closed chat socket, so you can return and finish the approval.

## PITFALL

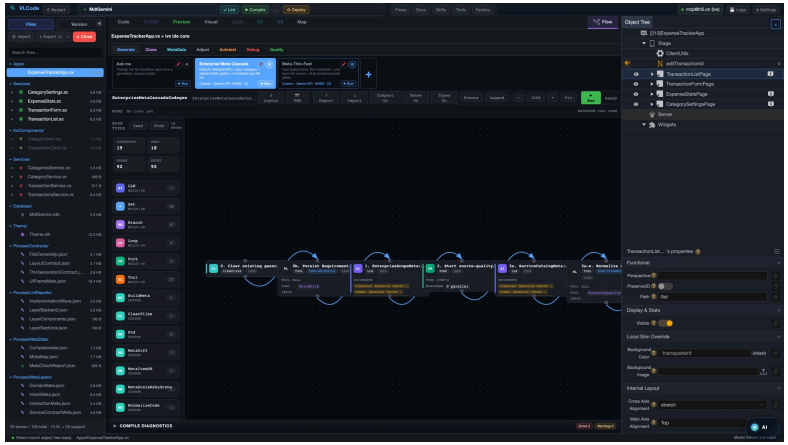
Piling all human review at the very end, only to discover after generating a pile of artifacts that the direction was wrong — wasting tokens and obscuring the cause. Insert Pause/Review at the key decision points so humans intercept early at the node layer.

## CHECKPOINT

You can insert a Review node into a workflow, trigger the approval card on a run, and see the human decision recorded in the control plane.

PART I: WHAT VL AGENT IS

# Local-First, Cloud Optional



VL Agent shell — the DAG canvas with the floating AI Assistant

## CORE IDEA

Local-first is VL Agent's core principle: the workflow engine, control plane, and Agent App runtime all run on your machine, and the cloud is an optional delivery target, not the only surface. Generation, running, and debugging don't require a network; when the Resource Center is slow or offline, the bundled seed workflows and docs still work. Secrets are never written to committed project files — they live in Settings or the SecretRoot vault.

## OPERATIONS

1. Launch locally via desktop, Web IDE (:3200), or Gateway (:4310) — all running on your machine
2. Configure an LLM provider in Settings, with keys in the SecretRoot paste-in vault (masked inventory)
3. When offline or the Resource Center is slow, run generation against the bundled local seed workflows
4. Only when you want to share/ship, register the .vflow to the Resource Center and trigger cloud sync
5. When cloud delivery is needed, use the Header's Cloud Compile/Deploy as an optional target

## EXAMPLE / EXERCISE

Confirm it runs without a network — inspect the bundled registry and control plane:

```
ls .vl-code/registry/workflows   <- local seed workflows, usable offline
ls .vl-code/control-plane.sqlite <- local run ledger
```

The cloud (Resource Center sync, Cloud Deploy) is an optional enhancement; with no network you can still generate an Agent App and run it.

## PITFALL

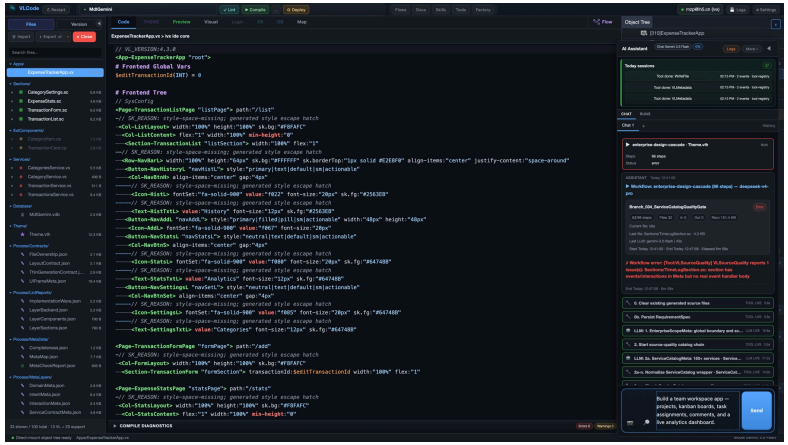
Treating cloud sync as a mandatory step and assuming things are "broken" when offline or when the Resource Center is slow. The cloud is an optional delivery target and the local path is complete; also never write secrets into project files for convenience — always use Settings/SecretRoot.

## CHECKPOINT

You can generate and run a workflow offline using the bundled local seed workflows, proving the cloud is optional.

PART I: WHAT VL AGENT IS

# Verified, Not Hopeful



Detail Log — the live run ledger and node results

## CORE IDEA

"Verified, not hopeful" is VL's root-cause-first principle inside VL Agent: never write fallback, error-swallowing, silent-fallback, or fake-success code. Run states settle reliably to done/error/aborted (terminal-state integrity), and errors surface to the caller/ledger/user instead of being quietly swallowed. With the control-plane ledger and node results, you can verify with evidence that a run truly succeeded — not just that it looks like it did.

## OPERATIONS

1. After running a workflow, confirm the run state settles to one of done/error/aborted
2. Verify node-by-node results in Detail Log or vlflow ledger, not just the Chat summary
3. On failure, read the error stack in the control plane to find the root cause before any broad rewrite
4. When the same error repeats, return to source + manifest for the root cause instead of trial-and-error
5. After a fix, re-run and verify the terminal state is done in the ledger before calling it complete

## EXAMPLE / EXERCISE

Terminal-state integrity means runs never get stuck in an "in-between" state:

```
run-event ... state: running
run-event ... state: error    <- the real error surfaces, with a stack
# rather than quietly returning an empty result and faking success
```

Root-cause-first: capture the real error, inspect the call path and data, fix the root cause, then change code — a fix counts only when you can explain the cause, the change, and the verification.

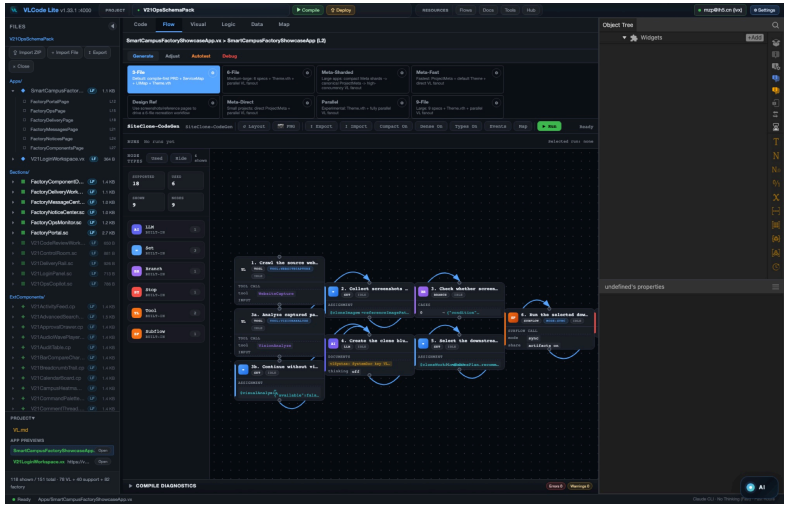
## PITFALL

Adding fallbacks, swallowing errors, or returning default values just to "look done", which masks the real error and makes future debugging harder. Errors must surface to the caller/ledger/user; never catch-and-only-log or fake success unless it is an explicit, documented product requirement.



## CHECKPOINT

# When To Reach For VL Agent



Flow Mode — workflow picker, codegen, autotest

## CORE IDEA

Reach for VL Agent when you want "a process that runs" — multi-step, combining tools/LLM/human gates, replayable and auditable. Its sweet spot is generating a runnable Agent App from a single intent, authoring reusable workflows, and generating the .vflow that ships to VLCode. If you just want a VL enterprise app (pages/services/tables), use VLCode; the two connect through .vflow.

## OPERATIONS

1. Decide whether the task is "a process": multi-step + tools/LLM + possibly human decisions
2. For a ready-to-run app, generate an Agent App in Flow Mode with generate-agent-app-quick/guided/cascade
3. For a team-reusable workflow, generate a .vflow with workflow-factory-main
4. If it's just an enterprise web app (.vx/.sc/.vs/.vdb), switch to VLCode
5. Ship VL Agent's output to VLCode for reuse via .vflow

## EXAMPLE / EXERCISE

Deciding which tool:

```
"Build a CustomerCRM intake Agent App: capture leads -> Review qualification -> write to DB"
   -> this is a process, use VL Agent: generate-agent-app-guided

"Build an enterprise CRM website with 8 pages and 20 service methods"
   -> this is an enterprise web app, use VLCode
```

The connector between them is always the .vflow.

## PITFALL

Using VL Agent to directly emit page source for a plain enterprise website, or conversely hand-building a multi-step human-gated process inside VLCode. Picking the wrong tool means the long way around — split by "process vs app", then bridge with .vflow.

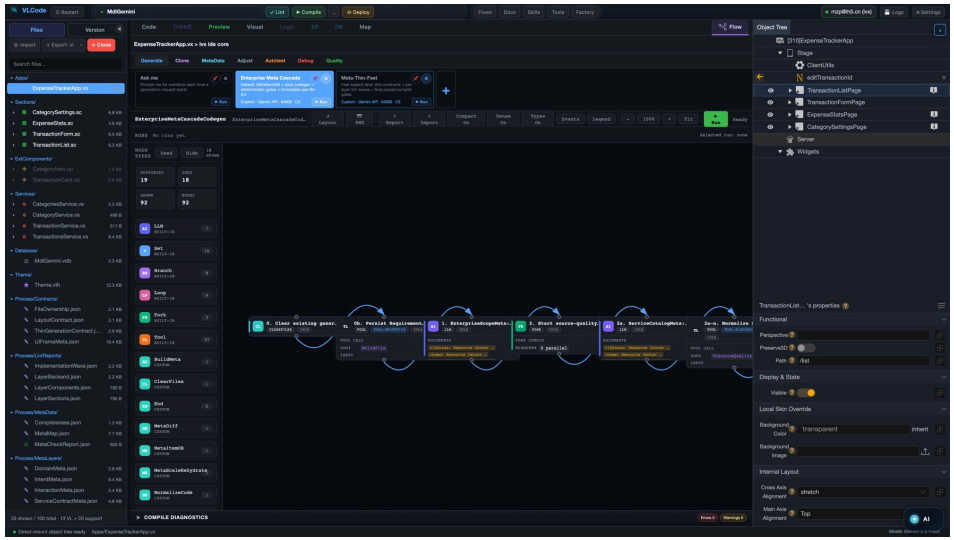


## CHECKPOINT

CHAPTER

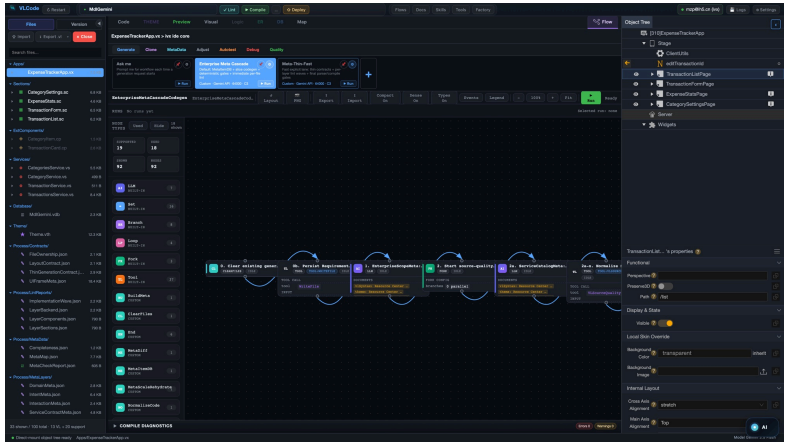
# Part II: Installation, Startup, And Workspace

- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



VL Agent shell — the DAG canvas with the floating AI Assistant

# Choose Desktop, Browser, Or Gateway



VL Agent shell — the DAG canvas with the floating AI Assistant

## CORE IDEA

VL Agent ships in three launch forms for three audiences: the Electron desktop app (one-click, bundled Node runtime, no terminal) for everyday authoring and running; the Web IDE from source on ports 3200-3209 for developers who want to be close to the code; and the gateway/headless runtime on ports 4310-4319 for remote access and automation with no full UI. All three share the same workflow engine and control plane — they differ only in entry point and interface density.

## OPERATIONS

1. Decide whether you are an end user or a developer: to visually build DAGs and Agent Apps, take the desktop app or Web IDE
2. For zero-install, no-terminal, fastest start, choose the Electron desktop app (.dmg / .exe / AppImage)
3. To stay close to the source or debug on port 3200, run npm start from source for the Web IDE on :3200
4. To be driven by CI, scripts, or a remote machine with no UI, run npm run start:gateway for the :4310 gateway
5. Confirm the target machine's port is free before launching

## EXAMPLE / EXERCISE

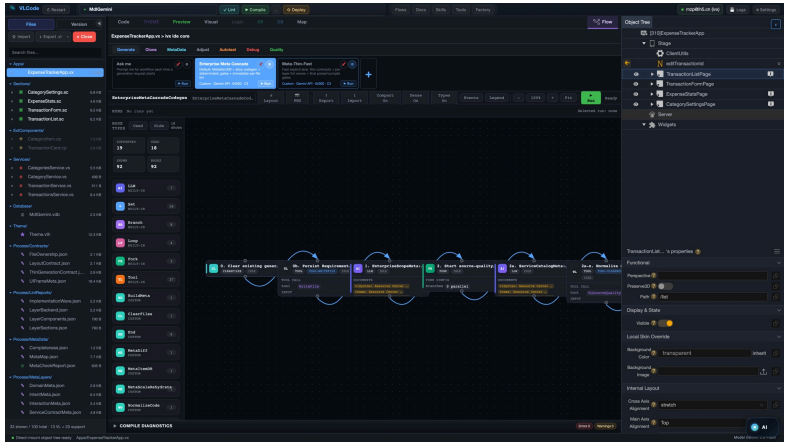
A typical team split: a product manager authors an approval workflow in the Electron desktop app on a Mac; a backend engineer edits node logic in the source Web IDE locally; a CI server runs only the gateway to execute a published .vflow on schedule:

```
# Developer: Web IDE from source
npm start                # Web IDE on :3200 (range 3200–3209)
# Automation server: headless gateway
npm run start:gateway    # listens on :4310
```

## PITFALL

Treating the gateway like a full IDE — it is a headless automation entry with no DAG canvas and no Agent App Studio. To visually author workflows you must use the desktop app or Web IDE.

# Install The Electron Desktop App



VL Agent shell — the DAG canvas with the floating AI Assistant

## CORE IDEA

The Electron desktop app is VL Agent's first public local package; it bundles the Node runtime so it works the moment you install it — no terminal commands needed. It includes the full Workflow Editor, Agent App Studio, and control plane, making it the lowest-friction entry for non-engineers. Installing is just like any desktop app: download the platform installer from the release page, drop it into your applications folder, and open it.

## OPERATIONS

1. Open the visuallogic.ai download/release entry in a browser (same site and unified account as VLCode)
2. Pick the installer for your OS: .dmg on macOS, .exe on Windows, AppImage on Linux
3. macOS: drag the app into Applications; Windows: run the .exe installer; Linux: make the AppImage executable and run it
4. On first open, confirm the top header shows the project title, workspace switcher, and LLM provider/model selector
5. Go to Settings and paste in one provider's API key to prepare for the first run

## EXAMPLE / EXERCISE

On macOS, verify the app is in place after installing:

```
ls /Applications | grep -i 'VL Agent'
```

Seeing the app name means the desktop install succeeded; double-click to open — no process to start from a terminal.

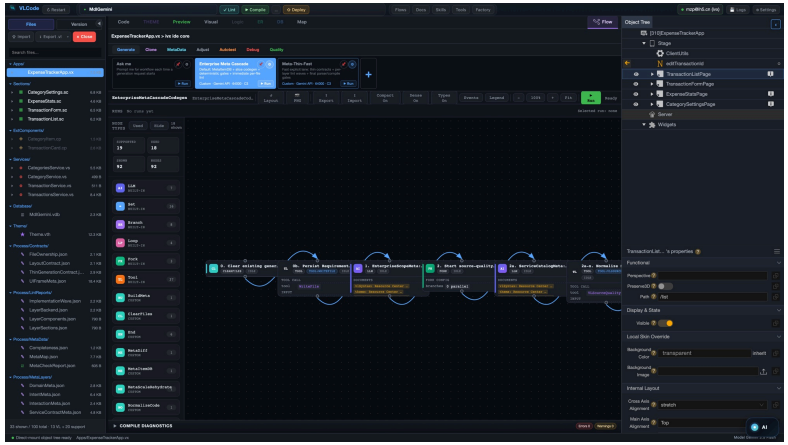
## PITFALL

On macOS, running directly from the mounted .dmg instead of dragging into Applications — the app then disappears on the next update or restart and may hit read-only-volume errors. Always copy into Applications first.



## CHECKPOINT

# Start The Web IDE From Source (:3200)



VL Agent shell — the DAG canvas with the floating AI Assistant

## CORE IDEA

The Web IDE from source is the developer entry: after cloning the repo you run npm start to launch a local server, defaulting to port 3200 (range 3200-3209). In the browser it presents the same Workflow Editor, Map/Flow/Flow Bundle modes, ASSETS/UNITS/FLOW TREE rails, and floating AI Chat as the desktop app — ideal for staying close to the source while debugging or customizing the IDE itself.

## OPERATIONS

1. In the source directory, make sure dependencies are installed
2. Run npm start to launch the Web IDE (defaults to :3200)
3. Open http://localhost:3200 in your browser
4. Confirm the left file tree + workflow picker and the center canvas tabs (Workflow Editor / Map / Flow / Flow Bundle) all render
5. Configure provider/model in Settings, then create or open a workflow on the canvas

## EXAMPLE / EXERCISE

Standard launch command and URL:

```
npm start
# open http://localhost:3200 in the browser
```

If 3200 is taken, follow port discipline and move to the next fixed port in range (e.g. PORT=3201 npm start) — never let it silently drift to a random port.

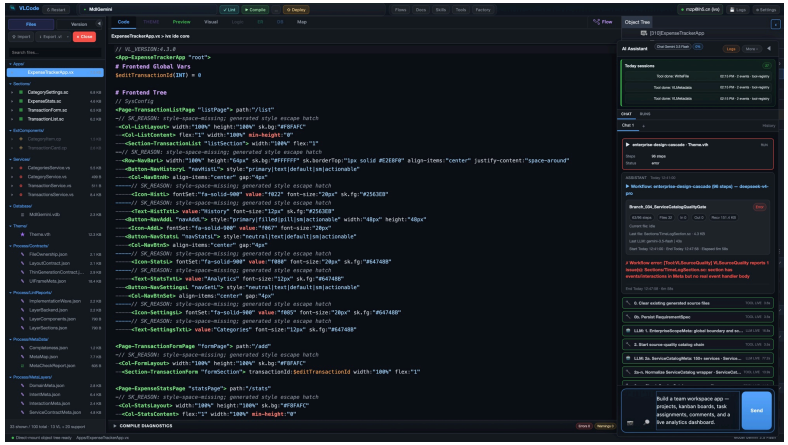
## PITFALL

Grabbing a port outside 3200 or allowing a silent fallback — this breaks fixed-port discipline and desyncs the gateway, docs, and team conventions. When the port is busy, explicitly pick the next port in range.

## CHECKPOINT

You can start the Web IDE on :3200 with one command, see the Workflow Editor and all four canvas tabs in the browser, and your port stays within 3200-3209.

# Gateway / Headless Runtime (:4310)



Detail Log — the live run ledger and node results

## CORE IDEA

The gateway/headless runtime lets VL Agent run workflows with no GUI, built for remote access and automation. You start it with npm run start:gateway, listening by default on :4310 (range 4310-4319). It suits deploying an already-authored .vflow to a server where scripts, cron, or another machine trigger execution — results still land in the control-plane ledger for later traceability.

## OPERATIONS

1. In the source directory on the server or target machine, run npm run start:gateway
2. Confirm the gateway listens on :4310 (range 4310-4319) and that the port is free
3. Place the workflows/.vflow you want to run into that workspace's registry
4. Trigger execution from a script or automation — no UI required
5. Afterward, inspect that workspace's control-plane.sqlite run ledger to verify results

## EXAMPLE / EXERCISE

Bring up the headless gateway on an automation server:

```
npm run start:gateway   # listens on :4310
```

It shares the same workflow engine (Workflow-Engine 4.13.1) and event schema vl.workflow.run-event.v4 as the desktop app/Web IDE, so gateway results match a local manual run.

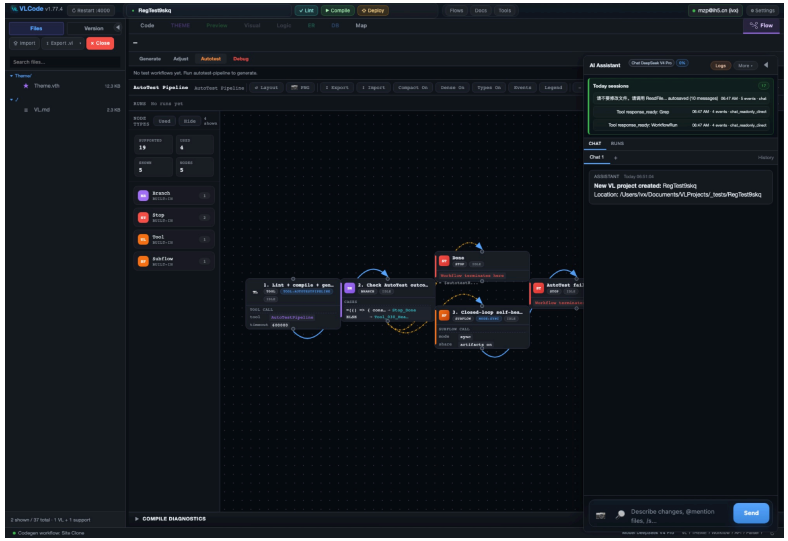
## PITFALL

Expecting a DAG canvas or interaction cards on the gateway — it is headless with no UI. Flows that need human Pause/Review gating must run in the desktop app, or rely on an external system to consume those gate events.

## CHECKPOINT

You can start the :4310 gateway with npm run start:gateway on a headless machine and trigger one workflow run whose results appear in the control-plane ledger.

# Port Convention And Fixed-Port Discipline



Bottom status bar: provider, model, ports, compile

## CORE IDEA

VL Agent assigns a fixed port range to each process class to avoid conflicts and keep team and doc conventions stable. Key ports: VL parser 3456; IDE 3200-3209; kernel 4300; gateway 4310-4319; agent-app runtime 4610-4699. Fixed-port discipline means you clear or avoid a busy port before starting and never silently fall back to a random one — this is the product's 'root-cause-first, no fallback' principle applied at the port layer.

## OPERATIONS

1. Before starting any service, check whether the target fixed port is in use
2. Memorize the split: IDE in the 3200 range, kernel 4300, gateway 4310 range, agent-app runtime 4610 range, parser 3456
3. When a port is busy, explicitly move to the next fixed port in the same range (e.g. 3200→3201), never outside the range
4. Do not enable any 'auto-pick a random port if busy' fallback behavior
5. Record the actual ports in your project/team convention and keep them consistent across machines

## EXAMPLE / EXERCISE

Check whether 3200 is free before launching:

```
lsof -i :3200   # output means it is occupied
```

If taken, follow discipline and move to the next in-range port:

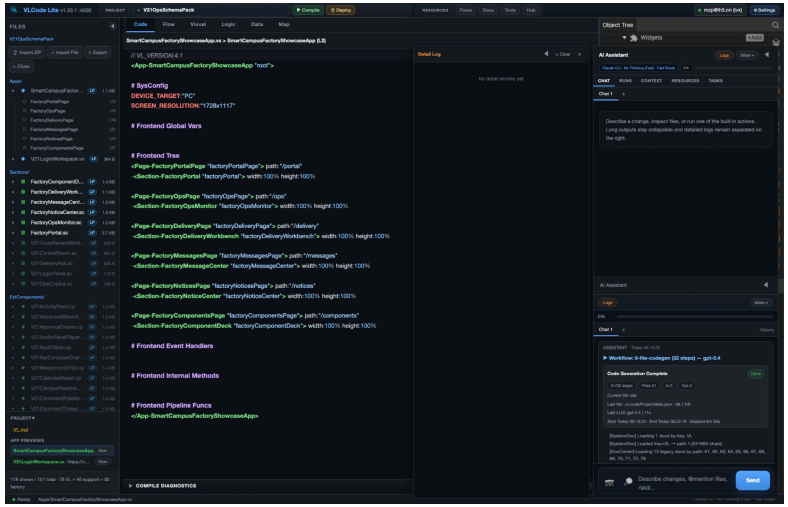
```
PORT=3201 npm start
```

rather than letting it pick an unknown port.

## PITFALL

Relying on 'auto-switch to another port if busy' logic — exactly the silent fallback the product forbids; it leaves the gateway unable to reach the IDE and teammates hitting the wrong

# Configure The LLM Provider And Model



Settings — provider, model, and runtime configuration

## CORE IDEA

VL Agent connects to large models through the top-header LLM provider/model selector and Settings; it supports Anthropic (Claude), OpenAI (GPT), DeepSeek, Kimi/Moonshot, GLM/Zhipu, Gemini, MiniMax, and Qwen. For cost reasons the defaults are DeepSeek V4 Pro and Kimi 2.5. Every LLM node and the AI Chat run on this configuration — without a key, no LLM step will run.

## OPERATIONS

1. Open Settings and pick a provider in the LLM section (e.g. anthropic / openai / deepseek / kimi / glm / gemini / minimax / qwen)
2. Paste or enter that provider's API key
3. In the top-header provider/model selector, confirm the active provider and model
4. For cost-sensitive work keep the defaults: DeepSeek V4 Pro for chat, Kimi 2.5 for workflow/fan-out
5. Save, then run a minimal LLM node to confirm the call succeeds

## EXAMPLE / EXERCISE

Cost-sensitive default configuration (Settings → LLM):

```
provider:                deepseek-api   # chat defaults to DeepSeek V4 Pro
workflowFanoutProvider:  kimi-api       # workflow + fan-out default Kimi 2.5
```

In AI Chat, send "List the workflows in the current workspace" to confirm the provider is active; the status bar shows the current LLM provider/model.

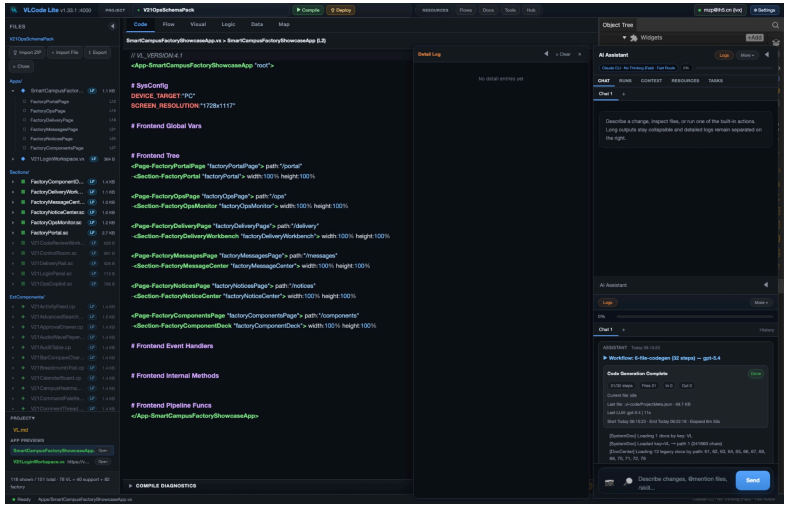
## PITFALL

Splitting chat and workflow across providers from different companies — unless you are deliberately mixing, keep the provider/company consistent to avoid hidden account, quota, and behavior differences.

## CHECKPOINT

You selected a provider and saved a key in Settings, see the active provider/model in the top selector and status bar, and a single LLM call returns successfully.

# Chat Model Versus Workflow Model



Settings — provider, model, and runtime configuration

## CORE IDEA

VL Agent lets the model used for Chat differ from the model used for Workflow runs, but the provider/company should stay consistent by default. The design reflects two different loads: Chat is interactive and wants fast responses; Workflow is batch and fan-out heavy, prioritizing stability and cost. The defaults encode exactly this split — DeepSeek V4 Pro for chat, Kimi 2.5 for workflow and fan-out.

## OPERATIONS

1. In Settings, set the chat model and the workflow/fan-out model separately
2. Keep both from the same provider/company unless you are explicitly running a mixing experiment
3. For cost sensitivity, keep the default split: chat → DeepSeek V4 Pro, workflow/fanout → Kimi 2.5
4. Confirm in the top header which model is active for chat versus workflow
5. Run a workflow with several LLM nodes and confirm it uses the workflow model, not the chat model

## EXAMPLE / EXERCISE

Recommended split configuration (Settings → LLM):

```
chat model:              DeepSeek V4 Pro
workflowFanoutProvider:  kimi-api (Kimi 2.5, runs workflow + fan-out)
allowMixedLlmProviders:  false   # default: no cross-company mixing
```

Follow-up questions in chat use the fast chat model; clicking Run on a multi-step DAG routes nodes to the workflow model automatically.

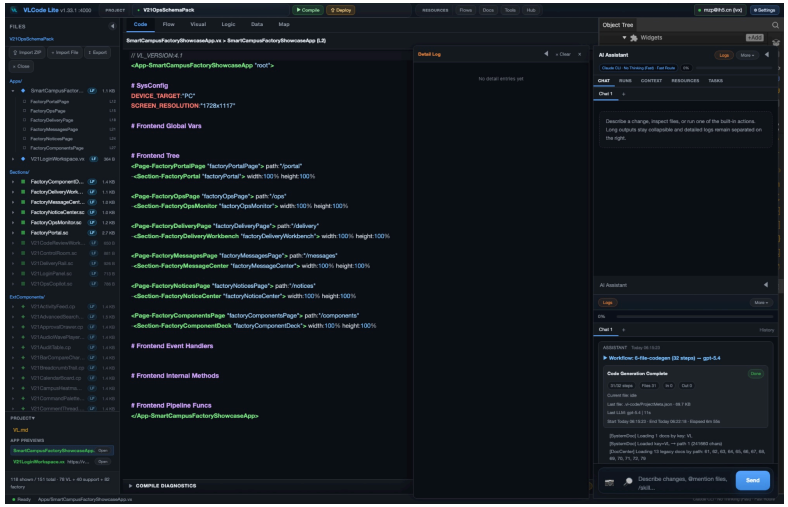
## PITFALL

Assuming the model picked in Chat is also what Workflow runs use — they are independent settings. To change batch-execution cost, change the workflow model, not just the chat model.

## CHECKPOINT

You can set chat and workflow models separately while keeping one provider, and explain why the defaults put DeepSeek V4 Pro on chat and Kimi 2.5 on workflow.

# Manage Keys With SecretRoot



Settings — provider, model, and runtime configuration

## CORE IDEA

Keys can be entered directly in Settings (shown masked) or managed centrally in SecretRoot, a paste-in vault. SecretRoot accepts pasted KEY=VALUE lines, auto-detects common prefixes (such as sk-ant-, sk-proj-, AIza...), and shows existing keys in a masked inventory. The core discipline: secrets are never written into project source or any committable file.

## OPERATIONS

1. Open Settings and find the SecretRoot vault
2. Paste multiple KEY=VALUE lines at once
3. Confirm it auto-detects prefixes and lists them masked (no plaintext echo)
4. Reference these keys from provider settings rather than hardcoding them into workflow JSON
5. Re-check the project directory to ensure no secret landed in a committable file

## EXAMPLE / EXERCISE

Format pasted into SecretRoot (prefixes auto-detected):

```
ANTHROPIC_API_KEY=sk-ant-xxxxxxxx
OPENAI_API_KEY=sk-proj-xxxxxxxx
GEMINI_API_KEY=AIzaxxxxxxxx
```

After pasting, the inventory shows only the masked tail; plaintext is never echoed.

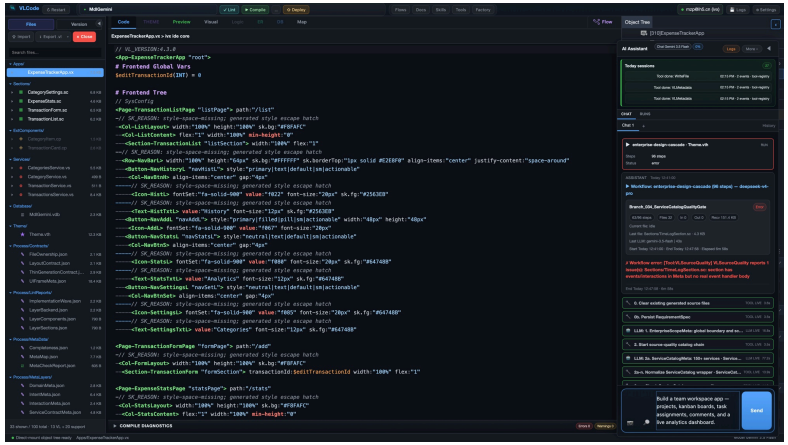
## PITFALL

Hardcoding a real API key into workflow JSON, a doc, or anything committed with the project — this leaks credentials. Always inject through SecretRoot/Settings so secrets stay in the vault, not the source.

## CHECKPOINT

You pasted several KEY=VALUE lines into SecretRoot, see them listed masked and referenced by a provider, and no plaintext secret exists in the project directory.

# Workspace And Control-Plane Database



Detail Log — the live run ledger and node results

## CORE IDEA

Each workspace has its own control-plane database, .vl-code/control-plane.sqlite, recording the run ledger (history, timestamps, error stacks), node results, review resources (human decisions and audit), artifact lineage, and actor tasks. It is the foundation of VL Agent's traceability and verifiability — every step that ran can be checked afterward, not just trusted from a 'success' line. Note that an Agent App's control plane is instance-scoped, separate from the IDE workspace.

## OPERATIONS

1. Confirm each workspace root has a .vl-code/control-plane.sqlite
2. Treat one workspace's data as one control plane: run ledger, node results, review records, and artifact lineage all live there
3. After a workflow finishes, check each node's status and timestamps in the run ledger
4. For audit, inspect the human decisions in the review resources table
5. Note that the registry (.vl-code/registry/workflows | apps | viewkits) and the control-plane DB are different resource classes — don't conflate them

## EXAMPLE / EXERCISE

Key control-plane-related paths inside a workspace:

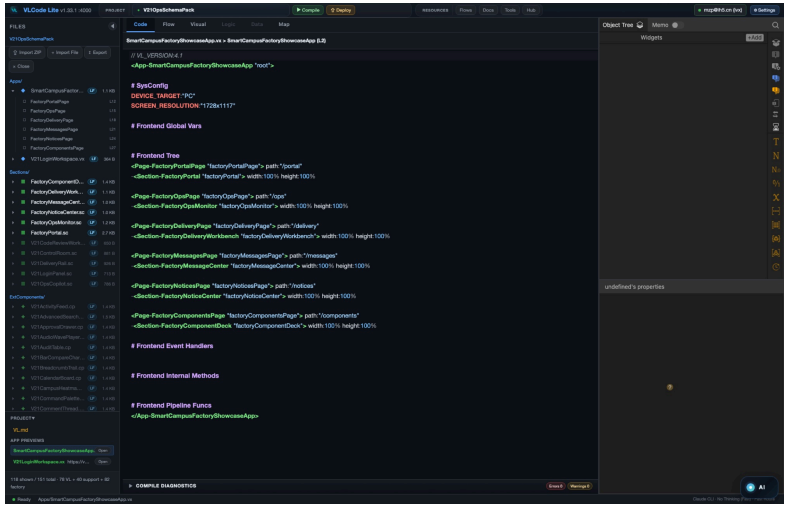
```
.vl-code/control-plane.sqlite      # run ledger / node results / reviews / lineage / actor tasks
.vl-code/registry/workflows/       # registered workflows
.vl-code/registry/apps/            # registered Agent Apps
.vl-code/registry/viewkits/        # ViewKit components
```

Launching with --isolated gives a per-port database.

## PITFALL

Cramming several projects into one workspace sharing a single control plane — run ledgers and lineage get tangled and traceability suffers. One project per workspace keeps control-plane boundaries clean.

# Switch Workspaces



Left file tree and workflow picker

## CORE IDEA

Switching workspaces is a core VL Agent operation: the top header has a workspace switcher, and switching reloads that workspace's control-plane database. That means the run ledger, registry, and node results all follow the workspace — move to another project and you see its own history and resources, with no cross-contamination.

## OPERATIONS

1. Find the workspace switcher in the top header
2. Select the target workspace to switch to
3. Confirm the control-plane database reloaded — the left file tree and ASSETS/UNITS/FLOW TREE rails update accordingly
4. Verify the run ledger now shows the new workspace's history, not the old one's
5. When you need a per-port isolated DB, launch with --isolated

## EXAMPLE / EXERCISE

Confirm the active workspace's resources switched, in the floating AI Chat:

```
User: List the workflows registered in the current workspace
```

The result should be the workflows under the new workspace's .vl-code/registry/workflows/, not the previous workspace's list.

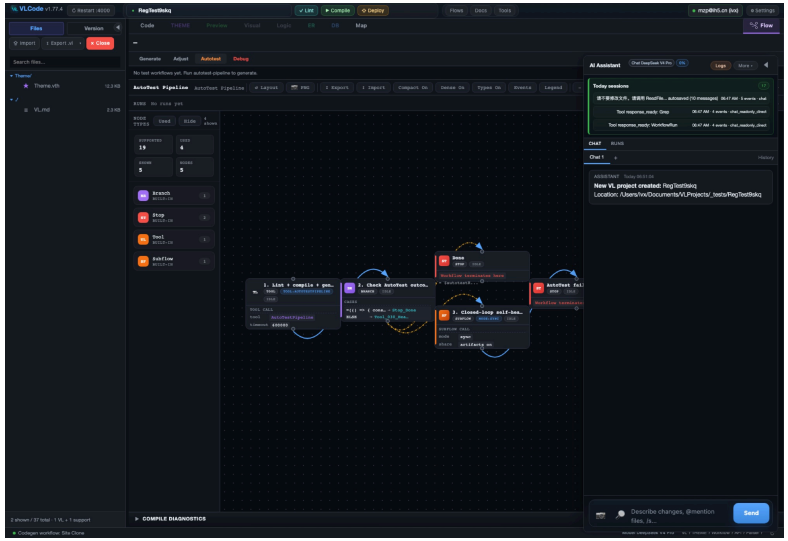
## PITFALL

Not noticing the control plane changed after switching and still thinking you're viewing the old project's run records — after switching, always confirm the run ledger and rails belong to the new workspace.

## CHECKPOINT

You can switch to another workspace via the top switcher and confirm both the control-plane DB and the rail resources reloaded to the new workspace's content.

# First-Launch Health Check



Bottom status bar: provider, model, ports, compile

## CORE IDEA

After the first launch, run a health check so every VL Agent subsystem is in place before you start authoring. The point is to confirm, one by one: the UI (header, canvas, rails, Chat, Detail Log, status bar) renders; ports sit in the correct fixed ranges; the provider/model is configured and callable; and the control-plane database exists. This catches missing configuration before you invest in real work.

## OPERATIONS

1. Confirm the top header (project title, workspace switcher, provider/model selector) and the center canvas tabs all render
2. Check the bottom status bar: current LLM provider/model, ports, and other info show correctly
3. Verify ports sit in their fixed ranges (IDE 3200-3209 / gateway 4310-4319, etc.)
4. Send a simple instruction in the floating AI Chat to confirm the provider config is live
5. Confirm .vl-code/control-plane.sqlite exists under the current workspace root

## EXAMPLE / EXERCISE

Smoke-test with a side-effect-free chat instruction:

```
User: Which workflows are registered in the current workspace?
```

A normal list confirms the provider, control plane, and registry are all in place; the status bar simultaneously shows the active provider/model.

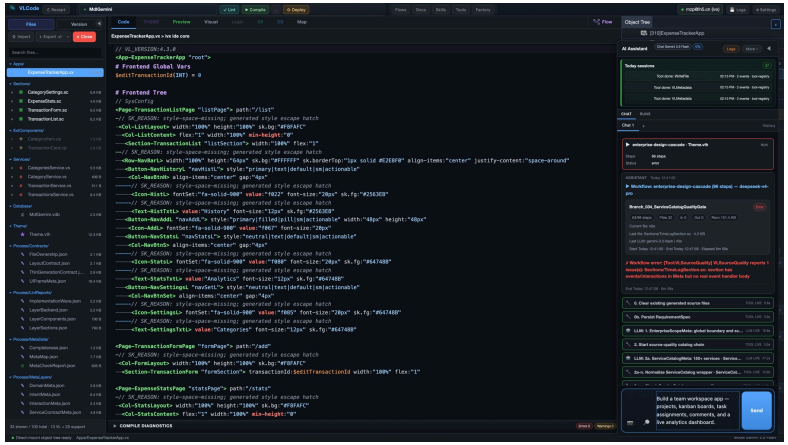
## PITFALL

Skipping the health check and immediately running a large workflow — if the provider key is missing or a port drifted, it surfaces mid-run and wastes a run. Do a lightweight smoke check before real authoring.

## CHECKPOINT

After first launch you can confirm UI rendering, port ranges, a callable provider, and the control-plane DB, and pass a single chat smoke test.

# CLI: vlflow And The Daemon



Detail Log — the live run ledger and node results

## CORE IDEA

VL Agent offers command-line entries for scripting and automation. bin/vlflow.js handles the workflow lifecycle: list/show/validate/run/resume/state/ledger/artifacts; to keep the workflow engine running as a background/headless service, use npm run start:gateway for the :4310 gateway. This CLI lets you drive, validate, and inspect workflows without opening the UI — the backbone of gateway/headless scenarios and CI.

## OPERATIONS

1. Use vlflow list to list the workflows available in the current workspace
2. Use vlflow validate to run structural validation before executing
3. Use vlflow run to trigger execution, and resume to continue from a pause/checkpoint
4. Use vlflow state / ledger / artifacts to check run status, the ledger, and produced artifacts
5. When you need a long-running background/headless service, use npm run start:gateway for the :4310 gateway

## EXAMPLE / EXERCISE

A typical scripted workflow lifecycle:

```
node bin/vlflow.js list
node bin/vlflow.js validate <workflowId>
node bin/vlflow.js run <workflowId>
node bin/vlflow.js ledger <runId>     # inspect the run ledger
```

These commands read and write the same workspace control plane, so a CLI run also appears in the UI's Detail Log and ledger.

## PITFALL

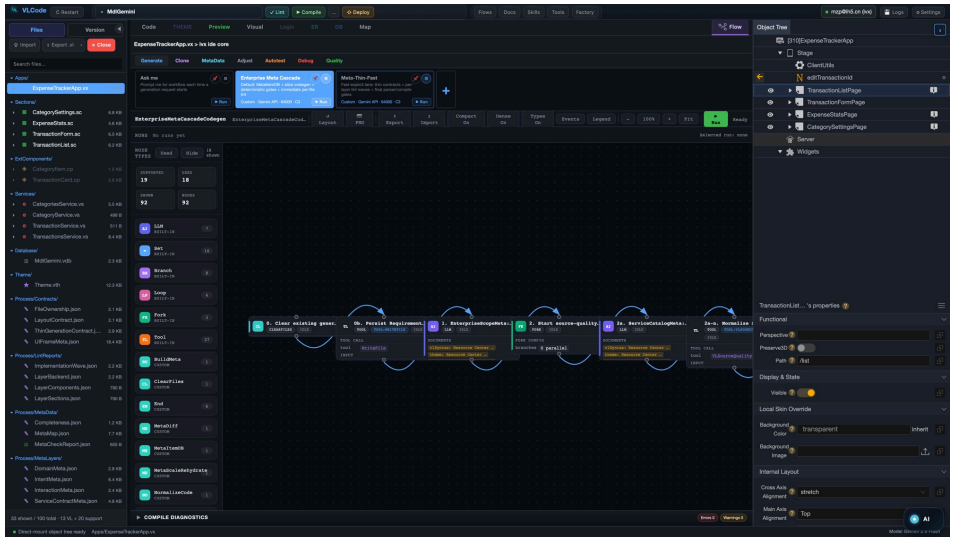
Running directly without vlflow validate — structural problems (like dangling edges) only surface mid-run. Validate before run to catch obvious structural errors at zero cost.

## CHECKPOINT

CHAPTER

# Part III: Interface Tour

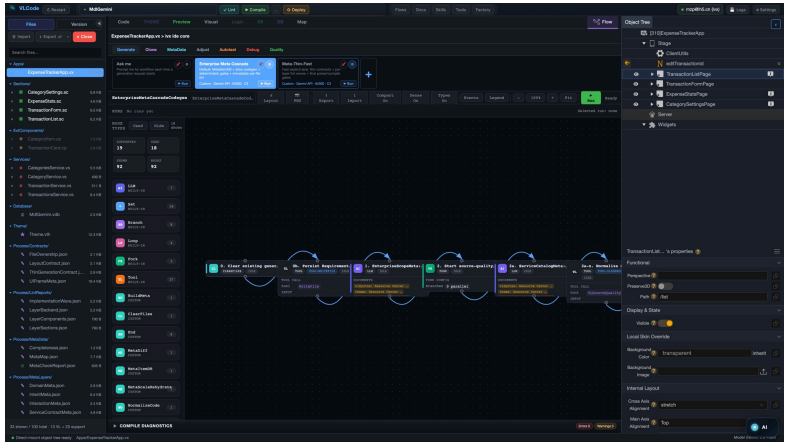
- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



VL Agent shell — the DAG canvas with the floating AI Assistant

PART III: INTERFACE TOUR

# Overall Layout



VL Agent shell — the DAG canvas with the floating AI Assistant

## CORE IDEA

The VL Agent IDE window has four zones: a left file tree + workflow picker; a top header (project title, workspace switcher, Settings, LLM provider/model selector, Cloud Compile/Deploy); a center canvas with mode tabs (Workflow Editor / Map Mode / Flow Mode / Flow Bundle Mode); and right rails (ASSETS / UNITS / FLOW TREE). The floating AI Chat panel and Detail Log overlay the canvas. Separating the picture (center DAG) from the ledger (right-side Detail Log) is exactly how the local-first, traceable design is expressed.

## OPERATIONS

1. Launch the desktop app, or run from source with `npm start` and open http://localhost:3200
2. Confirm the left side holds the file tree + workflow picker and the top is the header
3. Read the mode tabs across the top of the center canvas: Workflow Editor, Map Mode, Flow Mode, Flow Bundle Mode
4. Toggle between the three right rails: ASSETS / UNITS / FLOW TREE
5. Open the floating AI Chat panel and note it overlays the canvas rather than taking a fixed column
6. Open the right-side Detail Log and confirm run events appear only there, never duplicated in Chat

## EXAMPLE / EXERCISE

Entering a TeamPM workspace for the first time, the typical eye path is: left tree finds the `agent-app-scaffold-main` workflow -> center Workflow Editor shows its DAG -> right FLOW TREE lists the nodes -> bottom status bar confirms provider/model and ports.

```
Center tabs: [ Workflow Editor | Map Mode | Flow Mode | Flow Bundle Mode ]
Right rails:  [ ASSETS | UNITS | FLOW TREE ]
```

## PITFALL

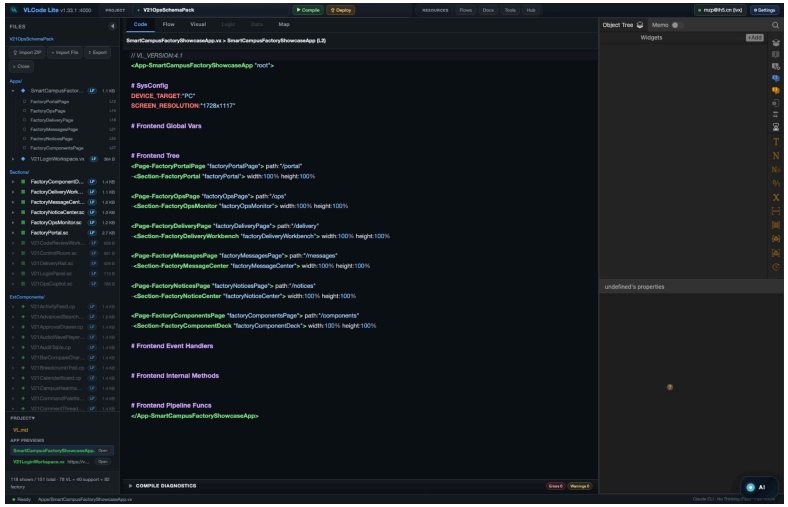
Reading the Detail Log as if it were Chat. They have different jobs: Chat carries only final summaries and human decisions; per-node progress, file writes, and tool I/O all live in Detail Log. Hunting for run details in Chat turns up nothing.



## CHECKPOINT

PART III: INTERFACE TOUR

# Left File Tree And Workflow Picker



Left file tree and workflow picker

## CORE IDEA

The left rail does two jobs at once: the file tree shows files in the current workspace (app files, .vflow bundles, and so on), while the workflow picker lists runnable workflows — VL Agent ships 150+ seed workflows (153 protected seeds; the live .vl-code/workflows registry is a superset of ~197) such as agent-app-scaffold-main, agent-app-3file-main, generate-agent-app-quick, and workflow-factory-main. Pick a workflow here and the center Workflow Editor renders its DAG. This is the "what am I going to run" entry point.

## OPERATIONS

1. Distinguish the file tree section from the workflow picker section at the top of the left rail
2. Browse the workspace directory in the file tree to find .vflow or app files
3. Search the workflow picker by name for a seed workflow, e.g. `agent-app-scaffold-main`
4. Click a workflow to select it and watch the center Workflow Editor render its DAG
5. After switching workspaces, confirm the file tree and workflow list reload alongside the control-plane DB

## EXAMPLE / EXERCISE

To scaffold an approval Agent App for a support team: search the workflow picker for `approval-flow-runtime-main`, click it, and the center canvas immediately draws that flow's DAG while FLOW TREE lists each node.

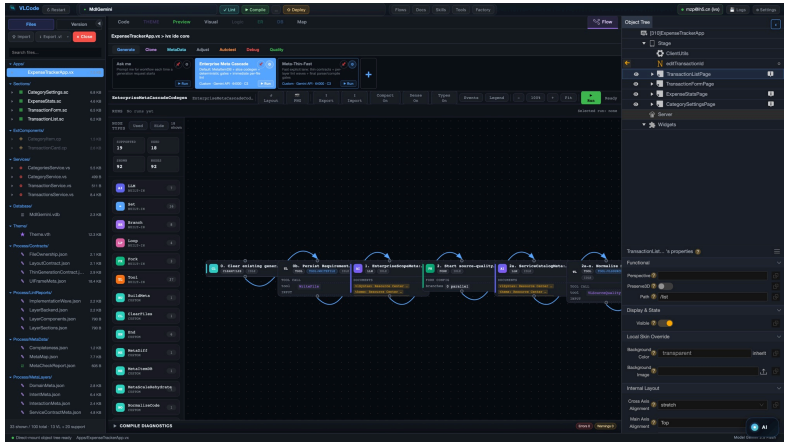
```
Workflow picker:
  agent-app-scaffold-main
  approval-flow-runtime-main   <- selected
  bpmn-approval-flow-main
  6-file-codegen
```

## PITFALL

Treating a .vflow file in the tree as the "run" entry point and double-clicking around it. Selecting and running a workflow goes through the workflow picker; editing a .vflow directly belongs in Flow Bundle Mode. Don't mix the two paths.

PART III: INTERFACE TOUR

# Top Header And Workspace Switcher



VL Agent shell — the DAG canvas with the floating AI Assistant

## CORE IDEA

The top header concentrates project-level controls: project title, workspace switcher, Settings, the LLM provider/model selector, and Cloud Compile/Deploy. The workspace switcher matters most — each VL Agent workspace has its own .vl-code/control-plane.sqlite control plane, and switching workspaces reloads that DB, so the run ledger, node results, and audit resources all swap with it.

## OPERATIONS

1. Find the header row across the top of the window
2. Use the workspace switcher to move between workspaces
3. Confirm the center canvas, file tree, and right-side Detail Log all refresh to the new workspace
4. Click the provider/model selector in the header to switch provider on the fly
5. Use the header's Cloud Compile/Deploy entry when you need to go to the cloud
6. Open Settings for global configuration

## EXAMPLE / EXERCISE

Switching from the TeamPM workspace to CustomerCRM for a demo: click the workspace switcher, pick CustomerCRM, and the Detail Log immediately clears the old ledger and loads CustomerCRM's control-plane.sqlite history.

```
Header:  [TeamPM ▾]  Settings  [Provider: deepseek-api / V4 Pro ▾]  Cloud Compile  Deploy
```

## PITFALL

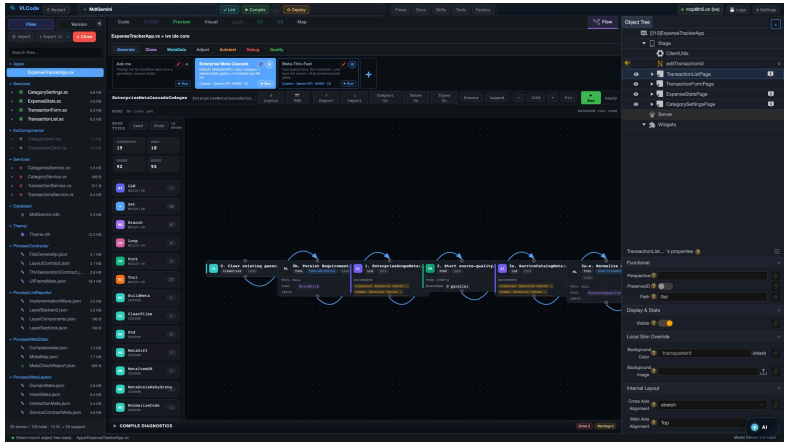
Forgetting that switching workspaces swaps the entire control-plane DB. People think their run history is "lost" when they actually switched to a different workspace that can't see the original ledger. Switch back and it reappears.

## CHECKPOINT

You can switch between two workspaces via the header's workspace switcher and see the Detail Log run ledger change accordingly.

PART III: INTERFACE TOUR

# Workflow Editor (DAG Canvas)



Workflow Editor — nodes and directed edges on the DAG canvas

## CORE IDEA

The Workflow Editor is the core mode of the center canvas: a directed acyclic graph (DAG) node canvas with multi-flow tabs and drag/drop node editing. Node types include LLM, Tool, Set, Write/Download/Unzip, Pause, Review, Actor, Swarm, and Loop/Iteration/Channel/Score. This is where you "draw" an AI execution process — turning a one-shot, point-in-time AI call into a verifiable, resumable process.

## OPERATIONS

1. Select the Workflow Editor tab at the top of the center canvas
2. Use multi-flow tabs to switch between several workflows
3. Drag/drop nodes and draw edges to build or modify the DAG
4. Click a node to see its upstream/downstream connections in the graph
5. After a run, watch each node's run state (done/error/aborted)
6. Cross-check the node list against the right-side FLOW TREE

## EXAMPLE / EXERCISE

A minimal approval flow as a DAG: an LLM node drafts the decision -> a Review node gates human approval -> a Write node persists the result.

```
[LLM draft] --> [Review approve] --> [Write archive]
                 approve / reject / request_changes
```

On the canvas, drag the Review node after the LLM node and draw one edge to complete the basic structure.

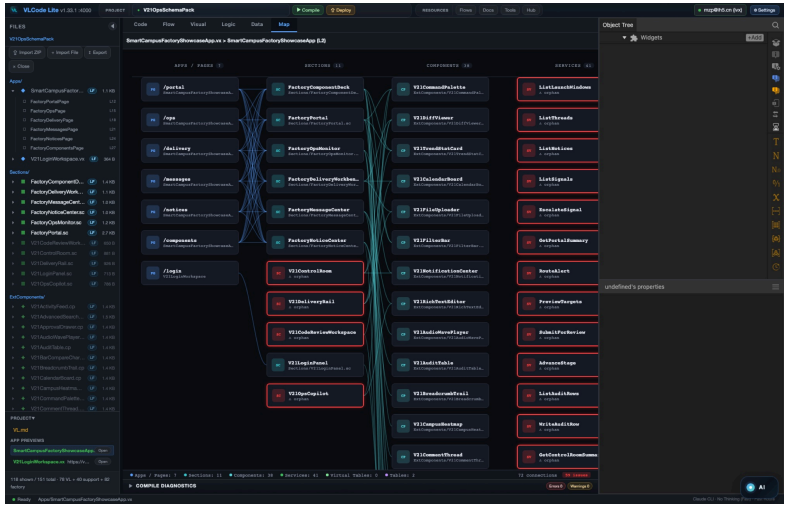
## PITFALL

Leaving a dangling edge while wiring — a node connected to a missing or deleted target. Always run structural validation before a run; dangling edges keep the DAG from settling cleanly into a terminal state.

## CHECKPOINT

PART III: INTERFACE TOUR

# Map Mode Metadata Bridge



Map Mode — the metadata and visual bridge

## CORE IDEA

Map Mode is the metadata/visual bridge: it presents a workflow's or app's metadata as a human-readable graph so you can see how resources relate without reading JSON line by line. The difference from the Workflow Editor is that the editor draws the execution graph while Map Mode draws the structure/metadata graph — the two are cross-references for each other.

## OPERATIONS

1. Switch the center canvas to the Map Mode tab
2. View the metadata graph of the current workflow/app
3. Trace nodes, resources, and their relationships through the graph
4. Cross-check against the Workflow Editor's DAG to confirm the structure matches
5. Pair it with the Manifest Inspector when you need planned vs actual

## EXAMPLE / EXERCISE

After an agent-app-modify-main change, switch to Map Mode to verify the metadata correctly registered the newly added Pause node into the structure; then switch back to the Workflow Editor to confirm the DAG also contains that node. The change is correct only when both sides agree.

## PITFALL

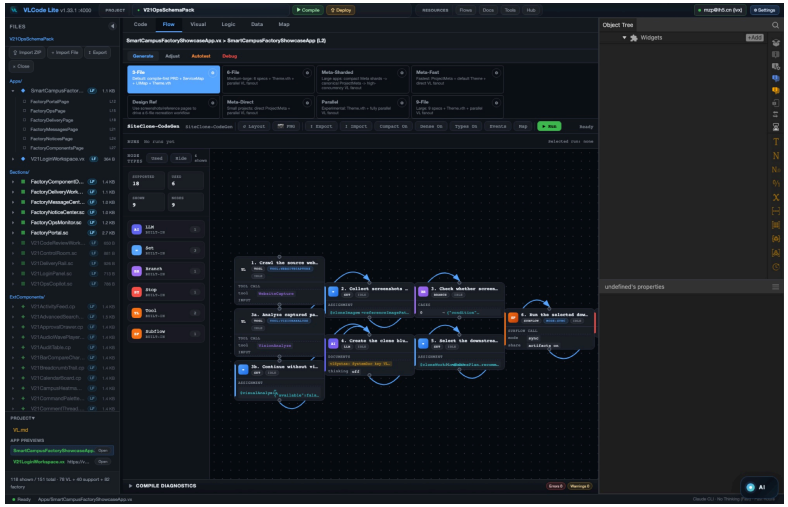
Treating Map Mode as a canvas where you drag nodes and run flows. It is a metadata bridge/view; author the execution graph in the Workflow Editor instead.

## CHECKPOINT

You can read the current workflow's metadata structure in Map Mode and point to how it corresponds to the DAG in the Workflow Editor.

PART III: INTERFACE TOUR

# Flow Mode



Flow Mode — workflow picker, codegen, autotest

## CORE IDEA

Flow Mode gathers the workflow picker, code generation (codegen), automated testing (autotest), and incremental adjust into one working mode — the console for "pick a flow and run a class of task." It targets flow-level daily work: choose a seed workflow, generate, test, adjust. Unlike editing a single .vflow directly (Flow Bundle Mode), Flow Mode is about getting work done with ready-made flows.

## OPERATIONS

1. Switch the center canvas to the Flow Mode tab
2. Pick a seed flow with the built-in workflow picker
3. Kick off a codegen run for generation-class flows
4. Run autotest against the generated result
5. Use an adjust-class flow for incremental changes
6. Track run events in Detail Log until a terminal state

## EXAMPLE / EXERCISE

Quickly scaffolding a CustomerCRM project: in Flow Mode pick `6-file-codegen` and start generation; when done pick `autotest-pipeline` to run automated tests; then use `add-page` to incrementally add a customer-detail page — all within Flow Mode.

## PITFALL

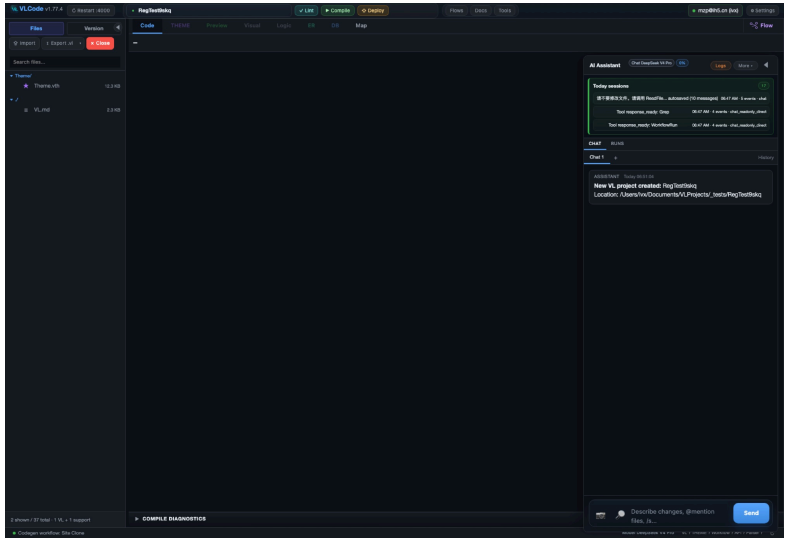
Expecting per-node drag editing of the DAG in Flow Mode. That is the Workflow Editor's job. Flow Mode is a "pick a flow + run a flow" console, not a node-level editor.

## CHECKPOINT

You can select a codegen seed flow in Flow Mode, start a run, and then kick off an autotest against the result.

PART III: INTERFACE TOUR

# Flow Bundle Mode: Edit A .vflow Directly



Flow Bundle Mode — editing a .vflow directly

## CORE IDEA

Flow Bundle Mode lets you open, edit, and save a .vflow file directly. The .vflow is VL Agent's single app file — a self-contained bundle format (workflow JSON + manifest + optional subflows/docs/prompts). This mode is for fine-tuning a specific .vflow bundle — editing the DAG and managing the other in-bundle resources — the "author a deliverable" perspective.

## OPERATIONS

1. Switch the center canvas to the Flow Bundle Mode tab
2. Open an existing .vflow file
3. Edit the in-bundle workflow JSON / manifest plus optional subflows, docs, and prompts
4. Save back to the .vflow (or Save-As a new .vflow)
5. Validate the bundle structure before registering it to the local Resource Center

## EXAMPLE / EXERCISE

Before handing a tuned approval flow to a VLCode team, open `approval-flow.vflow` in Flow Bundle Mode, confirm the manifest declares all Pause/Review steps, ensure docs/prompts are packed in, then Save as a self-contained .vflow.

```
approval-flow.vflow
  |- workflow.json   (DAG)
  |- manifest        (planned surfaces / lifecycle)
  |- docs/ prompts/  (optional, embedded)
```

## PITFALL

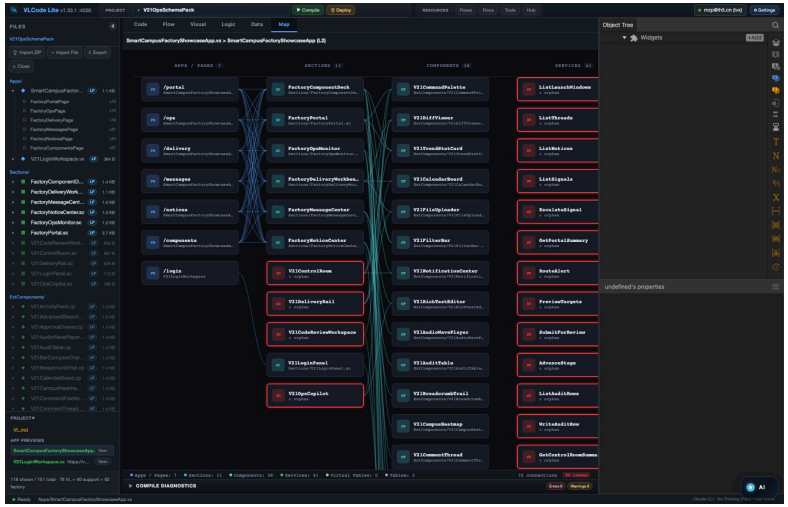
Assuming editing a .vflow means splitting it into multiple files. VL Agent uses a single-.vflow app model with no separate .app.json — everything lives in one bundle. Creating an external app.json makes the importing side fail to find the manifest.

## CHECKPOINT

You can open a .vflow in Flow Bundle Mode, change its contents, and Save-As a new .vflow while keeping the bundle structure intact.

PART III: INTERFACE TOUR

# ASSETS / UNITS / FLOW TREE Rails



ASSETS / UNITS / FLOW TREE rails and the Unit editor

## CORE IDEA

The three right rails give the current workflow/app three side views: ASSETS lists app files (resources in the app pack), UNITS lists the workflow's units (the left-side Unit of Agent App Studio's three-layer editing, carrying I/O contracts), and FLOW TREE lists the DAG's node tree. They let you reconcile resources, units, and nodes from a list perspective without panning around the canvas.

## OPERATIONS

1. Switch the right rail to ASSETS to view app file resources
2. Switch to UNITS to browse workflow units and their I/O contracts
3. Switch to FLOW TREE to check the DAG node list as a tree
4. Click a node in FLOW TREE to sync the selection with the center canvas
5. After a change, return to the matching rail to confirm resources/units/nodes updated as expected

## EXAMPLE / EXERCISE

After adding a human-input step to a TeamPM approval app: confirm in the UNITS rail that a corresponding unit with its input contract appeared, check the FLOW TREE rail that a Pause node was added to the DAG, and look at the ASSETS rail for any new ViewKit resource pulled in.

## PITFALL

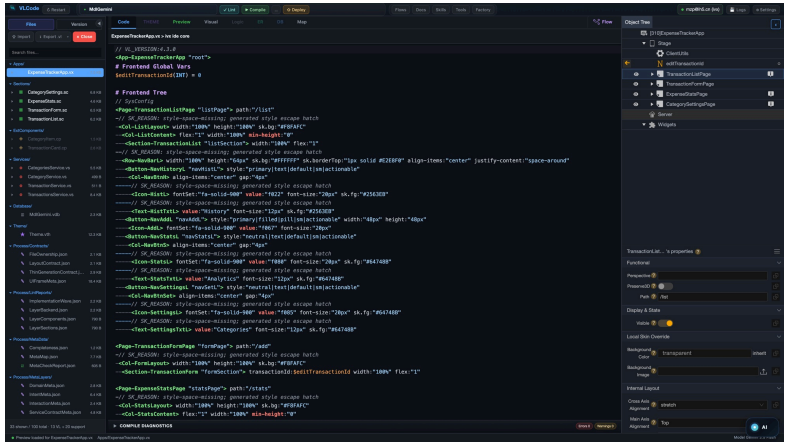
Conflating UNITS (units with I/O contracts) with FLOW TREE (compiled DAG nodes). A Unit is a human-readable logic unit; a DAG node is a compiled execution node — one Unit may compile to several nodes.

## CHECKPOINT

You can find the trace of the same change in each of the ASSETS, UNITS, and FLOW TREE rails and explain what each one shows.

PART III: INTERFACE TOUR

# Floating AI Chat Panel



The floating AI Assistant panel and chat console

## CORE IDEA

The floating AI Chat panel is the main entry for talking to VL Agent: multi-turn conversations, /slash commands, and streaming output. It overlays the canvas (floating, not a fixed column) and can both drive workflows in natural language and control a run directly through the DAG Operator with no LLM round-trip (query_status, control_pause, control_resume, and so on). Chat shows only final summaries and human decisions; per-node detail lives in Detail Log.

## OPERATIONS

1. Open the floating AI Chat panel and confirm it overlays the canvas
2. State a goal in natural language, e.g. ask it to add or delete workflow nodes
3. Use /slash commands to control an app run (e.g. /start, /status)
4. Send natural-language control to a running workflow (DAG Operator)
5. Watch streaming output and final summaries in Chat; verify details in Detail Log

## EXAMPLE / EXERCISE

Mid-run, to stop on a failing prerequisite, say in Chat:

```
"if LLM_430 fails, stop"
```

This registers a watch (action=stop); the watch-fired event returns to Chat. To stop the current run immediately, just say "pause this run" to trigger control_pause with no LLM round-trip.

## PITFALL

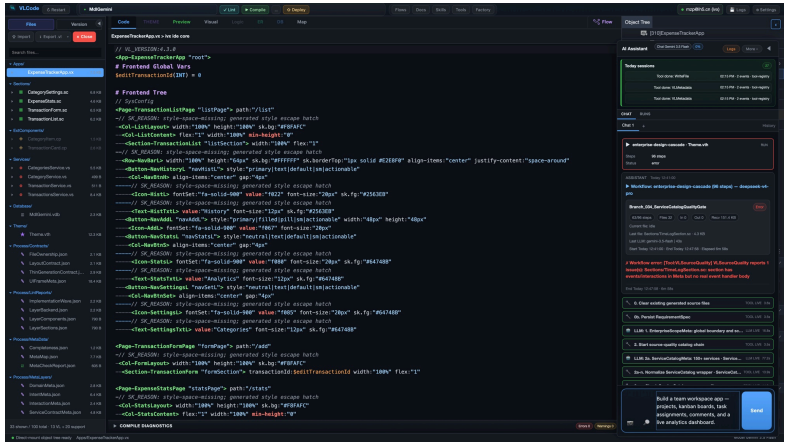
Cramming run details that belong in Detail Log into Chat. Chat never repeats the per-node ledger, and full tool I/O is not shown there either — those live only in Detail Log.

## CHECKPOINT

You can register a watch rule with a single natural-language sentence in the floating Chat, or pause a running workflow with control_pause.

PART III: INTERFACE TOUR

# Detail Log



Detail Log — the live run ledger and node results

## CORE IDEA

The Detail Log on the right is the full run ledger: per-node progress, file writes, tool I/O, validation results, and error stacks, all recorded as an event stream (event schema vl.workflow.run-event.v4). It is strictly divided from Chat — Chat holds decisions and summaries; Detail Log holds the line-by-line evidence of what happened. This is how "verified, not hopeful" is realized: to confirm a step actually ran, look here.

## OPERATIONS

1. Open the right-side Detail Log after running a workflow
2. Follow the event stream to see each node's start/done/error
3. Expand a Tool node to view its full tool I/O
4. Check validation results to confirm whether Lint/Compile passed
5. On a node error, read the error stack here to find the root cause
6. Reconcile the run's terminal state (done/error/aborted) against the result

## EXAMPLE / EXERCISE

A Write node errors; in Detail Log, expand that node and the error stack points to a file name containing Chinese characters. The fix: rename to English PascalCase and rerun the node.

```
[node Write] ERROR
  cause: non-ASCII file name '客户列表.sc'
  fix:   rename -> CustomerList.sc
```

## PITFALL

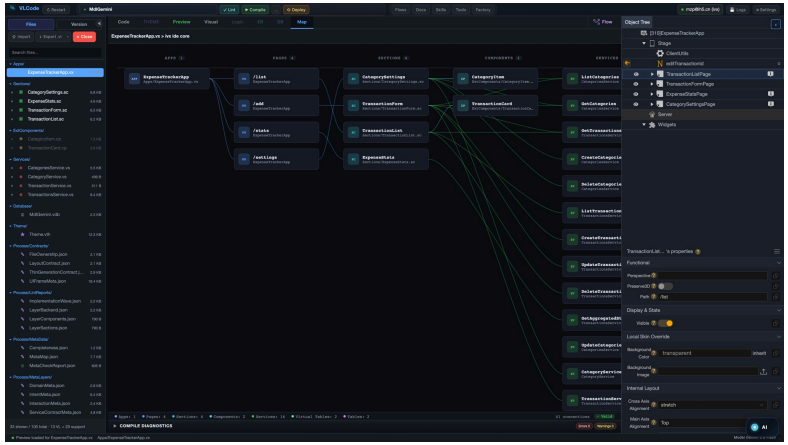
Going to Chat first to diagnose a failed run. Error stacks and tool I/O are not in Chat. The real root-cause evidence is always in Detail Log — walk the event stream back to the earliest error node.

## CHECKPOINT

You can locate a failed node's error stack in Detail Log, state the root cause, and point to the run's terminal state.

PART III: INTERFACE TOUR

# Manifest Inspector



Manifest Inspector — planned versus actual surfaces

## CORE IDEA

The Manifest Inspector is a left slide-in panel dedicated to planned vs actual: it shows the surfaces an Agent App declares (planned), the surfaces actually exposed at runtime (actual), lifecycle dots, and resource lineage. For an Agent App, the manifest is derived at load time from the app block plus the Pause/Review steps, and this panel is where you check that the derivation matches your intent.

## OPERATIONS

1. Slide out the Manifest Inspector from the left
2. Compare planned (declared) surfaces with actual (exposed) surfaces
3. Check the lifecycle dots (install -> release -> runtime)
4. Trace resource lineage for where resources come from and go to
5. When planned and actual disagree, go back to the app block or Pause/Review steps to investigate

## EXAMPLE / EXERCISE

An approval app declares two approval surfaces, but after a run the Manifest Inspector shows actual exposes only one — following the lineage reveals the second Review step was never wired into the DAG. After fixing the edge, planned and actual align and all lifecycle dots light up.

## PITFALL

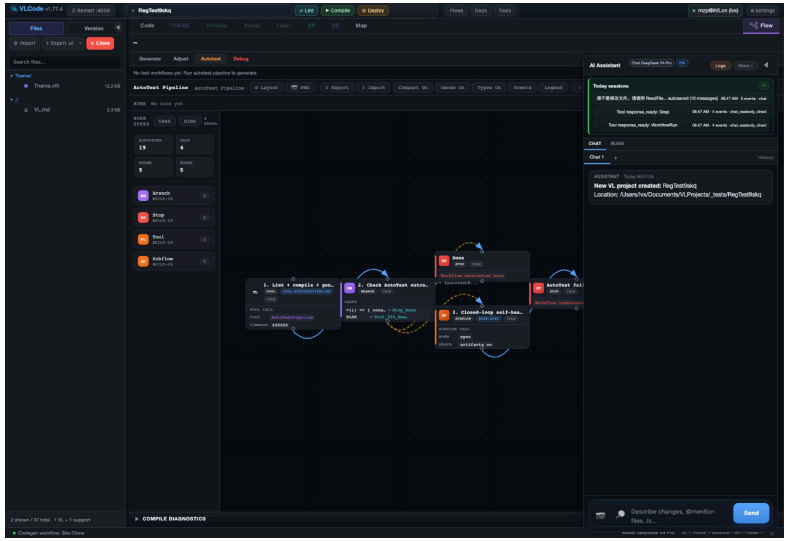
Looking only at planned and assuming the app is fine. A planned/actual mismatch usually means a Pause/Review step wasn't correctly derived into the manifest. Always read the actual column too.

## CHECKPOINT

You can read planned vs actual surfaces in the Manifest Inspector and, when they disagree, point to the likely missing step.

PART III: INTERFACE TOUR

# Bottom Status Bar



Bottom status bar: provider, model, ports, compile

## CORE IDEA

The bottom status bar is the at-a-glance line for the current state: current run step, LLM provider/model, compile status, and ports. VL Agent uses fixed ports (VL parser 3456, IDE 3200-3209, kernel 4300, gateway 4310-4319, agent-app runtime 4610-4699), and the status bar lets you confirm at any moment that you're on the right port with the right provider — enforcing fixed-port discipline with no silent fallback to a random port.

## OPERATIONS

1. Read the status bar line at the bottom of the window
2. Confirm the current LLM provider/model (e.g. deepseek-api / V4 Pro)
3. Confirm the current run-step name and compile status
4. Check the port falls in the conventional range (IDE 3200-3209, etc.)
5. If the port or provider is wrong, adjust in Settings/Header instead of ignoring it

## EXAMPLE / EXERCISE

After running the Web IDE from source, the status bar should show a port in the 3200-3209 range and the provider you picked in Settings.

```
Status bar:  Step: idle  |  deepseek-api / V4 Pro  |  Compile: ok  |  :3200
```

If it shows an odd port outside :3200, the port was likely taken and silently swapped — per fixed-port discipline, clear the old process and restart instead.

## PITFALL

Ignoring the port/provider hints in the status bar. A drifted port or a wrong provider makes later run results not line up; the status bar is the cheapest one-glance self-check.

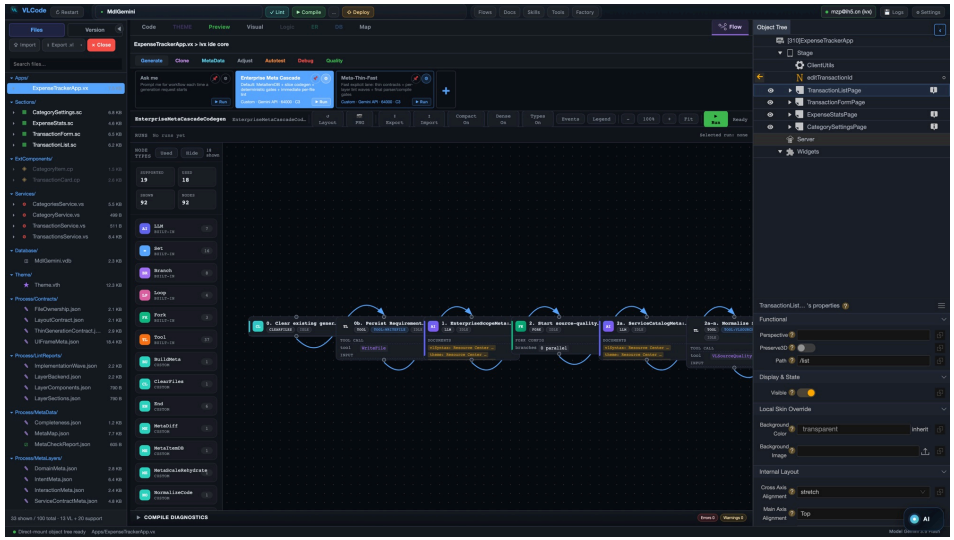
## CHECKPOINT

You can read the current provider/model, compile status, and port from the status bar and judge whether the port is in VL Agent's conventional range.

CHAPTER

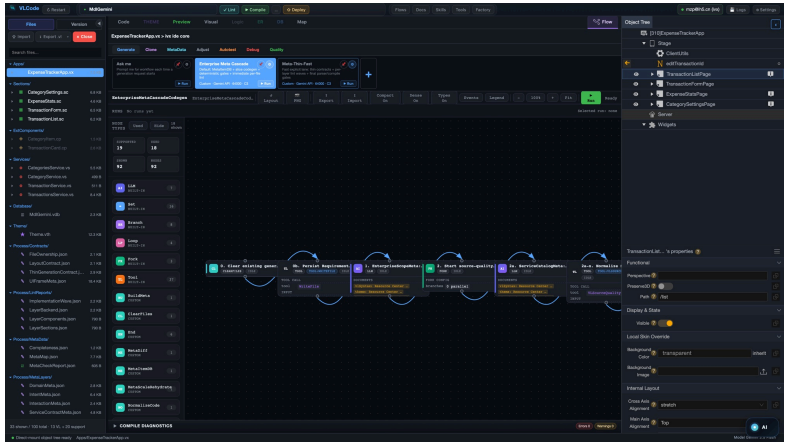
# Part IV: Workflow And DAG Basics

- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



Workflow Editor — nodes and directed edges on the DAG canvas

# Workflow Spec 4.1 Overview



Workflow Editor — nodes and directed edges on the DAG canvas

## CORE IDEA

VL Agent workflows follow Workflow Spec 4.1, executed by Workflow-Engine 4.13.1, with run events emitted under the vl.workflow.run-event.v4 schema. A workflow is a directed acyclic graph (DAG): nodes are steps, directed edges are execution order, and the spec defines the allowed node types, how variables pass between them, and the checkpoint/resume semantics. Understanding this spec is what lets you read why nodes are wired the way they are and why a run pauses at a given step.

## OPERATIONS

1. Open the Workflow Editor tab on the center canvas and note the node canvas and multi-flow tabs
2. Confirm the LLM provider/model and compile status look healthy in the bottom status bar
3. Open a seed workflow (e.g. agent-app-3file-main) from the left file tree or workflow picker
4. Run it and watch vl.workflow.run-event.v4 events arrive in order in the Detail Log right rail
5. Map each step to its node type (LLM / Tool / Set / Pause / Review, etc.) to understand its role

## EXAMPLE / EXERCISE

Validate a workflow against Spec 4.1 from the CLI:

```
node bin/vlflow.js validate <workflowId>
```

If the structure is legal it reports a pass; if a node wiring is disallowed or an edge is dangling, the output names the offending node.

## PITFALL

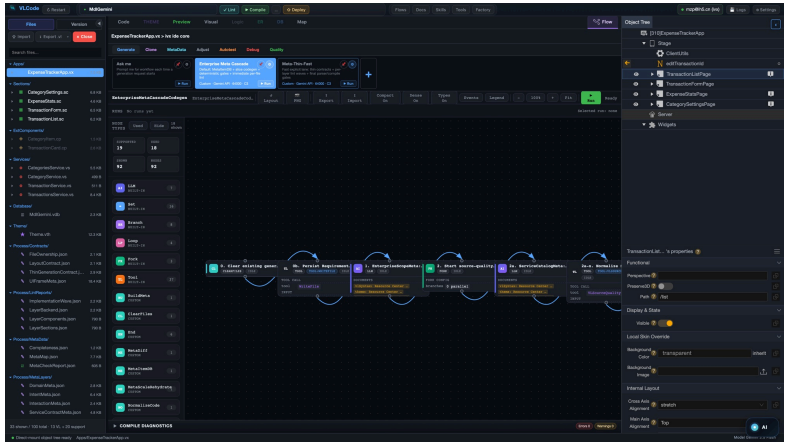
Treating a workflow like a linear script and expecting nodes to run top-to-bottom. It is a DAG — branches expand in parallel, so read the directed edges, not a node's vertical position.

## CHECKPOINT

You can state that the current workflow uses Workflow Spec 4.1, engine 4.13.1, and the vl.workflow.run-event.v4 event schema, and can point to at least three distinct node types in the Workflow Editor.

PART IV: WORKFLOW AND DAG BASICS

# Nodes And Directed Edges



Workflow Editor — nodes and directed edges on the DAG canvas

## CORE IDEA

A node is one step in a workflow (an LLM call, a tool execution, a variable assignment); a directed edge expresses order and data dependency — a downstream node may only reference variables produced upstream. Because the graph is acyclic (a DAG, no cycles), the engine can strictly determine execution order and checkpoint safely. On the DAG canvas each node carries an N# badge so you can refer to it precisely in Chat and the Detail Log.

## OPERATIONS

1. Click a node on the Workflow Editor canvas and inspect its incoming and outgoing edges
2. Follow an outgoing edge to its downstream node to confirm the data-dependency direction
3. Drag from one node's output port to another node's input port to create a directed edge
4. Read the N# node badges in the DAG Map and remember each node's number
5. After a run, match each Detail Log event to its node by N#

## EXAMPLE / EXERCISE

Read the edges of a minimal approval flow like this:

```
N1 Set(request)  ->  N2 LLM(risk score)  ->  N3 Review(manager approval)  ->  N4 Write(archive)
```

N3 can only run after N2's score variable is produced, because the approval card displays that score.

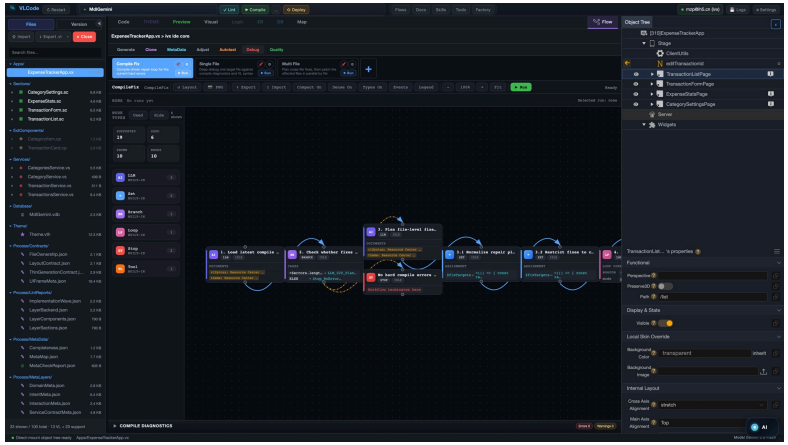
## PITFALL

Wiring two unrelated nodes together so the downstream node references a variable that was never produced. Before connecting an edge, confirm the upstream node actually writes the variable the downstream node needs.

## CHECKPOINT

You can manually draw an A→B directed edge on the canvas and explain why B must wait for A to finish.

# LLM Nodes



A node detail drawer open over the DAG

## CORE IDEA

An LLM node is the AI step in a workflow — it hands a prompt to the selected language model and writes the text/structured result back into a variable for downstream use. It is the only node type that actually calls a model and incurs token cost, so it uses the workflow provider/model configured in Settings (default Kimi 2.5) and can enable thinking and a reasoningLevel as needed.

## OPERATIONS

1. Place an LLM node on the Workflow Editor canvas and wire in its upstream variables
2. Open the node detail drawer and write the prompt template (which can reference upstream variables)
3. Confirm the provider/model it uses, overriding it for this node if needed
4. Run, then watch the node's streaming LLM output and the final written variable in the Detail Log
5. If it errors twice in a row, return to the prompt and upstream variables for root cause instead of re-running blindly

## EXAMPLE / EXERCISE

An LLM node generating acceptance criteria for a TeamPM task could use this prompt:

```
From task title {{task.title}} and description {{task.desc}}, output 3 verifiable acceptance criteria, one per line.
```

The three returned lines are written into the downstream variable acceptanceCriteria.

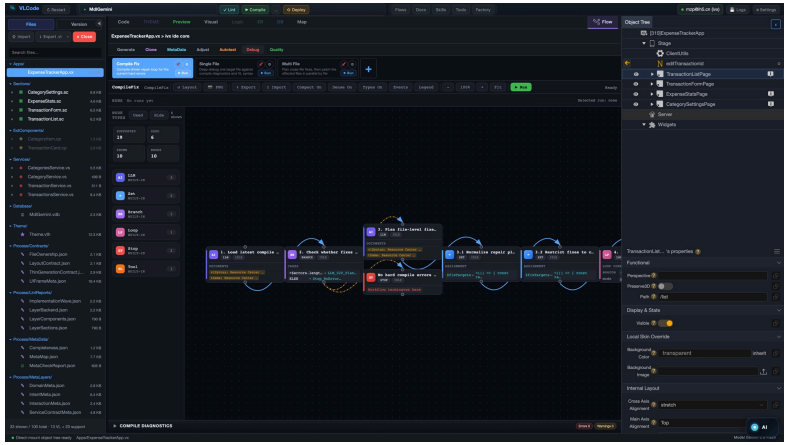
## PITFALL

Referencing a variable in the prompt that hasn't been produced yet (e.g. {{task.desc}} from a node that hasn't run), so an empty value is substituted. Confirm a directed edge sequences that node first.

## CHECKPOINT

You can add an LLM node, write a prompt that references an upstream variable, and see its streaming output and resulting variable in the Detail Log.

# Tool Nodes



A node detail drawer open over the DAG

## CORE IDEA

A Tool node is the executor step in a workflow — it does not call a model; it deterministically performs an action: file I/O, an HTTP request, a Git operation, a VL compile, invoking the component factory, and more. It plugs the outside world into the DAG so that AI-produced content lands as real files and real compile results — a key piece of the "verified, not hopeful" principle.

## OPERATIONS

1. Place a Tool node on the Workflow Editor canvas
2. Open the node detail drawer and pick the specific executor (e.g. HTTP, Git, VL compile)
3. Map upstream variables onto the tool's input parameters
4. Run, then inspect the tool's full I/O (expandable/collapsible) in the Detail Log
5. Feed the tool's real return value to downstream nodes instead of letting an LLM guess the result

## EXAMPLE / EXERCISE

A Tool node using the VL compile executor to validate freshly generated code shows the real compile output in the Detail Log; on failure the error surfaces as-is rather than being swallowed:

```
Tool(VL compile) -> compileResult = { ok:false, errors:["Section Login: missing route"] }
```

Downstream can branch on this to repair instead of faking success.

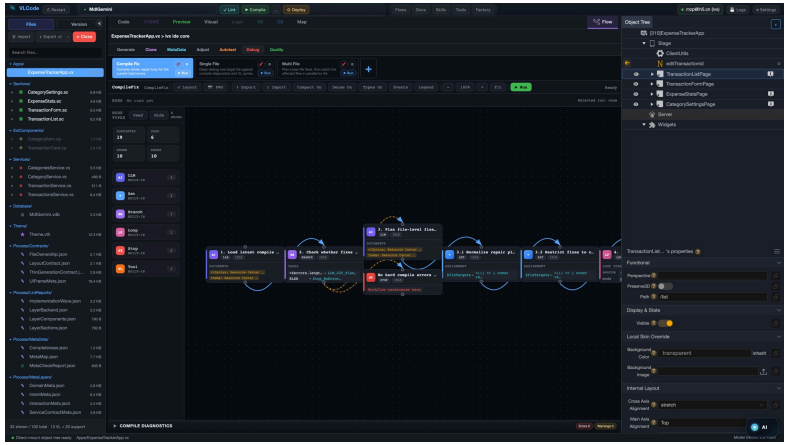
## PITFALL

Asking an LLM node to "describe" an action that needs deterministic execution (calling an external API, running a compile), and getting only text with no real side effect. Use a Tool node whenever a real action is required.

## CHECKPOINT

You can configure a Tool node to perform a real action (e.g. VL compile or an HTTP request) and see its real I/O return value in the Detail Log.

# Set / Variable Nodes



A node detail drawer open over the DAG

## CORE IDEA

A Set node assigns a variable — it normalizes a constant, an expression result, or an upstream output into a named variable for later nodes to reference. It is the glue that organizes data flow in a workflow: it lays down variables between LLM and Tool nodes so prompts and tool parameters have something concrete to read. It calls no model and has no external side effect, so it runs fast and repeatably.

## OPERATIONS

1. Place a Set node on the Workflow Editor canvas
2. Open the node detail drawer and define the variable name and value (a constant or a reference to an upstream variable)
3. Wire it with a directed edge to the downstream nodes that need the variable
4. After a run, confirm the written value in the variable-update summary in the Detail Log
5. Reference it downstream with {{variableName}}

## EXAMPLE / EXERCISE

Prepare threshold variables for a CustomerCRM approval flow:

```
N1 Set: approvalThreshold = 50000
N2 Set: region = "APAC"
```

A downstream Review node uses {{approvalThreshold}} to decide whether a second-level approval is required.

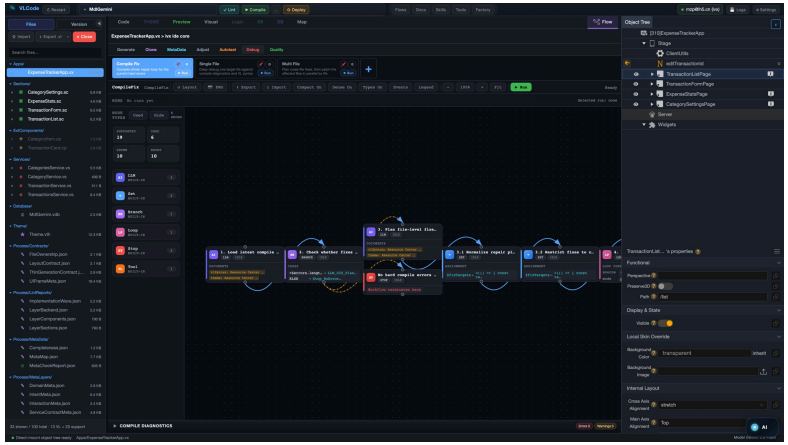
## PITFALL

Hard-coding a complex result that should come from LLM reasoning or tool computation into a static Set constant, leaving the run out of sync with real data. Use Set for stable config/threshold values; let LLM or Tool nodes produce dynamic results.

## CHECKPOINT

You can define a named variable with a Set node and successfully reference it with {{variableName}} in a downstream node's prompt or parameter.

# Write / Download / Unzip Nodes



A node detail drawer open over the DAG

## CORE IDEA

Write, Download, and Unzip are a group of I/O nodes that land a workflow's output onto disk: Write writes variable content to a file, Download fetches a resource from a URL, and Unzip extracts an archive. They make a generative workflow produce real, inspectable, deliverable files rather than just chat output — squarely in line with the local-first and verifiable ethos.

## OPERATIONS

1. Place a Write node on the Workflow Editor canvas
2. Set the target path and the variable content to write in the node detail drawer
3. For remote assets, add a Download node to fetch them, then an Unzip node to extract
4. Use directed edges to enforce order: Download first, then Unzip, then Write/compile
5. After a run, confirm each produced file in the file-write records of the Detail Log

## EXAMPLE / EXERCISE

Write an LLM-generated Section into the project directory:

```
N3 Write: path = "Sections/TaskBoard.sc", content = {{generatedSection}}
```

After the run the Detail Log lists this file write, which you can open straight from the left file tree to verify.

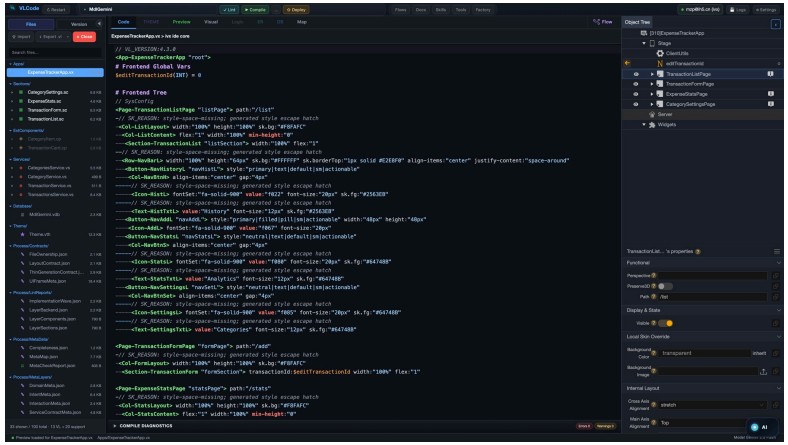
## PITFALL

Giving Write a non-English file name, violating the English PascalCase convention, which makes a later compile throw strange errors. VL source file names must be English PascalCase (e.g. TaskBoard.sc).

## CHECKPOINT

You can use a Write node to land variable content as a file on disk and see it in both the Detail Log file-write records and the left file tree.

# Pause Human-Gate Nodes



A Pause/Review interaction card waiting for a human

## CORE IDEA

A Pause node is the human-in-the-loop gate in a workflow — the run halts here and presents a human input form (text/textarea/number/select/checkbox), resuming only after the person fills it in. It hands control back to a human at critical points, and graceful pause survives a socket close, so the run resumes without losing state when you return.

## OPERATIONS

1. Place a Pause node on the Workflow Editor canvas
2. Define the fields to collect (e.g. text, number, select) in the node detail drawer
3. Wire it with a directed edge before the step that needs human-supplied information
4. When the run reaches it, fill the input card that appears in the AI Chat
5. After submitting, watch the Detail Log confirm the run continues from that node with state intact

## EXAMPLE / EXERCISE

A procurement flow uses a Pause to collect budget confirmation before placing an order:

```
Pause: fields budget(number) + costCenter(select) + note(textarea)
```

The user fills budget=80000, costCenter=Marketing in the Chat card; on submit the workflow continues to the order node carrying those variables.

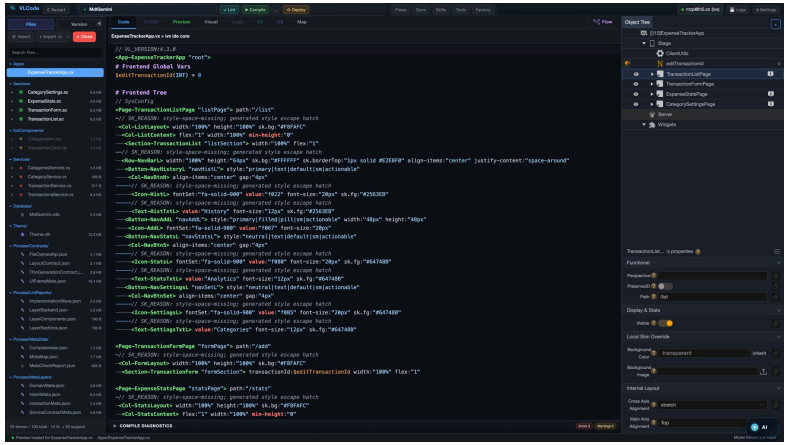
## PITFALL

Letting an LLM node auto-approve a critical decision that needs human judgment, skipping the Pause gate. Any step needing human confirmation or supplied data should explicitly include a Pause (or Review) node.

## CHECKPOINT

You can make a run halt at a Pause node, submit the form fields in the Chat card, and see the run continue carrying those values.

# Review Decision Nodes



A Pause/Review interaction card waiting for a human

## CORE IDEA

A Review node is a structured decision gate — the run reaches it and presents an approval card where a human picks one of approve / reject / request_changes, steering which branch the flow takes. Unlike Pause (which collects arbitrary input), Review is purpose-built for the audit-bearing three-state "approve/reject/request_changes" decision, and the choice is recorded in the control plane as a traceable approval record.

## OPERATIONS

1. Place a Review node on the Workflow Editor canvas
2. Configure the context shown on the card (e.g. the item under review, a score) in the node detail drawer
3. Wire one outgoing edge each for approve / reject / request_changes to the corresponding branch
4. When the run reaches it, make the decision on the approval card in the AI Chat
5. Verify the human decision is recorded in the control plane's review resources

## EXAMPLE / EXERCISE

A CustomerCRM discount approval uses a Review node to judge an over-threshold discount:

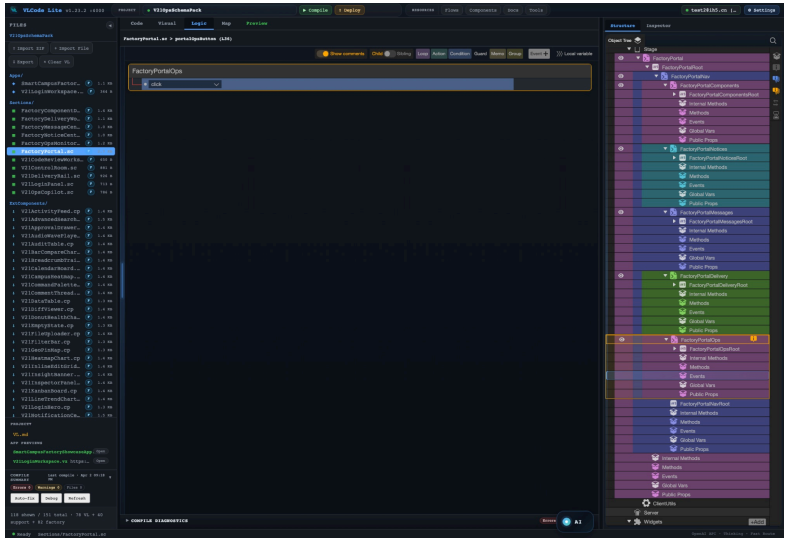
```
Review(shows discount=35%, threshold=20%)
  approve -> apply discount
  reject -> notify sales
  request_changes -> send back to re-fill
```

The manager clicks request_changes and the flow returns to the form node for edits.

## PITFALL

Treating Review as a two-state pass/fail and ignoring the request_changes branch, leaving cases that need edits with nowhere to go. Wire a downstream branch for all three outcomes.

# VisualLogic Event Nodes



VisualLogic event editor — graphical if/then/switch

## CORE IDEA

A VisualLogic node is a graphical if/then/switch editor — you build conditional branching logic visually inside the node, and at runtime the engine compiles it to runEvent(ctx) to execute. It expresses conditional decisions without hand-written code, making it the go-to node for control flow like "branch differently based on some variable" inside a DAG.

## OPERATIONS

1. Place a VisualLogic node on the Workflow Editor canvas
2. Open the node detail drawer and enter the graphical if/then/switch editor
3. Configure conditional branches based on upstream variables (e.g. amount ranges, status enums)
4. Wire a corresponding directed outgoing edge for each branch outcome
5. After a run, confirm in the Detail Log that it compiled to runEvent(ctx) and took the matched branch

## EXAMPLE / EXERCISE

A TeamPM task priority router decides with VisualLogic:

```
switch(priority)
  case "P0": -> notify on-call immediately
  case "P1": -> add to this week's board
  default:   -> enter the triage queue
```

A P0 task hits the first branch and fires the notification node directly.

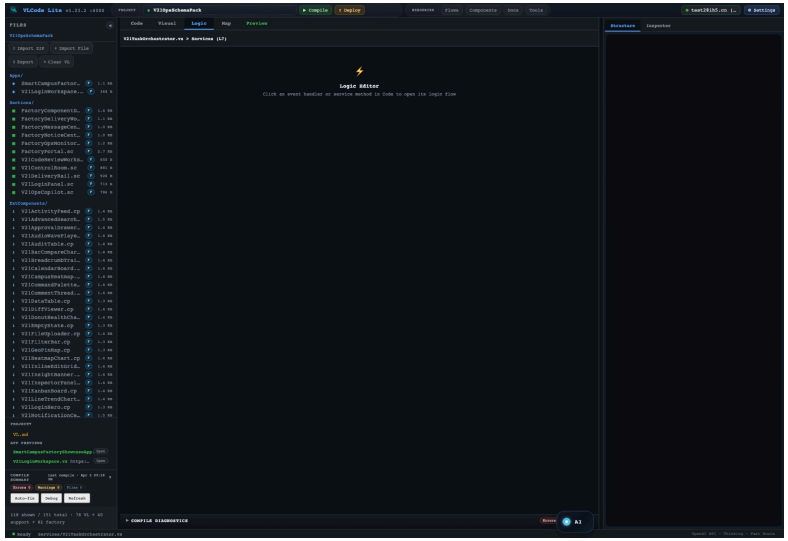
## PITFALL

Forcing a simple conditional onto an LLM node to "decide", which is slow and non-deterministic. Deterministic if/then/switch belongs in a VisualLogic node — cheaper and reproducible.

## CHECKPOINT

You can build a variable-driven switch in a VisualLogic node, have it hit the expected branch at runtime, and see runEvent execute in the Detail Log.

# Swarm Nodes

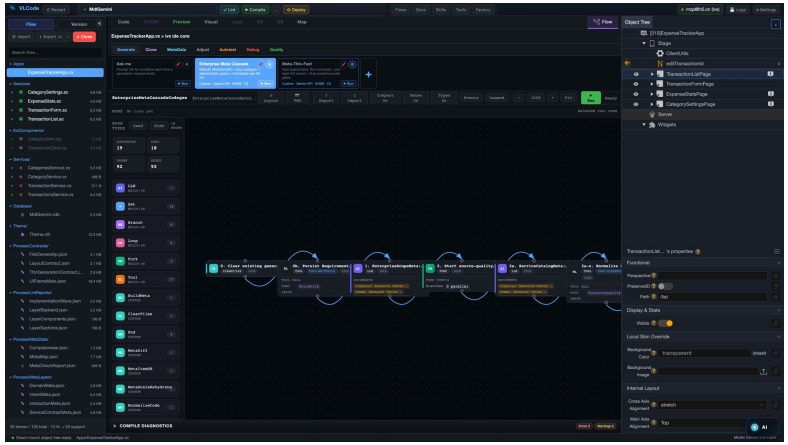


Formula node editor — a single computed expression

## CORE IDEA

(Content pending.)

# Actor And Distributed Tasks



Workflow Editor — nodes and directed edges on the DAG canvas

## CORE IDEA

An Actor node dispatches a workflow step as a distributed actor-task, letting independent work units carry their own responsibilities in parallel — for example splitting PM intake, frontend/backend modules, and QA review across different actors running concurrently. It is the mechanism for expanding a single DAG into multi-role collaboration, and actor tasks are recorded in the control plane for tracing.

## OPERATIONS

1. Place an Actor node on the Workflow Editor canvas
2. Define this actor task's responsibility and input contract in the node detail drawer
3. Wire multiple Actor nodes in parallel to share different modules
4. After launching, govern how many run at once with workflowMaxConcurrency
5. Verify each actor's dispatch and result in the control plane's actor tasks

## EXAMPLE / EXERCISE

Following the seed actor-demo pattern, split a release board into parallel actors:

```
Actor: pm-intake/triage
Actor: frontend module
Actor: backend module
Actor: qa-review
```

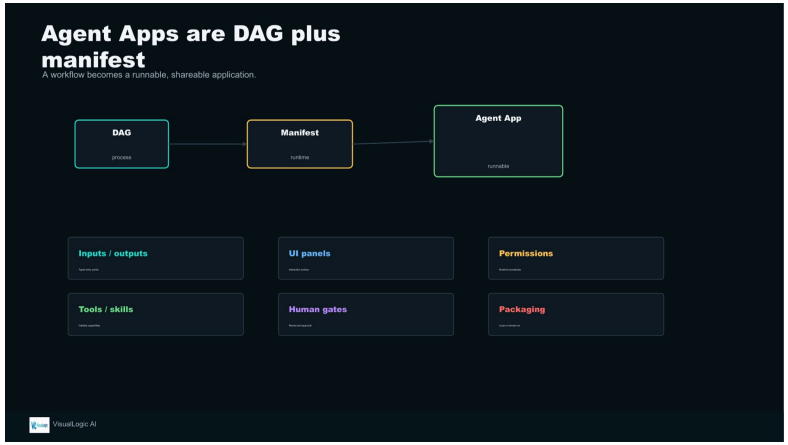
The four actors advance their tasks concurrently, then results converge into the release-board.

## PITFALL

Chaining all actors into a single line, losing the parallelism; or ignoring workflowMaxConcurrency (default 5) and dispatching too many tasks at once. Wire edges to real dependencies and let the concurrency cap act as a guardrail.

## CHECKPOINT

# Loop / Iteration / Channel / Score




Master Agent — branch scoring, pruning, and promotion

## CORE IDEA

Loop / Iteration / Channel / Score are the advanced "DAG-complexity" nodes in Workflow Spec 4.1: Loop repeats a sub-flow, Iteration walks a collection processing each item, Channel passes/merges messages across parallel branches, and Score rates candidate results (which the Master Agent uses to prune and promote). They lift a workflow from linear steps to process intelligence that can fan out, iterate, and pick the best.

## OPERATIONS

1. Place Loop / Iteration / Channel / Score nodes on the Workflow Editor canvas as needed
2. Use Iteration to walk a collection variable (e.g. a batch of tasks), running the same sub-flow per item
3. Use Channel to merge results across parallel branches
4. Use Score to rate multiple candidates, working with the Master Agent to prune/promote
5. After a run, watch the iteration count and per-round events in the Detail Log

## EXAMPLE / EXERCISE

Generate acceptance criteria per item for a batch of TeamPM tasks, then pick the best:

```
Iterate(tasks as t)
  -> LLM(generate acceptance criteria for t)
  -> Score(rate clarity)
Channel: merge per-task results
```

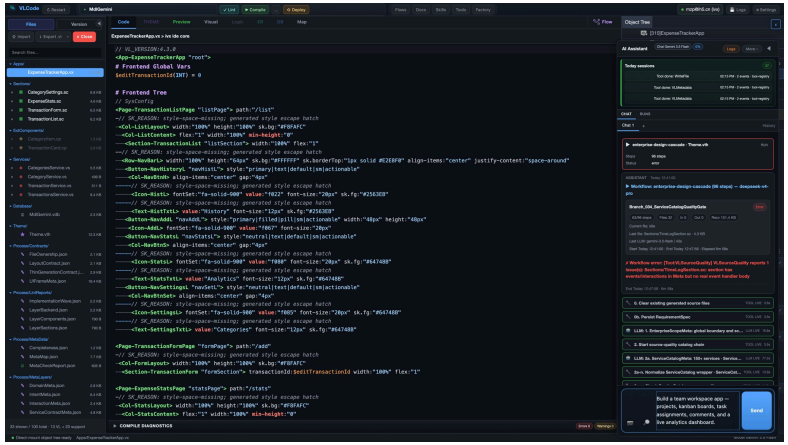
The higher-Score version is promoted to mainline by the Master Agent.

## PITFALL

Using Iteration over a large collection with no concurrency cap, fanning out too many LLM calls at once and losing control of cost and rate. Combine workflowMaxConcurrency with Score-based pruning to bound the fan-out.

## CHECKPOINT

# Checkpoint, Resume, And Rerun-From-Step



Detail Log — the live run ledger and node results

## CORE IDEA

VL Agent's engine natively supports checkpoint / resume / rerun-from-step: each completed node drops a checkpoint into the control plane, a run can resume after a break, and you can rerun from a specific step without starting over. Combined with graceful pause (which survives a socket close), this makes debugging long flows both time-saving and free of lost intermediate state.

## OPERATIONS

1. Run a workflow and let it advance in the Detail Log, dropping checkpoints
2. Pause it when needed, or let it stop after a node fails
3. After fixing the upstream variable or node config, choose 'rerun from this step'
4. Use the CLI's vlflow resume / state for headless resume and state inspection
5. Confirm the rerun starts only from the chosen node and reuses earlier checkpoint results

## EXAMPLE / EXERCISE

Inspect run state from the CLI, then resume from the break point:

```
node bin/vlflow.js state <runId>
node bin/vlflow.js resume <runId>
```

If node 5 was misconfigured, fix it and rerun from that node — the outputs of the first 4 nodes are not recomputed.

## PITFALL

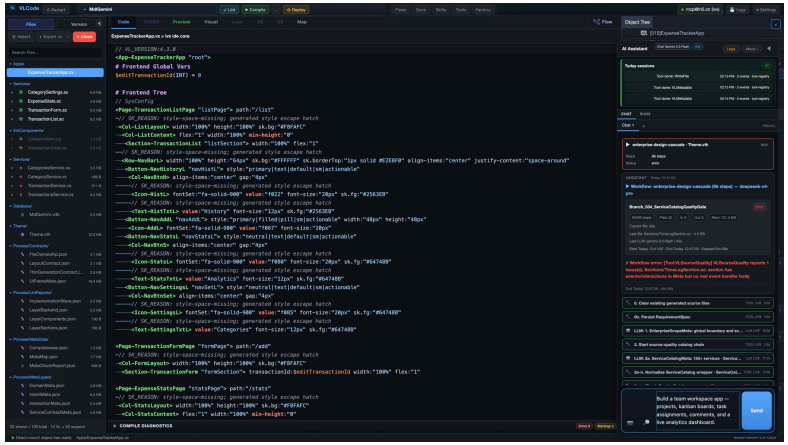
Re-running the whole thing from scratch after a failure, needlessly recomputing expensive LLM nodes that already succeeded. Look at the checkpoints and rerun from the step that actually failed to save tokens and time.

## CHECKPOINT

You can stop a run, fix the problem, rerun from a specific step, and confirm previously successful node results are reused rather than recomputed.

PART IV: WORKFLOW AND DAG BASICS

# Event Schema And The Run Ledger



Detail Log — the live run ledger and node results

## CORE IDEA

Every event a workflow run emits follows the vl.workflow.run-event.v4 schema (node start/done/error, file writes, variable updates, tool I/O, and more); these stream live into the Detail Log and are persisted into each workspace's control-plane database, .vl-code/control-plane.sqlite, as the run ledger. The ledger records history, timestamps, error stacks, and node results — the single reliable source for after-the-fact audit and debugging.

## OPERATIONS

1. Run a workflow and watch vl.workflow.run-event.v4 events arrive live in the Detail Log right rail
2. Expand an event to see the full tool I/O or variable-update detail
3. On failure, read the error stack in the event to locate the failing node
4. For history, consult the control-plane ledger (.vl-code/control-plane.sqlite)
5. Use the CLI's vlflow ledger to export run history in headless scenarios

## EXAMPLE / EXERCISE

Inspect a run's ledger and artifacts from the CLI:

```
node bin/vlflow.js ledger <runId>
node bin/vlflow.js artifacts <runId>
```

The ledger lists each node's start/done timestamps, with error stacks on failed nodes, for precise post-mortems.

## PITFALL

Hunting for run detail in the AI Chat — Chat holds only final summaries and human decisions; per-node progress and tool I/O live in the Detail Log and the control-plane ledger, so looking in the wrong place finds nothing.

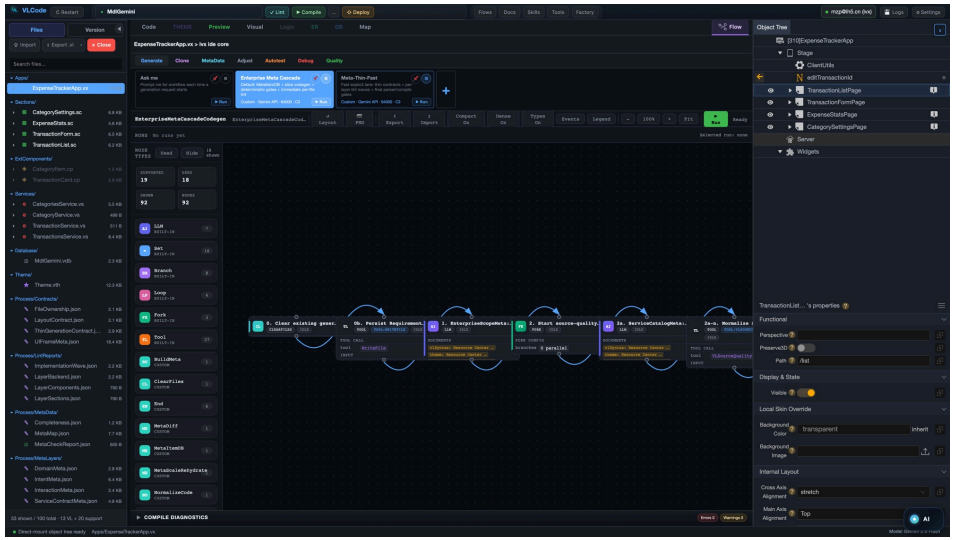
## CHECKPOINT

You can state that run events follow vl.workflow.run-event.v4, are persisted in the .vl-code/control-plane.sqlite ledger, and can pull a run's history with vlflow ledger.

CHAPTER

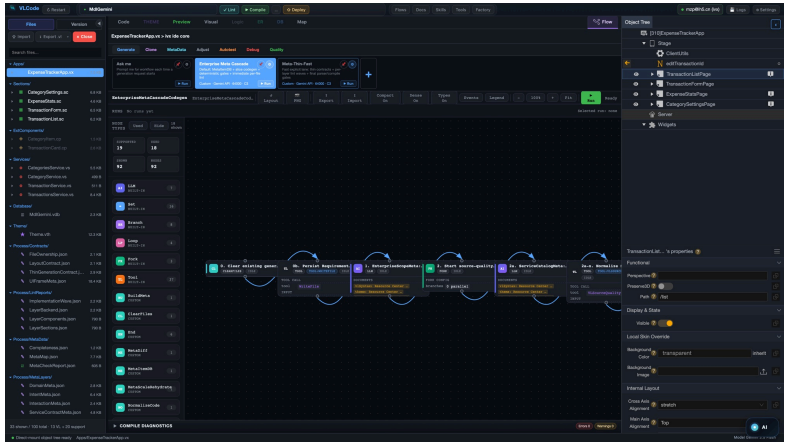
# Part V: Authoring Workflows On The Canvas

- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



Workflow Editor — nodes and directed edges on the DAG canvas

# Create A New Workflow



Workflow Editor — nodes and directed edges on the DAG canvas

## CORE IDEA

In VL Agent a workflow is a directed acyclic graph (DAG) that follows Workflow Spec 4.1 and runs on Workflow-Engine 4.13.1. Creating a new workflow means opening a blank Workflow Editor canvas onto which you then drop LLM, Tool, Set, Pause, and other nodes and wire them together. The new workflow lives as a multi-flow tab on the center canvas and can be saved, run, and validated independently.

## OPERATIONS

1. Click new in the left workflow picker, or start an empty workflow from Flow Mode
2. Confirm the center canvas is on the Workflow Editor (DAG node canvas) tab
3. Name the workflow in English PascalCase to honor the naming discipline
4. Begin from the blank canvas by placing a first node (usually a Set to initialize a variable, or an LLM to kick off)
5. Save so it is registered in the control plane as a runnable workflow

## EXAMPLE / EXERCISE

Create a 'task auto-assign' workflow for TeamPM: click new in the workflow picker, name it `TaskAutoAssign`, and once the canvas is on the Workflow Editor, drop a Set node to initialize a `boardId` variable:

```
Set  boardId = "teampm-sprint-7"
```

This just opens the blank graph and seeds the first variable; LLM and Tool nodes come next.

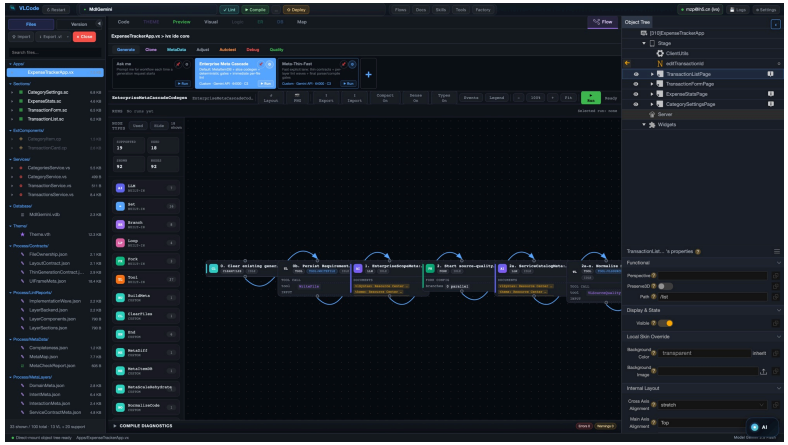
## PITFALL

Naming the workflow in Chinese or with spaces — the VL ecosystem enforces English PascalCase file names, and non-ASCII characters break saving/compiling. Use a form like `TaskAutoAssign` from the start.

## CHECKPOINT

You can create a clearly named empty workflow that appears as its own tab in the Workflow Editor and saves successfully

# Place And Connect Nodes



Workflow Editor — nodes and directed edges on the DAG canvas

## CORE IDEA

The Workflow Editor is a drag-and-drop node canvas: you place nodes, then drag a directed edge from one node's output port to the next node's input port, and the edge direction is the execution order. The node palette is rich — LLM, Tool, Set, Write/Download/Unzip, Pause, Review, Actor, Swarm, and more — and their connections form the DAG topology. The edges you draw decide how runtime data flows from upstream to downstream.

## OPERATIONS

1. Drag a node from the palette onto the canvas, or place a new node in empty space
2. Place a second node downstream of the first
3. Drag from the upstream node's output port to the downstream node's input port to draw a directed edge
4. Repeat placing and wiring to build a chain like LLM → Tool → Write
5. Save so the engine derives execution order from the edge topology

## EXAMPLE / EXERCISE

Build a minimal generation chain: place an LLM node to draft a spec, a Tool node to run VL compile, and a Write node to persist output, wired in sequence:

```
LLM (draft spec) ⟶ Tool (vl compile) ⟶ Write (Sections/Board.sc)
```

Edge direction is data direction: the LLM output feeds the Tool, whose artifact then writes the file.

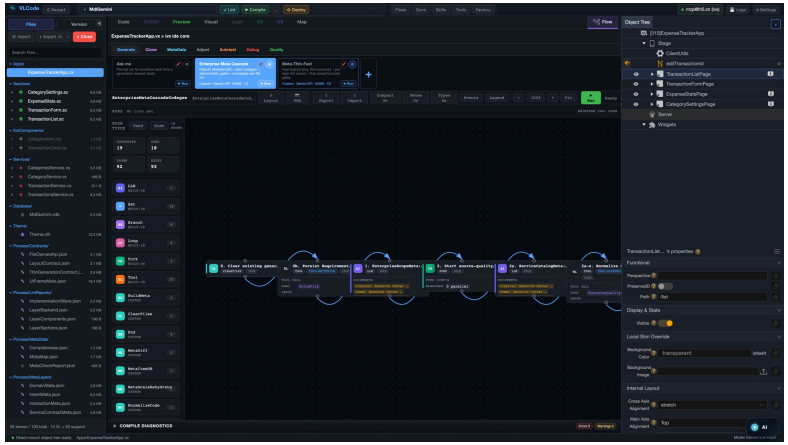
## PITFALL

Reversing an edge's direction or forgetting one, so a downstream node never receives its upstream variable. Always wire 'upstream output → downstream input' and confirm there are no dangling edges with structural validation after saving.

## CHECKPOINT

You can place at least three nodes and wire them into a correct chain with directed edges, and at run time they execute in the order you drew.

# Multi-Select, Delete, And Multi-Flow Tabs



Workflow Editor — nodes and directed edges on the DAG canvas

## CORE IDEA

The Workflow Editor supports multi-select and multi-flow tabs so you can edit larger DAGs efficiently. Multi-select lets you grab a group of nodes at once to delete or move them together; multi-flow tabs let you open several workflows side by side on the same center canvas and switch quickly between them. When you delete nodes, handle their edges too, or you leave dangling edges that structural validation will flag.

## OPERATIONS

1. Rubber-band or modifier-select a group of nodes on the canvas
2. Delete to remove the selected nodes together with their edges
3. Switch workflows in the tab bar above the canvas, or open a new tab
4. Carry ideas across tabs and compare topologies of different flows
5. Run structural validation after deleting to confirm no dangling edges remain

## EXAMPLE / EXERCISE

In `TaskAutoAssign`, remove a dead notification branch: select its LLM node and the following Tool(HTTP) node and delete them together, then switch to the adjacent `ApprovalFlow` tab to keep editing. Before vs after:

```
Before: ... ──► LLM(notify draft) ──► Tool(HTTP webhook)
After:  ... (branch removed entirely, no leftover edges)
```

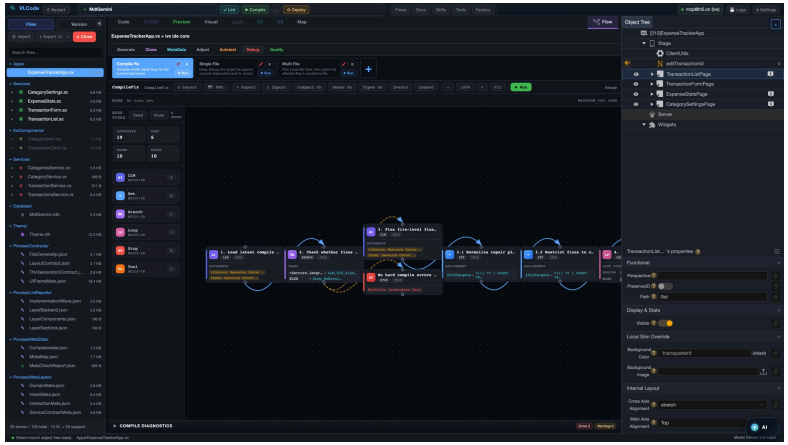
## PITFALL

Deleting only the nodes but leaving their in- or out-edges, so structural validation reports a dangling edge and the run errors. After deleting multi-selected nodes, always verify the related edges went with them.

## CHECKPOINT

You can multi-select and delete a group of nodes in one action, switch between multiple workflow tabs, and structural validation shows no dangling edges afterward.

# Open The Node Detail Drawer



A node detail drawer open over the DAG

## CORE IDEA

The node detail drawer is where you configure a single node: double-click or select a node to open it, and it shows that node type's parameters. An LLM node takes a prompt plus provider/model and reasoning; a Tool node picks an executor (file I/O, HTTP, Git, VL compile, component factory, etc.) and its inputs; a Set node takes a variable name and expression. The drawer is what turns a 'box on the canvas' into a real executable step.

## OPERATIONS

1. Select or double-click the target node to open its detail drawer
2. Confirm the node type (LLM / Tool / Set / Pause / Review …) is shown correctly
3. Fill the type's core parameters: prompt for LLM, executor + inputs for Tool, variable + expression for Set
4. Optionally override the node's provider/model in the drawer (see per-node override)
5. Close the drawer and save so the config writes back to the workflow

## EXAMPLE / EXERCISE

Write assignment logic into an LLM node: select it to open the drawer and fill the prompt with

```
You are a task-assignment assistant. Given task titles and a team skills table,
pick the best owner for each task and output a JSON array [{taskId, assignee}].
```

Then confirm the model as Kimi 2.5 (the workflow default) in the drawer, close, and save.

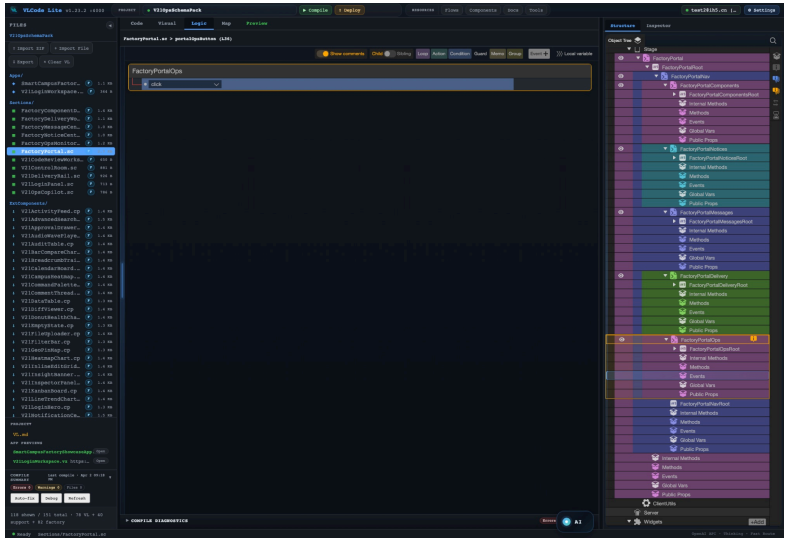
## PITFALL

Writing a vague prompt in the drawer (no specified output format), so a downstream Tool or Set node can't parse the result. When prompting an LLM node, explicitly require structured output (e.g. JSON) for easy downstream consumption.

## CHECKPOINT

You can open any node's detail drawer, understand its parameters, and fully configure an LLM node's prompt and model

# Author Conditional Logic In VisualLogic



VisualLogic event editor — graphical if/then/switch

## CORE IDEA

VisualLogic is a graphical if/then/switch editor node: you draw branching logic on the canvas and it compiles to runEvent(ctx). It lets you express 'if the condition holds take this edge, otherwise take another' control flow without writing code, which is ideal for value-based routing inside a workflow. Unlike a Set node, VisualLogic is about branching and flow control, not computing a single value.

## OPERATIONS

1. Place a VisualLogic node on the canvas and open its graphical editor
2. Build conditions with if/then/switch blocks that reference upstream variables
3. Wire a different downstream edge from each branch for each execution path
4. Save so the node compiles to runEvent(ctx)
5. At run time, watch Detail Log to see which branch actually fired

## EXAMPLE / EXERCISE

Route by amount in an approval workflow: use VisualLogic to test the expense amount

```
if  amount > 5000   then → go to Review node (manager approval)
else                then → go to Set node (auto-approve)
```

Large amounts go to human Review, small ones auto-pass, and the branch is obvious on the canvas.

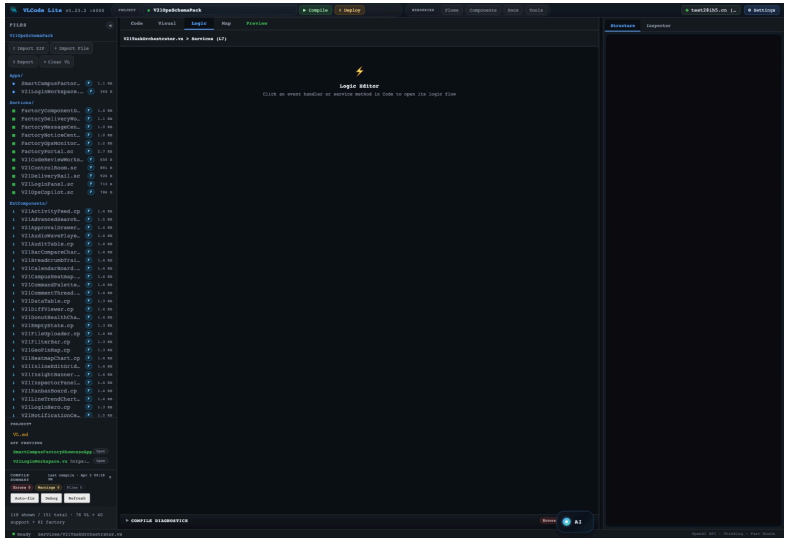
## PITFALL

Forcing a numeric-computation need into VisualLogic — it is for branching control; computing a single expression value belongs in a Set node. Mixing the two makes the logic hard to read and maintain.

## CHECKPOINT

You can draw a value-based if/else route in VisualLogic and verify at run time, via Detail Log, that the correct branch fired.

# Write Computed Values With Set Nodes

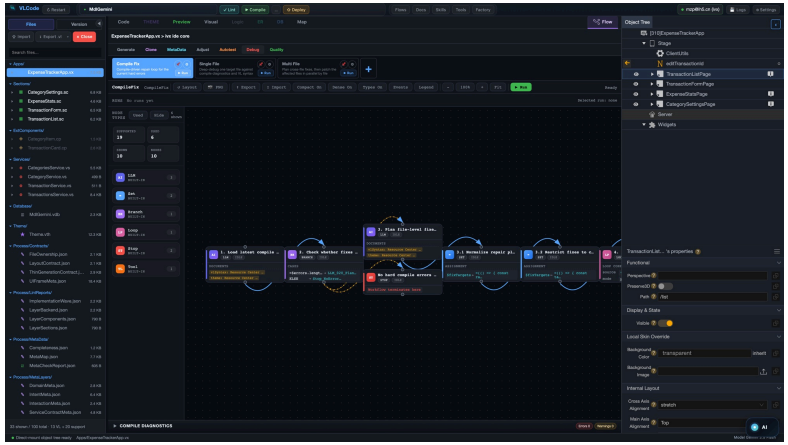


Formula node editor — a single computed expression

## CORE IDEA

(Content pending.)

# Override Provider/Model Per Node



A node detail drawer open over the DAG

## CORE IDEA

A workflow defaults to the workflow model chosen in Settings (the cost-sensitive default is Kimi 2.5), but you can override it in a single LLM node's detail drawer to give that one step a different provider/model. This lets you spend a cheap model on bulk, fault-tolerant steps and reserve a stronger model for the critical reasoning step, while still keeping the same provider where possible to avoid splitting credentials and billing.

## OPERATIONS

1. Select the LLM node to override and open its detail drawer
2. Change the default in that node's provider/model setting
3. Pick a stronger model for 'critical reasoning' and keep a cheap one for 'bulk drafting'
4. Confirm that provider's key is ready in Settings or the SecretRoot vault
5. Save and run, then verify in Detail Log / status bar which model the node actually used

## EXAMPLE / EXERCISE

Mix models in a generation workflow: a cheap model for drafting, a stronger one for final review.

```
LLM #1 (draft sections)   provider=deepseek-api  model=DeepSeek V4 Pro
LLM #2 (final review)     provider=anthropic-api model=Claude (overrides default)
```

Override only the critical node #2; the rest stay on the workflow default.

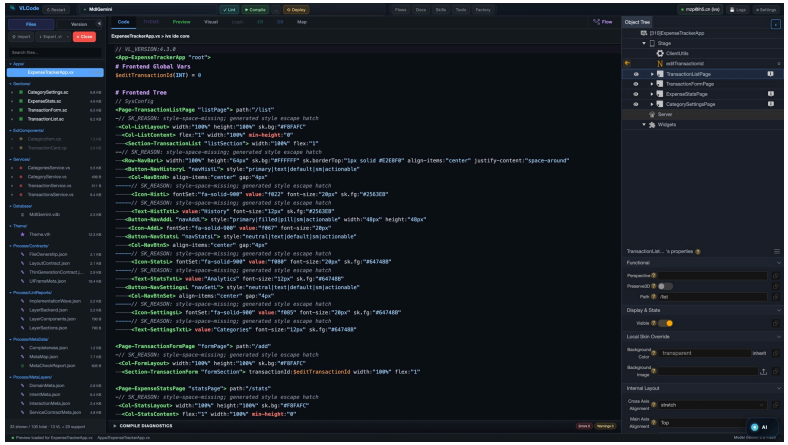
## PITFALL

Carelessly mixing providers from different companies across nodes, scattering credentials, billing, and rate limits. Unless deliberately mixing, keep one provider/company and override only the model on nodes that truly need it.

## CHECKPOINT

You can give a single LLM node a model different from the workflow default, and at run time that node provably uses the overridden model.

# Send Node Selection To Chat Context



The floating AI Assistant panel and chat console

## CORE IDEA

The floating AI Chat panel can take the node(s) you select on the canvas as context, so your natural-language instruction points precisely at that node instead of describing 'the third LLM node on the canvas.' Sending the node selection into Chat context is the key to 'point on the canvas, edit by talking' — whoever is selected is who Chat speaks to.

## OPERATIONS

1. Select a node or group of nodes in the Workflow Editor
2. Send the selection into the floating AI Chat context
3. Give a natural-language instruction about 'this node' in Chat
4. Confirm Chat's response/action targets the node you selected
5. Verify in Detail Log that the change landed on the correct node

## EXAMPLE / EXERCISE

Select the vaguely written assignment LLM node, send it into Chat context, and just say:

```
Change this selected node's prompt to require strict JSON output: [{taskId, assignee, reason}]
```

Because the node is already in context, Chat knows which one 'this node' is — no need to describe its position.

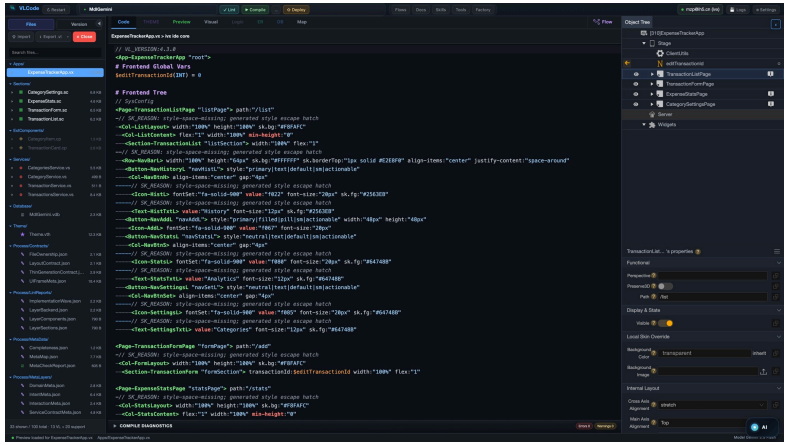
## PITFALL

Saying 'fix that LLM' without sending a node into context, so Chat can't tell which one and edits the wrong node. Select on the canvas and send to context first so the instruction has a clear target.

## CHECKPOINT

You can select a node, send it to Chat context, and have a single natural-language sentence precisely edit that node.

# Add, Delete, And Inspect Nodes Via Chat



The floating AI Assistant panel and chat console

## CORE IDEA

Beyond dragging on the canvas, you can add, delete, and inspect nodes directly through the floating AI Chat: say 'add a Tool node that runs VL compile,' 'delete that dead branch,' or 'what is this node configured with,' and Chat edits the DAG or reads node info back. In studio dev mode, intents like dev_list_nodes / dev_inspect_code list nodes and inspect node code. This path lets you move freely between canvas editing and conversational editing.

## OPERATIONS

1. Open the floating AI Chat and, if needed, send the relevant node into context first
2. Issue add/delete instructions in natural language, e.g. 'add a Write node after the LLM'
3. Use query intents to inspect nodes, e.g. list nodes or inspect a node's code
4. Confirm on the canvas that the added/removed nodes and edges match intent
5. Run structural validation after the change to confirm no dangling edges

## EXAMPLE / EXERCISE

Use Chat to add a persistence step and inspect the result:

```
User: Add a Write node after the assignment LLM that writes results to Process/assignments.json
User: List all nodes in the current workflow
```

The first line has Chat add and wire the node; the second (a dev_list_nodes-style intent) lists nodes to confirm the topology.

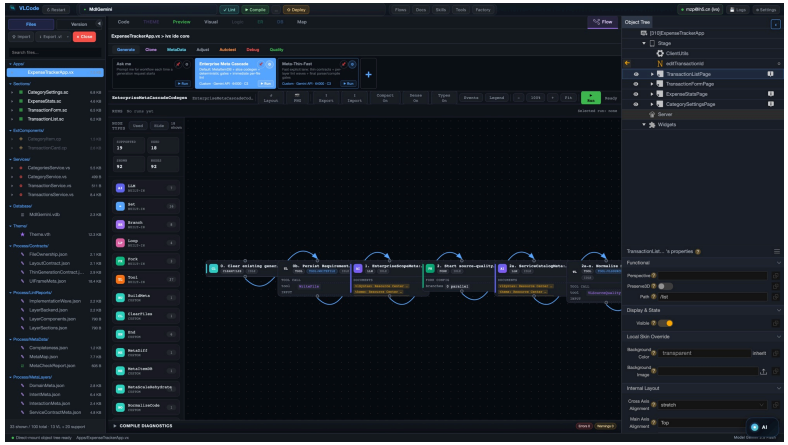
## PITFALL

Bulk-editing via Chat without returning to the canvas to check — Chat may wire an edge to an unintended node. After every structural change, return to the Workflow Editor to view topology and run structural validation.

## CHECKPOINT

You can add and delete nodes and list workflow nodes purely through Chat, and afterward the canvas topology is correct with no dangling edges.

# Structural Validation And Dangling Edges



Workflow Editor — nodes and directed edges on the DAG canvas

## CORE IDEA

Structural validation checks the DAG's topology before a run: whether both ends of every edge connect to real nodes, whether there are dangling edges (an edge with one end unconnected), and whether any node is missing required input. It is the 'root-cause-first' discipline applied to the canvas — fix the graph itself rather than letting a run fail halfway. Dangling edges are the most common structural problem, usually from deleting a node but leaving its edge.

## OPERATIONS

1. After placing and wiring, or after any add/delete/edit, trigger structural validation
2. Read the report for listed dangling edges or nodes missing input
3. Return to the canvas to locate the offending edge/node and fix it
4. Re-validate until the topology is clean
5. Only proceed to run once validation passes

## EXAMPLE / EXERCISE

After deleting an intermediate LLM node but forgetting its outgoing edge, structural validation reports:

```
ERROR: dangling edge — source node 'LLM_assign' not found (was deleted)
```

Fix: return to the canvas and delete the source-less edge, or rewire it to the correct upstream node, then re-validate to green.

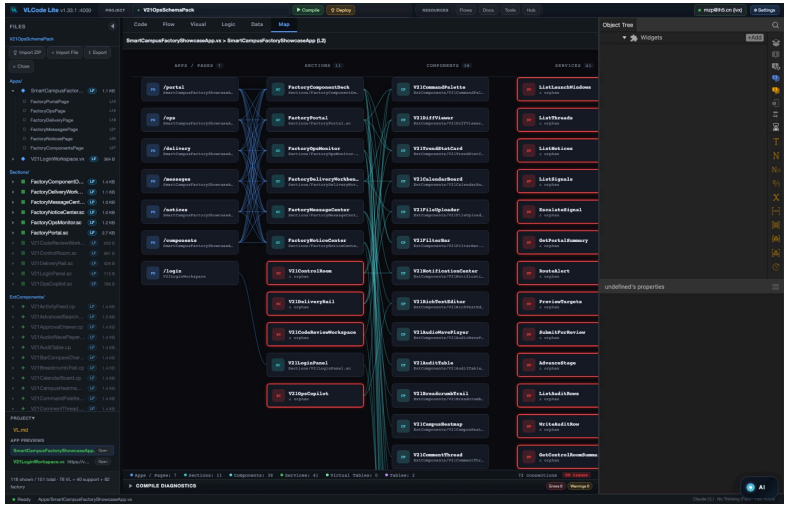
## PITFALL

Skipping structural validation and hitting run, only to fail mid-flow when a node has no input — wasting a whole run. Treat structural validation as a mandatory pre-run gate.

## CHECKPOINT

You can have structural validation report a dangling edge, locate and fix it, and run only after validation is fully green.

# Workflow Factory: Compile A DAG From Units



ASSETS / UNITS / FLOW TREE rails and the Unit editor

## CORE IDEA

Beyond wiring nodes one by one on the canvas, you can go Unit-first: define workflow Units and their I/O contracts in human-readable form in the UNITS rail, then have the workflow factory compile those Units into an executable DAG. This mirrors Agent App Studio's three-layer model — Unit editor + Unit Map (human-readable logic) + DAG Map (the compiled execution graph, whose nodes carry N# badges). When you want to state the business logic clearly first and let the system render the execution graph, this higher-level path is steadier.

## OPERATIONS

1. Create workflow Units in the UNITS rail, stating intent and I/O contracts first
2. Organize the logical relationships between Units on the Unit Map (human-readable view)
3. Trigger compilation of the Units into a DAG, producing the lower DAG Map execution graph
4. On the DAG Map, match the compiled nodes (with N# badges) back to their Units
5. Run structural validation on the compiled result before proceeding to run

## EXAMPLE / EXERCISE

Define two Units for an approval app, then compile to a DAG:

```
Unit: IntakeRequest   in: {form}      out: {request}
Unit: ManagerReview   in: {request}   out: {decision}
```

After compiling, the DAG Map shows the corresponding Pause/Review nodes (with N# badges) already wired per the Unit contracts.

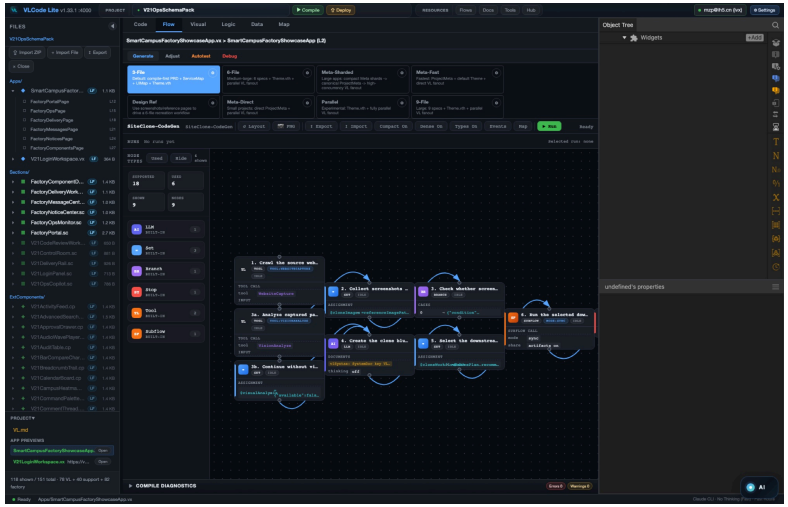
## PITFALL

Writing incomplete Unit I/O contracts (missing an input or output), so the compiled DAG has unconnectable edges or input-starved nodes. Pin down each Unit's in/out contract before compiling.

## CHECKPOINT

You can define workflow Units with I/O contracts in UNITS, compile them into a DAG with N# badges, and pass structural validation

# AutoTest Generation



Flow Mode — workflow picker, codegen, autotest

## CORE IDEA

AutoTest is a capability in Flow Mode alongside codegen and adjust: it auto-generates tests for the workflow you just authored, turning 'does this flow run as expected on typical input' into executable, repeatable verification instead of manual clicking. It embodies the 'verified, not hopeful' principle — a workflow shouldn't merely connect, it should be tested to truly produce the expected result. The matching seed workflow is autotest-pipeline.

## OPERATIONS

1. Switch the center canvas to Flow Mode (workflow picker + codegen + autotest + adjust)
2. Select the workflow to test and trigger AutoTest generation
3. Run the generated tests and watch step events and assertion results in Detail Log
4. For failing cases, return to the canvas to locate the node, fix it, and retest
5. Keep passing tests as a regression baseline and rerun after changing the workflow

## EXAMPLE / EXERCISE

Generate an AutoTest for `TaskAutoAssign` to verify the assignment output's shape:

```
In Flow Mode select TaskAutoAssign → AutoTest →
Assert: the assignment LLM node's output is a JSON array, each item has taskId and assignee
```

After the test runs, Detail Log shows the node output passing the assertion, ready as a regression baseline.

## PITFALL

Treating a workflow as passing just because it reached 'done', without using AutoTest to verify the output content is correct. A terminal state of done does not mean the result is right — confirm actual output with AutoTest assertions.

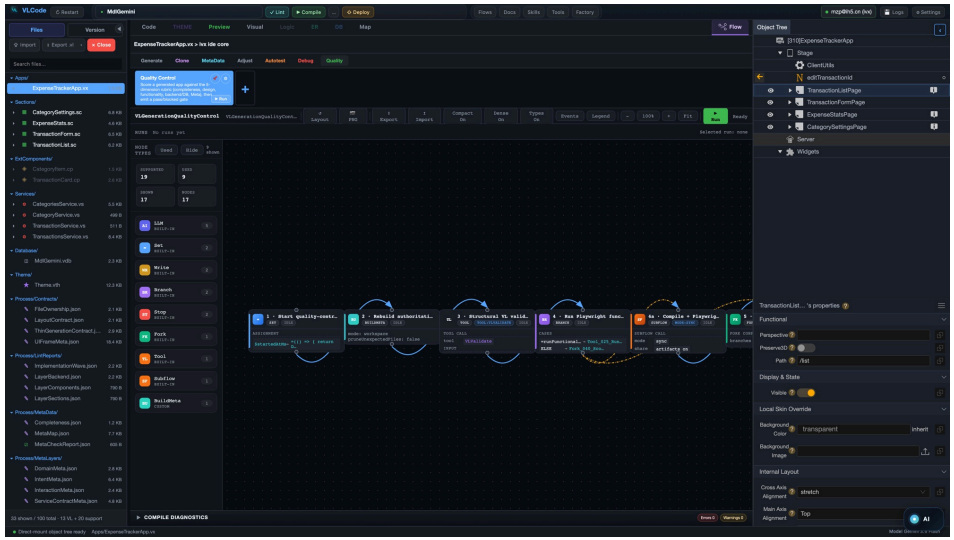
## CHECKPOINT

You can generate and run an AutoTest for a workflow in Flow Mode, see assertion results against node outputs, and judge from them whether the workflow is truly correct.

CHAPTER

# Part VI: Generating Agent Apps (The Core Capability)

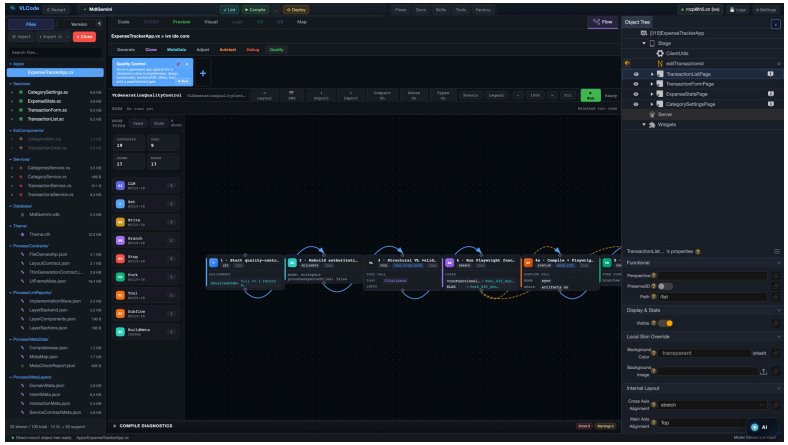
- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



Agent App Studio — Unit Map over the compiled DAG Map

PART VI: GENERATING AGENT APPS (THE CORE CAPABILITY)

# How VL Agent Generates Agent Apps



Agent App Studio — Unit Map over the compiled DAG Map

## CORE IDEA

VL Agent's signature power is not hand-authoring workflows but GENERATING a runnable Agent App from a natural-language intent — you describe what you want and a generation Flow produces a manifest plus an entry workflow (a .vflow). The generated app ships with human-interaction gates (Pause/Review), a control plane, and a run ledger, ready to run on first launch. VL Agent offers several generation lanes (quick / guided / cascade / with-components / scaffold / 3file / factory) spanning a one-liner to enterprise-grade quality gates.

## OPERATIONS

1. Switch the center canvas to Flow Mode and use the workflow picker to find the generate-agent-app-* family
2. In the floating AI Chat, write your app intent in natural language (what it does, what it collects, who approves)
3. Pick a generation lane (quick / guided / cascade / with-components) and click Run once
4. Track the generated intermediate artifacts in Detail Log (solution shape, manifest plan, file generation, validation)
5. When generation finishes, open App Home to see the DAG + manifest, then /start to test-run the new app

## EXAMPLE / EXERCISE

Pick generate-agent-app-guided in AI Chat and type the intent:

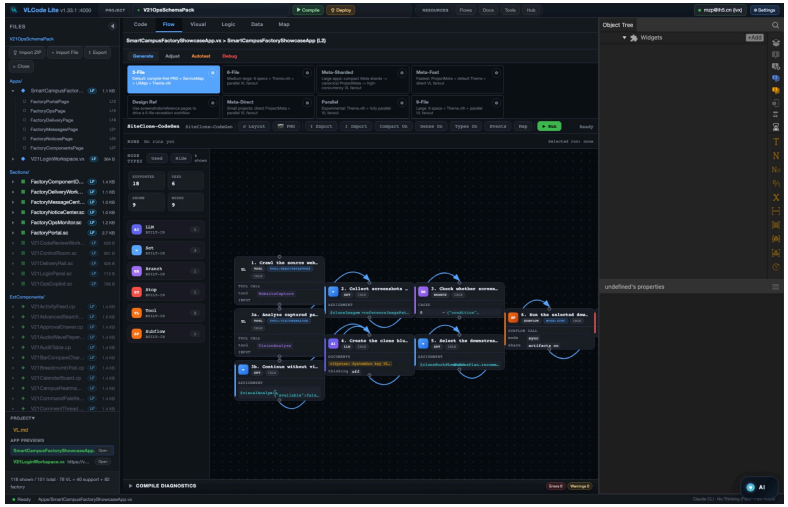
```
"Generate a procurement-approval Agent App: an employee submits a purchase request
(amount, purpose, vendor); anything over 5000 goes to manager approval; on approval
write the record and notify the requester"
```

The Flow turns that sentence into an entry workflow with a Pause (submission form) plus a Review (manager decision), and a derivable manifest.

## PITFALL

Beginners treat the generation Flow like an ordinary LLM chat and write a vague intent ("make an approval app"). Include input fields, decision conditions, who gates, and the output so the derived manifest and gates fit the real process.

# One-Line Quick Generate (generate-agent-app-quick)



Flow Mode — workflow picker, codegen, autotest

## CORE IDEA

generate-agent-app-quick is the fastest lane: no gates, a single LLM pass turns the intent directly into a runnable Agent App and deterministically normalizes the manifest. It is ideal for getting a working prototype on screen first and iterating later — one sentence in, a manifest + entry workflow out.

## OPERATIONS

1. Pick generate-agent-app-quick in the Flow Mode workflow picker
2. Write one complete intent in AI Chat, including inputs and outputs where you can
3. Click Run once and wait for the single LLM pass to produce the entry workflow and a normalized manifest
4. Confirm in Detail Log that the manifest was deterministically normalized (no dangling fields)
5. Open App Home and /start to test-run the prototype immediately

## EXAMPLE / EXERCISE

```
Pick generate-agent-app-quick and type:
"Generate a meeting-room booking Agent App: enter date, time slot, headcount;
on submit return whether it can be booked"
```

Within seconds you get RoomBooking.vflow (entry workflow + a normalized app-block manifest), runnable with no mid-run confirmation.

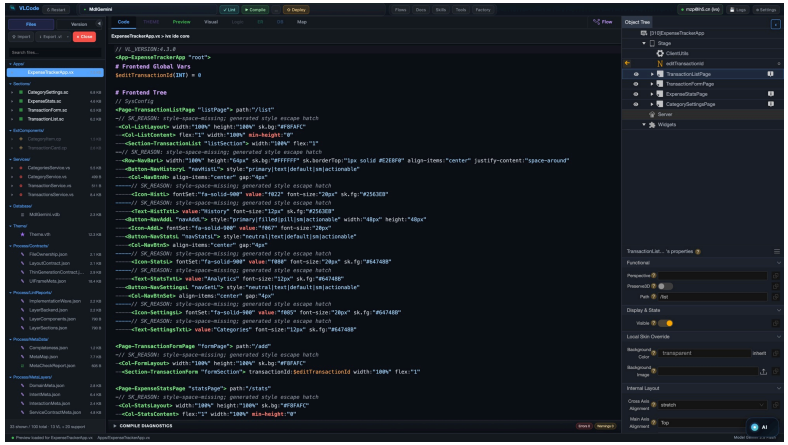
## PITFALL

Quick has no quality gate — do not use it for complex apps with compliance/approval needs, since it runs in one pass without review. When you need to gate, switch to guided or cascade.

## CHECKPOINT

Using generate-agent-app-quick you produce a /start-runnable Agent App from one intent, with no human gate in the process.

# Guided, Human-In-The-Loop Generation (generate-agent-app-guided)



The floating AI Assistant panel and chat console

## CORE IDEA

generate-agent-app-guided is "the headline Flow that generates an App": general-purpose and human-in-the-loop, it turns intent into a runtime-first Agent App — a manifest + entry workflow with Pause/Review gates. It confirms direction with interaction cards at key points, so results fit expectations; it is the default lane for most production apps.

## OPERATIONS

1. Pick generate-agent-app-guided in Flow Mode and write the intent in AI Chat
2. Click Run; in the planning stage the Flow surfaces Review/confirmation cards (driven by the ask_user chat event) to confirm scope
3. On a card, approve / request_changes, or fill in missing details to continue
4. Generate the entry workflow (with Pause/Review gates) and a derivable manifest
5. In App Home inspect the manifest and DAG, then /start to verify the human gates fire as expected

## EXAMPLE / EXERCISE

```
Pick generate-agent-app-guided and type:
"Generate an employee-expense approval Agent App: submit amount/category/invoice,
finance does a first review, the manager does the final review, and both must pass to book it"
```

Mid-generation a Review card appears asking you to confirm "finance + manager two-stage approval"; only after you approve does it lay down the two Review nodes.

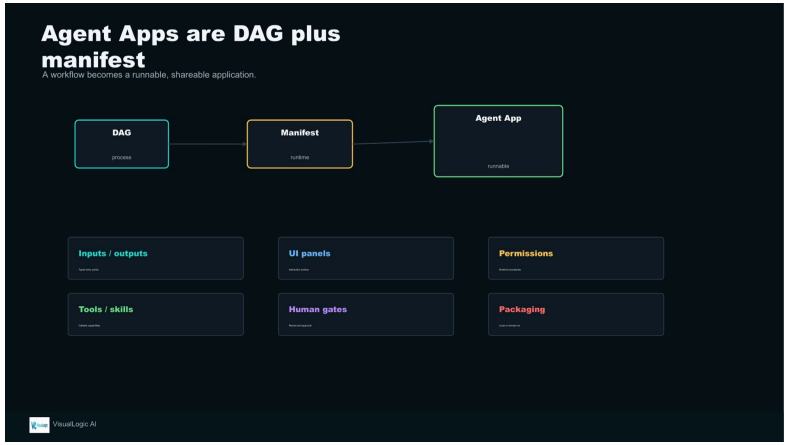
## PITFALL

Clicking approve on mid-run cards without reading them — guided's value is correcting direction at those gates, so blind approvals let generation drift from intent. Read each card fully before deciding.

## CHECKPOINT

Using generate-agent-app-guided you generate an Agent App that surfaced at least one interaction card during planning and landed Pause/Review gates.

# Enterprise Quality-Gated Generation (generate-agent-app-cascade)




Master Agent — branch scoring, pruning, and promotion

## CORE IDEA

generate-agent-app-cascade is the Enterprise Cascade, quality-gated lane: it splits generation into a staged planning cascade — solution shape → product brief → requirement bundle → interaction contract → surface contract → workflow… — and each stage emits a reviewable intermediate contract. This gates large, complex, audit-bound Agent Apps during generation rather than reworking after a one-shot pass.

## OPERATIONS

1. Pick generate-agent-app-cascade in Flow Mode and write an enterprise-level intent
2. Click Run; the cascade emits solution shape, product brief, requirement bundle, and so on in order
3. At each quality gate review the intermediate contract on a Review card, then approve to proceed
4. At the surface contract / workflow stage confirm the exposed interaction surfaces and nodes
5. When finished, compare planned vs actual in the Manifest Inspector for consistency

## EXAMPLE / EXERCISE

```
Pick generate-agent-app-cascade and type:
"Generate a vendor-onboarding approval Agent App: qualification intake, compliance
pre-screen, legal review, procurement sign-off, fully audit-traced end to end"
```

The cascade first emits a product brief for review, then a requirement bundle and interaction contract, approving gate by gate before generating the entry workflow.

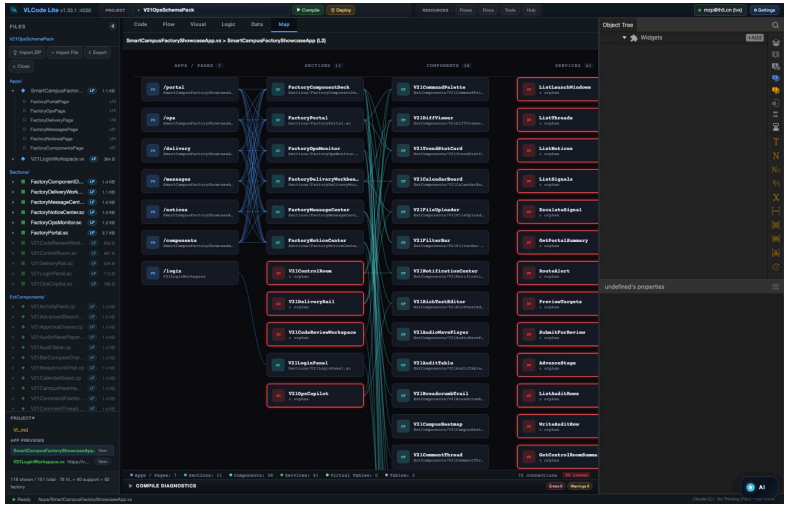
## PITFALL

Approving all the way through before reading the intermediate contracts degrades cascade into a slower quick. Its value is the per-stage contract review; skipping gates buries structural problems in large apps.

## CHECKPOINT

Using generate-agent-app-cascade you generate an enterprise Agent App that passed multiple quality gates and produced intermediate contracts like solution shape / requirement bundle.

# Generate With The Component Factory



ASSETS / UNITS / FLOW TREE rails and the Unit editor

## CORE IDEA

generate-agent-app-with-components generates the app AND equips it from the local Component Factory library — it filters the component catalog and selects pre-built components that match the app's interaction surfaces. The resulting Agent App has not just logic but ready-made UI units, saving you from hand-building the interface.

## OPERATIONS

1. Pick generate-agent-app-with-components in Flow Mode and write an intent that includes UI needs
2. Click Run; the Flow plans the app first, then queries the Component Factory for matching components
3. In Detail Log watch it filter the component catalog and pick table / form / approve components
4. Generate the entry workflow + manifest and register the selected components in manifest assets
5. In App Home / Studio check that the equipped UI units match expectations

## EXAMPLE / EXERCISE

```
Pick generate-agent-app-with-components and type:
"Generate a customer-ticket Agent App: a list of tickets, a form to create a ticket,
a detail page with an approval button"
```

The Flow picks table-generic (ticket list), form-generic (create form), and approve-decision (approval button) from the Component Factory and equips the app with them.

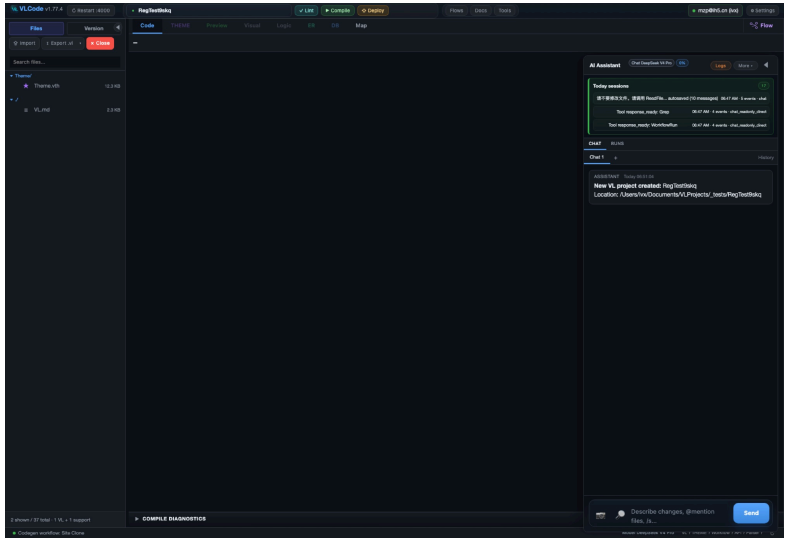
## PITFALL

If the intent omits UI shape (list/form/approval), component filtering has nothing to match and the app degrades to pure logic. Name the interface shapes you need so the Component Factory can hit them.

## CHECKPOINT

Using generate-agent-app-with-components you generate an Agent App whose manifest assets register at least one Component Factory component.

# Scaffold From A Description (agent-app-scaffold-main)



Flow Bundle Mode — editing a .vflow directly

## CORE IDEA

agent-app-scaffold-main ("Agent App Scaffold") creates a complete Agent App from a description: it generates a manifest, an entry workflow, runtime directories, and an optional service / VL surface. It adds a "project scaffolding" dimension on top of generation — not just the workflow, but the directory skeleton and registration the app needs to run.

## OPERATIONS

1. Pick agent-app-scaffold-main in Flow Mode and write an app description in AI Chat
2. Click Run; the Flow generates the manifest and entry workflow
3. The Flow creates the runtime directory skeleton and, on demand, a service / VL surface
4. In Detail Log watch the Write nodes persist the scaffold files
5. In the ASSETS rail confirm the app files and directory structure are in place

## EXAMPLE / EXERCISE

```
Pick agent-app-scaffold-main and type:
"Scaffold a CustomerCRM lead-intake Agent App: capture leads, auto-dedupe,
sales claims a lead, with a simple service backend"
```

The Flow generates CustomerCrmIntake.vflow (manifest + entry workflow), runtime directories, and a service surface for the dedupe decision.

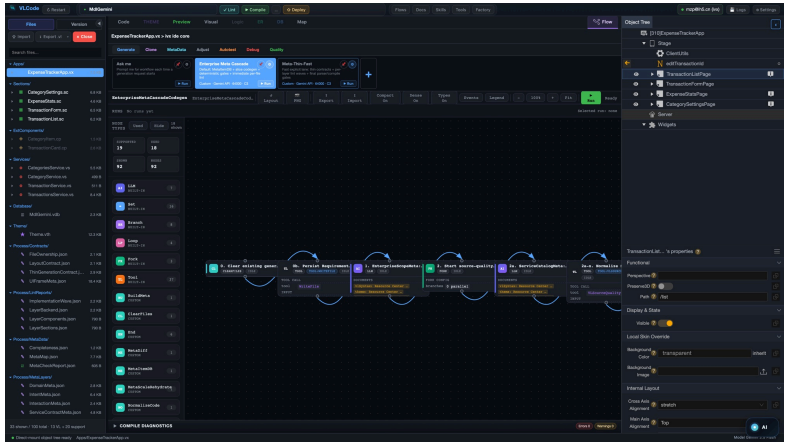
## PITFALL

Treating scaffold as generating only a single workflow file — it also builds runtime directories and an optional service; deleting those by hand breaks the app. Keep the full directory output the scaffold produces.

## CHECKPOINT

Using agent-app-scaffold-main you produce a complete Agent App skeleton with a manifest, entry workflow, and runtime directories from one description.

# 3-File Staged Generation (agent-app-3file-main)



Workflow Editor — nodes and directed edges on the DAG canvas

## CORE IDEA

agent-app-3file-main is a VLC-style staged chain: product brief → runtime surface spec → workflow contract → manifest plan → file generation → validation → scaffold/register. It splits generation into ordered contract stages, feeding each stage's output to the next, then lands and registers the app. It is more controlled than quick and lighter than cascade — right for medium-complexity apps.

## OPERATIONS

1. Pick agent-app-3file-main in Flow Mode and write the app intent
2. Click Run; receive product brief, runtime surface spec, and workflow contract in order
3. The Flow derives a manifest plan from those, then does file generation
4. At validation confirm the structure has no dangling edges, then proceed to scaffold/register
5. In Detail Log track each stage's output and check the workflow contract matches the final DAG

## EXAMPLE / EXERCISE

```
Pick agent-app-3file-main and type:
"Generate a leave-approval Agent App: an employee enters leave days and reason,
the supervisor approves, and anything over 3 days adds an HR review"
```

The Flow emits a product brief, then a workflow contract (the two-Review condition), then a manifest plan, finally generating and registering LeaveApproval.vflow.

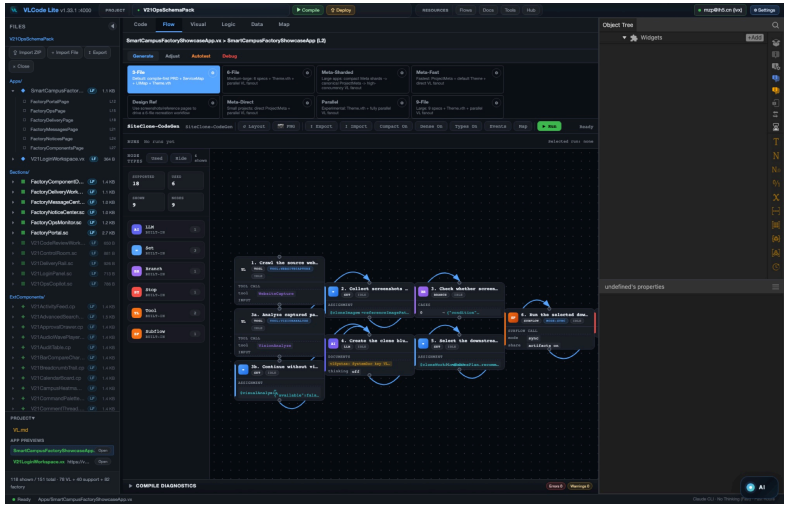
## PITFALL

Skipping the intermediate workflow contract and jumping to the final DAG — if the contract stage gets a condition wrong (e.g., misses the 3-day threshold), file generation faithfully executes the error. Verify business rules at the contract stage.

## CHECKPOINT

Using agent-app-3file-main you can see the ordered stages product brief → workflow contract → manifest plan → file generation and obtain a registered Agent App.

# The Factory Master Workflow (agent-app-factory-main)



Flow Mode — workflow picker, codegen, autotest

## CORE IDEA

agent-app-factory-main is the single official "mother" workflow for Agent App generation: intent → planning artifacts → validation → generation → scaffold. When you are unsure which lane to use, it is the safe unified entry — it internally coordinates planning, validation, and scaffolding, converging one intent into a registered Agent App. The quick/guided/cascade/3file lanes are specializations for specific scenarios.

## OPERATIONS

1. Pick agent-app-factory-main in Flow Mode as the unified generation entry
2. Write the intent in AI Chat and click Run
3. The Flow walks planning artifacts → validation → generation → scaffold in order
4. In Detail Log watch the validation gate catch structural problems
5. When finished, view the registered app in App Home and /start to test-run

## EXAMPLE / EXERCISE

```
Pick agent-app-factory-main and type:
"Generate a content-moderation Agent App: submit copy, AI does a first pass,
a human reviews flagged items, and on pass it publishes"
```

The factory mother workflow runs planning → validation → generation → scaffold and produces ContentModeration.vflow with an LLM first-pass node plus a Review human-check gate.

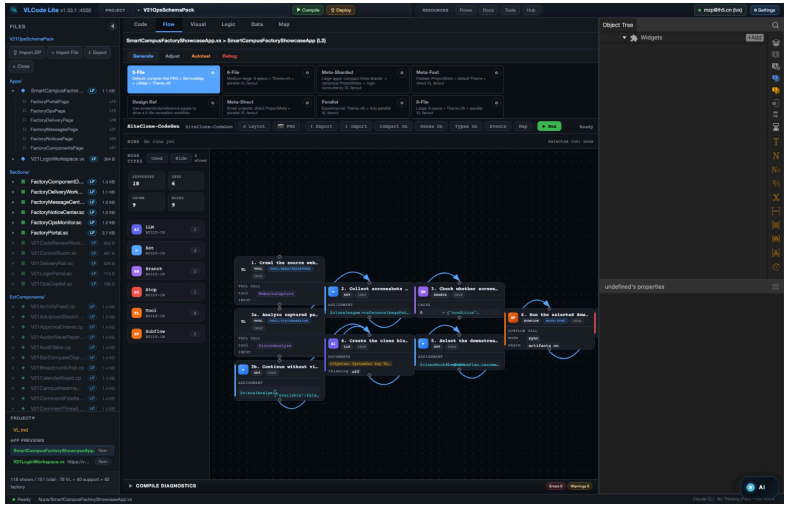
## PITFALL

Assuming the "mother" workflow is stronger than the specialized lanes and using it every time — quick is cheaper for pure prototypes, cascade is stricter for enterprise compliance. The mother workflow is the safe unified entry, not always the optimum.

## CHECKPOINT

You can explain that agent-app-factory-main is the unified official generation entry and walk it through planning → validation → generation → scaffold to get an Agent App.

# Generating VLCode-Bound Workflows (workflow-factory)



Resource Center — registering and syncing a .vflow

## CORE IDEA

workflow-factory-main is a VL Agent Agent App that GENERATES new reusable workflows from intent — this is precisely how you produce the .vflow workflows that ship to VLCode. In other words: build a .vflow with workflow-factory in VL Agent, register it to the Resource Center, and a VLCode developer imports the SAME .vflow to drive enterprise app generation. This is the generate-in-VL-Agent → Resource Center → VLCode chain made concrete.

## OPERATIONS

1. Pick workflow-factory-main in Flow Mode and write the reusable-workflow intent
2. Click Run to generate a new workflow (a .vflow bundle: workflow JSON + manifest)
3. Test-run it with Run and debug with Pause/Resume
4. Save as .vflow and register to the local Resource Center (.vl-code/registry/workflows/<id>.json)
5. After Resource Center cloud sync, a VLCode developer browses / imports / customizes the same .vflow

## EXAMPLE / EXERCISE

```
Pick workflow-factory-main and type:
"Generate a reusable 'add-approval-step' workflow so a VLCode team can insert
an approval stage into any already-generated app"
```

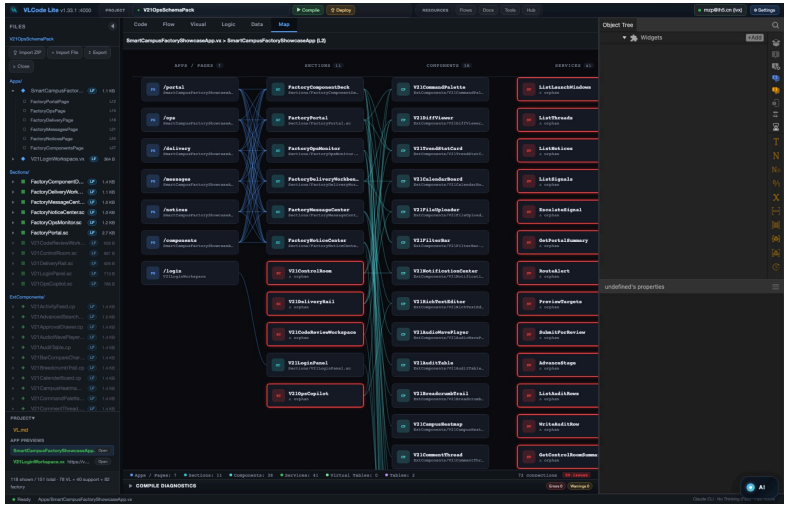
workflow-factory emits AddApprovalStep.vflow and registers it to the Resource Center; a VLCode developer imports it via the Flow panel's "+ Upload .vflow" and categorizes it under Adjust for reuse.

## PITFALL

Building the workflow in VL Agent but forgetting to register it to the Resource Center, so the VLCode side never sees it — generating is not shipping. After Save, register to .vl-code/registry/workflows and trigger cloud sync.

## CHECKPOINT

# Generate From A Paper (paper-to-agent-app)



Map Mode — the metadata and visual bridge

## CORE IDEA

paper-to-agent-app-main turns a research paper into a concrete Agent App: extract conclusions → split into Units → compose a DAG → review. It is for materializing a paper's method or process directly into a runnable, interactive app rather than leaving it as prose. Map Mode's metadata bridge helps you map abstract conclusions to concrete nodes.

## OPERATIONS

1. Pick paper-to-agent-app-main in Flow Mode and provide the paper text or key points
2. Click Run; the Flow first extracts the paper's conclusions
3. Split the conclusions into Units (I/O contracts), then compose them into a DAG
4. At the Review gate inspect how each Unit maps to the DAG
5. In Map Mode compare the paper's method to the generated nodes, confirm nothing is missing, then land the app

## EXAMPLE / EXERCISE

```
Pick paper-to-agent-app-main and type:
"Turn this 'multi-stage resume screening' paper into an Agent App: build an entry
workflow following the paper's three-stage scoring, one scoring node per stage"
```

The Flow extracts the three-stage conclusion → three scoring Units → compiles a DAG → Review checks the scoring logic stays faithful to the paper.

## PITFALL

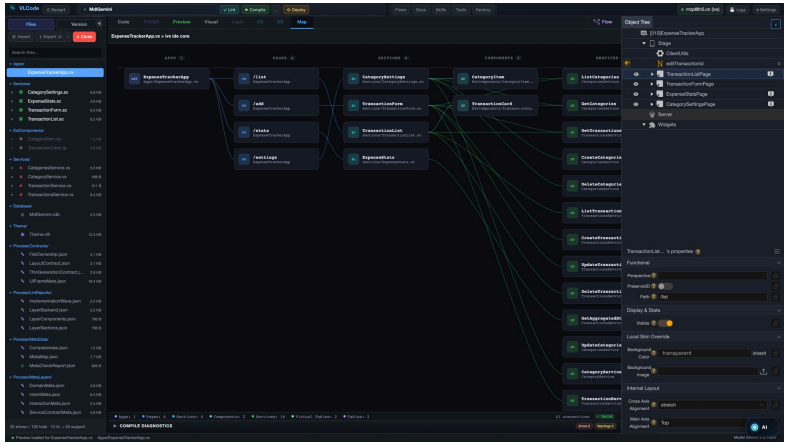
Dumping the whole paper in without highlights, so the extraction stage grabs the wrong thread. Name which method/process to materialize in the intent so extraction stays focused.

## CHECKPOINT

Using paper-to-agent-app-main you extract a paper's method into Units and compose it into a reviewable DAG app.

PART VI: GENERATING AGENT APPS (THE CORE CAPABILITY)

# The Output: Manifest + Entry Workflow



Manifest Inspector — planned versus actual surfaces

## CORE IDEA

Whatever lane you use, VL Agent's output has one shape: a manifest + an entry workflow, together one .vflow. The manifest is not hand-written; it is derived at load time from the top-level app block plus the Pause/Review steps — it declares the planned interaction surfaces, lifecycle, and resource lineage. Understanding this output shape lets you read any generation result.

## OPERATIONS

1. After generation, view the entry workflow's DAG and the derived manifest in App Home
2. Open the Manifest Inspector and compare the planned vs actual exposed surfaces
3. Confirm each Pause/Review node maps to one interaction surface in the manifest
4. In the ASSETS rail confirm the whole output lives in a single .vflow
5. Run the entry workflow with /start and verify the declared surfaces actually appear

## EXAMPLE / EXERCISE

```
After generating the approval app, the Manifest Inspector shows:
entry: approval_workflow
surfaces: [submit-form (Pause N3), manager-review (Review N5)]
lifecycle: install -> release -> runtime
```

The planned and actual columns agree, so the manifest has not drifted from the real nodes.

## PITFALL

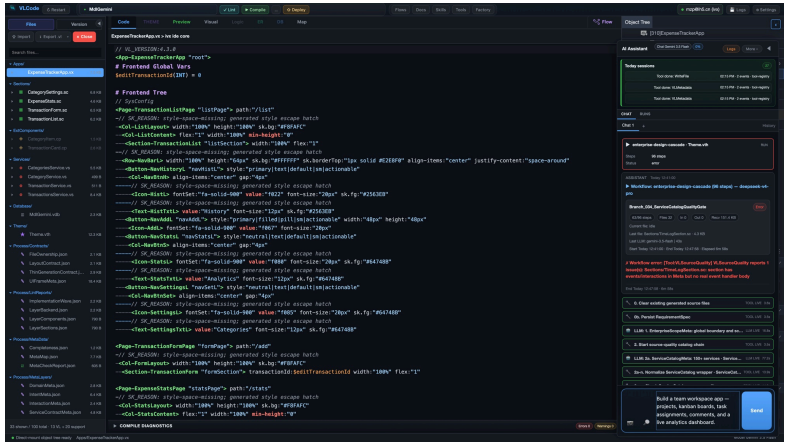
Hunting for an .app.json as if the manifest were a separate hand-written file — AAS v2.2.0 has no standalone manifest; it derives from the app block + Pause/Review. To change a surface, change a node, not an external manifest file.

## CHECKPOINT

In the Manifest Inspector you can point out that the generated manifest derives from the app block and Pause/Review steps, all within one .vflow.

PART VI: GENERATING AGENT APPS (THE CORE CAPABILITY)

# Modify, Debug, Test, Review, Package



Detail Log — the live run ledger and node results

## CORE IDEA

Generation is just the start: VL Agent provides a full lifecycle for generated Agent Apps — agent-app-modify-main (review-gated incremental change), agent-app-debug-main (ledger/run-state/node-result/artifacts/traces), agent-app-test-main (dry-run + smoke test), agent-app-review-main (manifest/workflow/permissions/runtime/control-plane boundary), agent-app-package-main (packages the .vflow app into a portable distribution snapshot + install review + release snapshot + rollback metadata). This keeps a generated app continuously maintainable, verifiable, and safely shippable.

## OPERATIONS

1. To change requirements pick agent-app-modify-main and review the increment at the Review gate before landing
2. When something breaks pick agent-app-debug-main to inspect the run ledger, node results, and error stacks
3. Before release pick agent-app-test-main for a dry-run + smoke test
4. Pick agent-app-review-main to audit permissions and the control-plane boundary
5. Pick agent-app-package-main to package the .vflow app into a portable distribution snapshot with install review, a release snapshot, and rollback metadata

## EXAMPLE / EXERCISE

```
Add an HR spot-check to the generated expense app:
pick agent-app-modify-main, type "add an HR spot-check Review node after manager approval"
→ the Review gate shows the diff → approve → land
then agent-app-test-main dry-run, finally agent-app-package-main builds a portable distribution snapshot
```

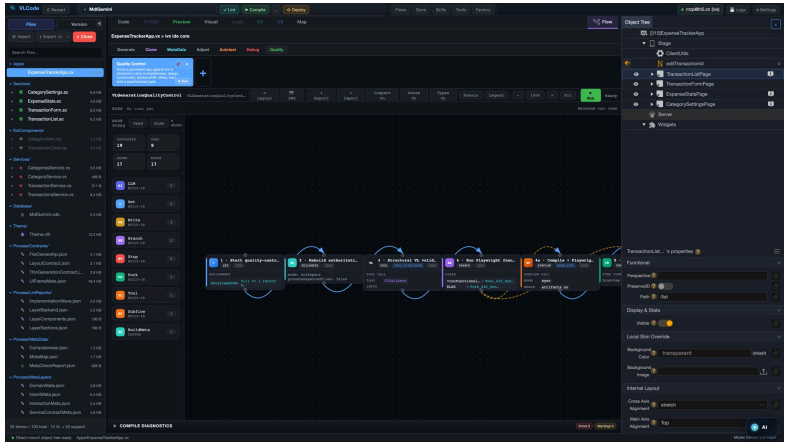
## PITFALL

Packaging and releasing right after a change, skipping agent-app-test-main and agent-app-review-main — without a dry-run and boundary audit, permission/control-plane problems surface only at runtime. Test then review before packaging.

## CHECKPOINT

On a generated Agent App you can run modify → test → review → package in sequence and obtain a portable distribution snapshot with rollback metadata

# Open The Generated App In Studio



Agent App Studio — Unit Map over the compiled DAG Map

## CORE IDEA

After generation, you open the .vflow in Agent App Studio (AAS) to edit and run it. Studio shows the same app in three editing layers: the left Unit editor (I/O contracts + manifest assets), the upper Unit Map (human-readable logic), and the lower DAG Map (the compiled execution graph with N# node badges). You can edit logic from a business view and still see the real underlying execution graph.

## OPERATIONS

1. Switch the center canvas to Flow Bundle Mode and open the generated .vflow
2. In the left Unit editor view/modify each Unit's I/O contracts
3. In the upper Unit Map read human-readable logic and adjust the connections between units
4. In the lower DAG Map see the compiled execution graph and N# node badges
5. Use /start in the floating AI Chat to one-click test-run inside Studio

## EXAMPLE / EXERCISE

```
Open the generated ProcurementApproval.vflow:
Unit editor: submit-unit (inputs: amount/purpose/vendor)
Unit Map: submit -> threshold-check -> manager-review -> write-record
DAG Map: N1 Pause -> N2 VisualLogic -> N5 Review -> N6 Write
```

The three views point at different representations of the same app.

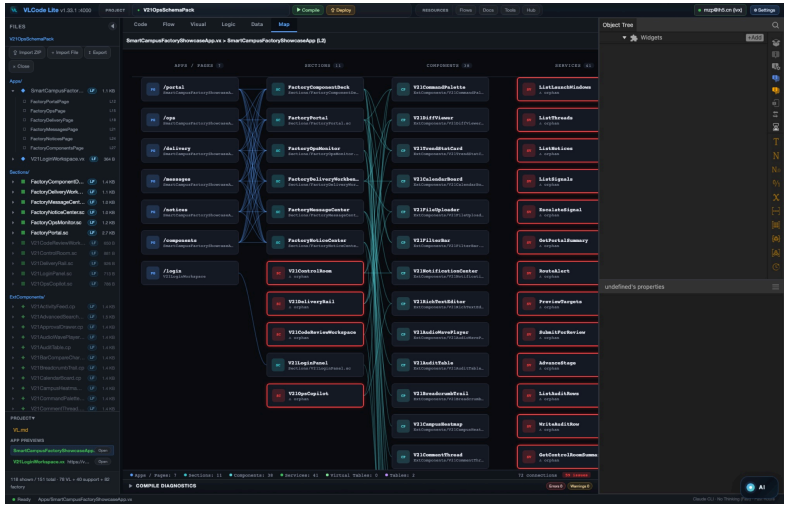
## PITFALL

Hand-editing nodes only in the DAG Map without syncing contracts back in the Unit editor makes the Unit Map and DAG drift. Edit logic in the Unit layer first and let it compile down to the DAG.

## CHECKPOINT

You can open a generated .vflow in Flow Bundle Mode and switch among the Unit editor / Unit Map / DAG Map layers.

# Unit Editor And Unit/DAG Map



ASSETS / UNITS / FLOW TREE rails and the Unit editor

## CORE IDEA

Three-layer editing is the heart of AAS: the Unit editor defines each unit's I/O contracts and manifest assets; the Unit Map organizes units into human-readable logic on the upper canvas; the DAG Map shows the compiled execution graph (with N# node badges) on the lower canvas. The Unit Map is "the logic you mean" and the DAG Map is "the graph that actually runs," kept consistent by the contracts — this is where Workflow Factory's "compile a DAG from Units" lands.

## OPERATIONS

1. In the Unit editor declare input/output contracts for each Unit to avoid data drift
2. On the Unit Map connect Units in business order into human-readable logic
3. Trigger the compile from Units to a DAG; matching N# nodes appear in the lower DAG Map
4. Compare the Unit Map and DAG Map to confirm each Unit has a corresponding execution node
5. Register the ViewKit components and resources a Unit uses in the manifest assets

## EXAMPLE / EXERCISE

```
Unit editor: review-unit
  in: { request: object }
  out: { decision: 'approve'|'reject'|'request_changes' }
Unit Map: submit -> review-unit -> on-approve
DAG Map: N4 Review (decision) -> N5 Write
```

The review-unit's out contract directly drives the three branches of the Review node in the DAG.

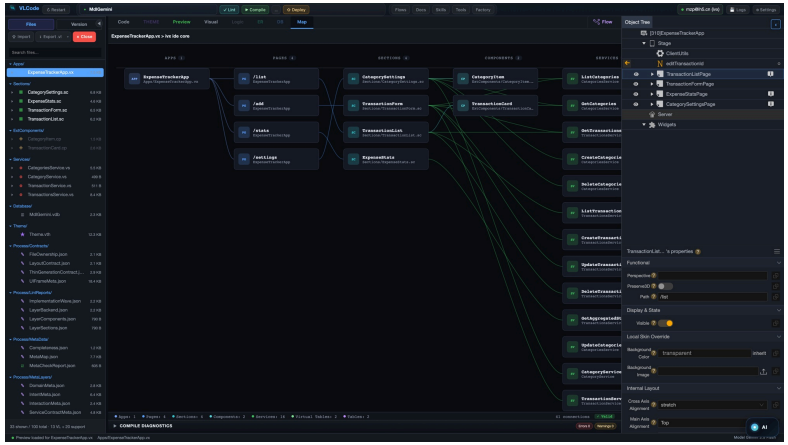
## PITFALL

Writing vague Unit I/O contracts (e.g., only in: data) leaves compiled DAG nodes loosely wired and data mismatched at runtime. Spell out field names and types in the contract.

## CHECKPOINT

You can define clear I/O contracts on a Unit and see it compile into a corresponding execution node in the DAG Map.

# The app Block, Manifest, And Lifecycle



Manifest Inspector — planned versus actual surfaces

## CORE IDEA

The app block is the top-level declaration in a .vflow that turns a workflow into an Agent App: title, entry workflow, exposed surfaces. Studio derives the manifest at load time from the app block plus the Pause/Review steps — there is no separate .app.json. The lifecycle is install → release → runtime, backed by an instance-scoped SQLite control plane (separate from the IDE workspace) that holds the run ledger and state. Understanding the app block is understanding how an app is "raised" from a workflow.

## OPERATIONS

1. Declare the top-level app block in the .vflow (title, entry, surfaces)
2. Add Pause/Review nodes so the manifest gains the matching interaction surfaces automatically
3. Run install to load the app into the instance-scoped control plane
4. Run release to make a release snapshot, entering runtime where it can be run
5. In the Manifest Inspector reconcile planned vs actual and the lifecycle dots

## EXAMPLE / EXERCISE

```
app:
- title: Procurement Approval
- entry: approval_workflow
# Pause N3 -> submit-form surface; Review N5 -> manager-review surface
lifecycle: install -> release -> runtime
```

Only after declaring the app block does App Home show the workflow-first landing of DAG + manifest + run ledger.

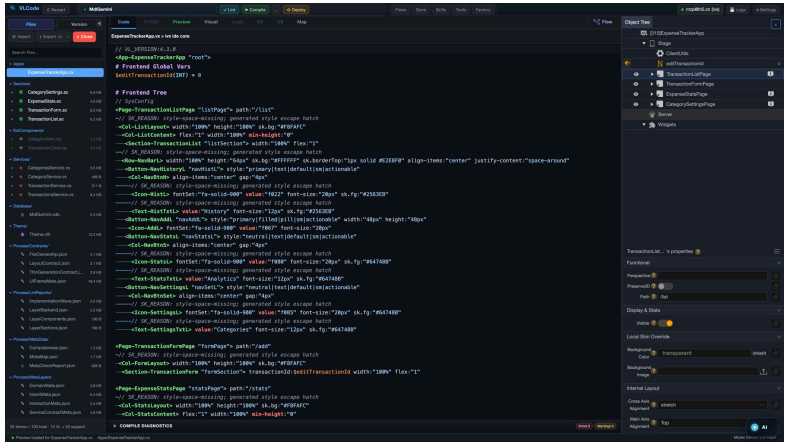
## PITFALL

Running a workflow as an Agent App without declaring the app block — without it no manifest is derived, so App Home and interaction cards never appear. Declare the app block before install/release.



## CHECKPOINT

# Pause / Review Interaction Cards



A Pause/Review interaction card waiting for a human

## CORE IDEA

Pause and Review nodes are the two human-interaction gates in an Agent App, rendered at runtime as Interaction Cards: a Pause node pops a human input form (text/textarea/number/select/checkbox), and a Review node pops an approval card (approve/reject/request_changes). They make a generated app not a script that runs straight through but one that stops at key points to collect input and wait for a human decision — humans gate at the node layer.

## OPERATIONS

1. Place a Pause node in the entry workflow and define the fields to collect (e.g., amount, reason)
2. Place a Review node after the decision with approve/reject/request_changes options
3. When the run hits Pause, fill the form in the app chat console; the node gracefully pauses and waits
4. When the run hits Review, decide on the approval card; the decision lands in the control-plane audit
5. Use /resume to continue the run past the gate

## EXAMPLE / EXERCISE

```
Pause N3 (form):
  amount: number, purpose: textarea, vendor: select
Review N5 (approval card):
  approve | reject | request_changes
```

Running the procurement-approval app first pops a form to collect details, then an approval card for the manager's decision, which is recorded in the control plane.

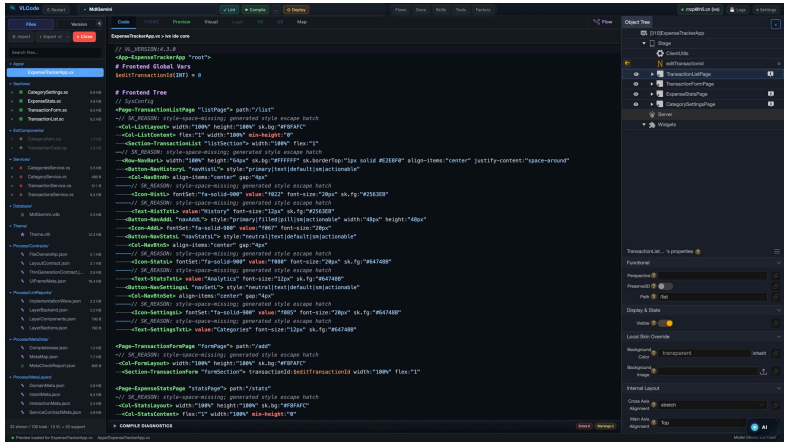
## PITFALL

Reading only the verdict on a Review node but ignoring the request_changes path — offering just approve/reject leaves requests needing more info nowhere to go but repeated rejection. Wire a request_changes branch on Review.

## CHECKPOINT

During a run you can trigger a Pause form card and a Review approval card separately and see the approval decision enter the control plane.

# ViewKit And The App Chat Console



The floating AI Assistant panel and chat console

## CORE IDEA

The ViewKit registry is a set of pre-built reusable UI components (human-task, table-generic, form-generic, approve-decision…) that a generated Agent App draws from instead of hand-building UI. The app chat console is the command surface for running an Agent App: /start /pause /resume /stop /status /rerun-step, driving the whole run of an app instance via natural language or slash commands. One governs "what the UI looks like," the other "how the app is driven."

## OPERATIONS

1. In manifest assets pick ViewKit components for each surface (e.g., approve-decision for a Review surface)
2. In the app chat console use /start to launch an app instance
3. When the run hits a gate, the ViewKit component renders as the matching form/approval card
4. Use /status to check instance state and /pause, /resume to control the run
5. Use /rerun-step when you need to rerun a particular step

## EXAMPLE / EXERCISE

```
Manifest assets:
  submit-form -> form-generic
  manager-review -> approve-decision
App chat console:
  /start
  /status        # see which step it's paused on
  /resume        # continue after the manager approves
```

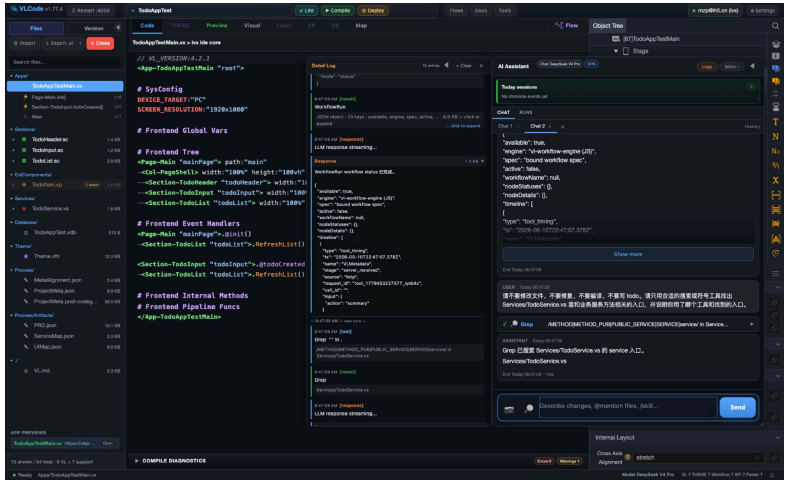
## PITFALL

Confusing the app chat console's /commands with DAG editing — the console drives a run instance, it is not where you change the graph. Edit nodes back on the Studio canvas; control the run with /commands.

## CHECKPOINT

PART VI: GENERATING AGENT APPS (THE CORE CAPABILITY)

# Agent App Home And Built-In Seed Apps



Agent App Home — the workflow-first landing

## CORE IDEA

App Home is an Agent App's workflow-first landing page: DAG + manifest + core data + artifacts + run ledger gathered on one screen, so you view and run the app workflow-first. VL Agent also ships many seed Agent Apps and workflows (150+ seed workflows) as ready starting points — such as approval-flow-runtime-main, bpmn-approval-flow-main, the actor-demo-* series (pm-intake/triage, frontend/backend module, qa-review, release-board), and code-evolution-studio-main — which you can open to learn from or remix.

## OPERATIONS

1. Browse the built-in seed Agent Apps / workflows in the Flow Mode workflow picker
2. Open a seed app (e.g., approval-flow-runtime-main) and view its DAG and manifest in App Home
3. Use /start to run it through and watch the run ledger and artifacts
4. Use agent-app-modify-main to remix it into your own app from that template
5. Reconcile core data, artifacts, and run history on the single App Home screen

## EXAMPLE / EXERCISE

```
Open the seed app actor-demo-pm-intake:
App Home shows the DAG (intake -> triage -> assign) + manifest + run ledger
/start runs a round; the run ledger shows each actor task's timestamps and results
```

Then use agent-app-modify-main to turn it into your team's intake app.

## PITFALL

Treating built-in seed apps as look-only examples instead of reusing them — they are the best starting points, and hand-writing from scratch is slower. Find the closest seed app, then modify it into your own.

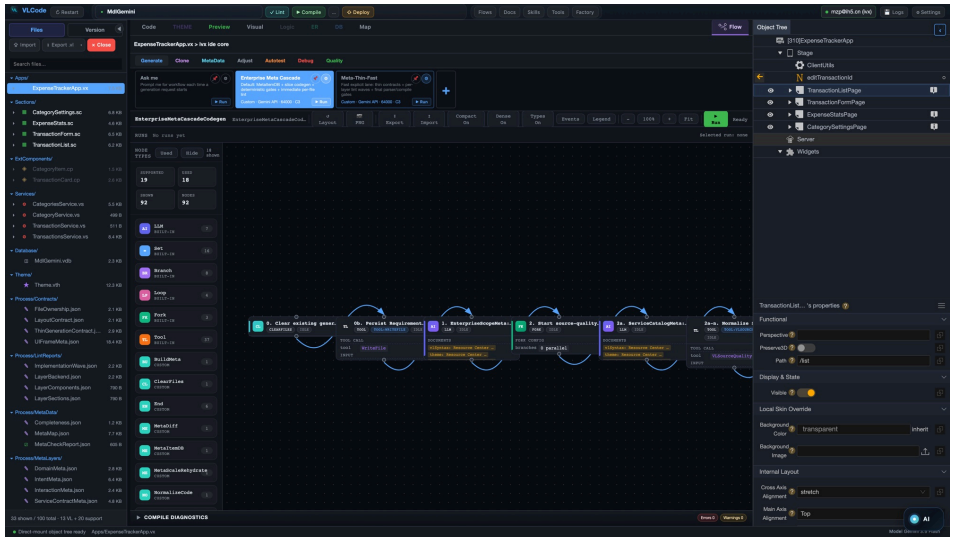
## CHECKPOINT

You can open a built-in seed Agent App in App Home, see its DAG + manifest + run ledger, and /start a run through.

CHAPTER

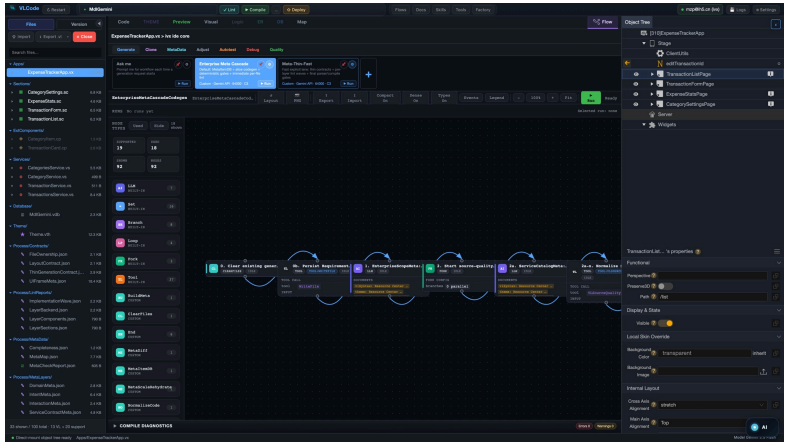
# Part VII: Run, Debug, And The Master Agent

- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



Workflow Editor — nodes and directed edges on the DAG canvas

# Run A Workflow With One Click



Workflow Editor — nodes and directed edges on the DAG canvas

## CORE IDEA

In VL Agent a finished workflow is a DAG, and one click on Run starts it from the entry node and walks the directed edges step by step. The local Workflow-Engine 4.13.1 schedules the run per Workflow Spec 4.1, emitting each node's start/finish/error as vl.workflow.run-event.v4 events, and the run always settles to exactly one terminal state: done, error, or aborted. You do not have to compile the whole project first — runs are local and immediate.

## OPERATIONS

1. Open the workflow you want in the Workflow Editor tab on the center canvas (switch with the multi-flow tabs at the top if you have several).
2. Confirm the entry node exists and there are no dangling edges; fill any required variables in the node detail drawer.
3. Click Run to start the run; the engine advances from the entry node along the directed edges.
4. Watch the final summary and any human-decision cards in the floating AI Chat panel, and watch per-node progress in the right-hand Detail Log.
5. Wait until the run reaches a terminal state (done / error / aborted) before your next move.

## EXAMPLE / EXERCISE

Take the 3-file-codegen seed workflow: open it in the Workflow Editor, click Run, and the engine executes the LLM planning node, the Tool compile node, then the Write node in order. You can also trigger it from AI Chat in one line:

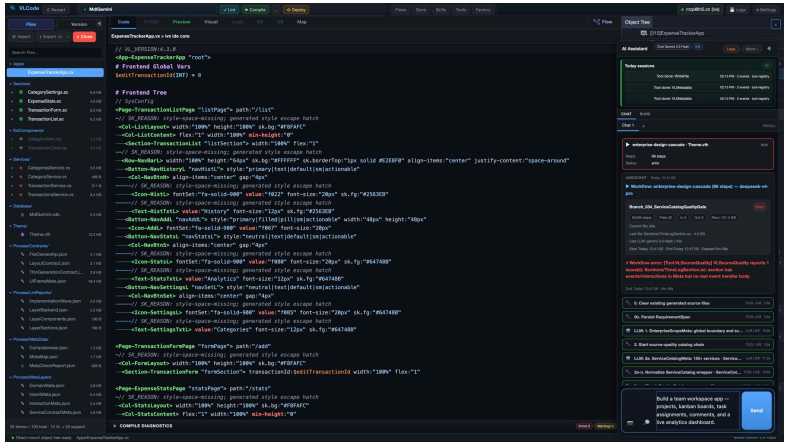
```
Run the current workflow with the goal: generate a TeamPM task board
```

When it finishes, Detail Log shows a terminal event line like `run finished: done`.

### PITFALL

A common mistake is treating Run like Compile — a workflow run advances node-by-node locally, so don't keep re-running just because you never saw a 'compile succeeded' banner. The other trap is clicking Run while the entry node is missing a required variable, which errors immediately; fill it in the node drawer first.

# Watch Live Events In Detail Log



Detail Log — the live run ledger and node results

## CORE IDEA

Detail Log is the full run ledger on the right, where every workflow step appears line by line: node progress, file writes, tool I/O, validation results, and error stacks. It is deliberately distinct from AI Chat — Chat carries only the final summary and human decisions, and the per-node stream is never repeated there. Every entry comes from the vl.workflow.run-event.v4 schema and is individually expandable.

## OPERATIONS

1. Run a workflow and let the right-hand Detail Log auto-scroll the event stream.
2. Locate by node: each LLM / Tool / Write node has its own started and finished line.
3. Expand an event to read the tool's raw input/output or the validation detail.
4. Cross-check the bottom status bar for the current step name, LLM provider/model, and ports.
5. When the run ends, scroll to the top of Detail Log to confirm the terminal-state line.

## EXAMPLE / EXERCISE

Running a workflow with a Tool (VL compile) node, Detail Log shows something like:

```
node N3 Tool(vl-compile) started
tool I/O: in=Sections/Board.sc  out=ok, 0 errors
node N3 Tool finished
```

If that Tool fails you instead get a `node N3 error` line carrying an error stack, which you can expand for the full trace.

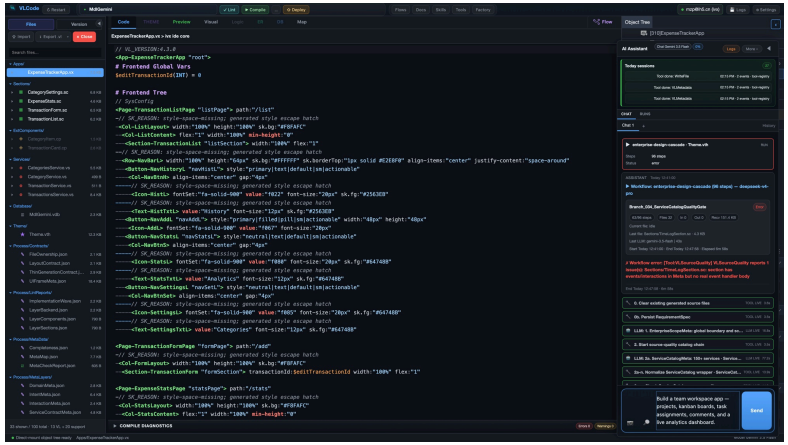
## PITFALL

Beginners hunt for per-node detail in AI Chat, see only a one-line summary, and assume information is missing. Per-node progress, file writes, and tool I/O all live in Detail Log, not Chat — build the habit of using both panels for their distinct roles.

## CHECKPOINT

During a run you can locate a specific node's started, finished (or error) events in Detail Log and expand one to read its tool I/O.

# Pause, Resume, And Graceful Pause



Detail Log — the live run ledger and node results

## CORE IDEA

A VL Agent run can stop at a human gate and then continue: a Pause node is the explicit human-in-the-loop gate, and the engine also supports native checkpoint / resume. The key property is graceful pause — it survives a socket close, so a run is not lost just because you closed a panel or the network blipped. This lets long-running workflows safely park and wait for a person.

## OPERATIONS

1. Place a Pause node so the run stops there and surfaces a human input-form card.
2. When the run hits the Pause, fill the requested inputs (text/number/select...) in the interaction card in AI Chat.
3. When needed, actively pause the whole run; the engine writes a checkpoint and enters the paused state.
4. Confirm in Detail Log that the run is truly stopped, not still advancing.
5. Resume when ready, and the engine continues from the checkpoint.

## EXAMPLE / EXERCISE

An approval-flow-runtime-style workflow halts at a Pause node waiting for an approver to fill a form:

```
[Pause N5] Please fill: review comment (textarea), budget cap (number)
```

After you submit, the run continues. Even if you close the floating panel and reopen it here, graceful pause keeps the run alive so it can be resumed without loss.

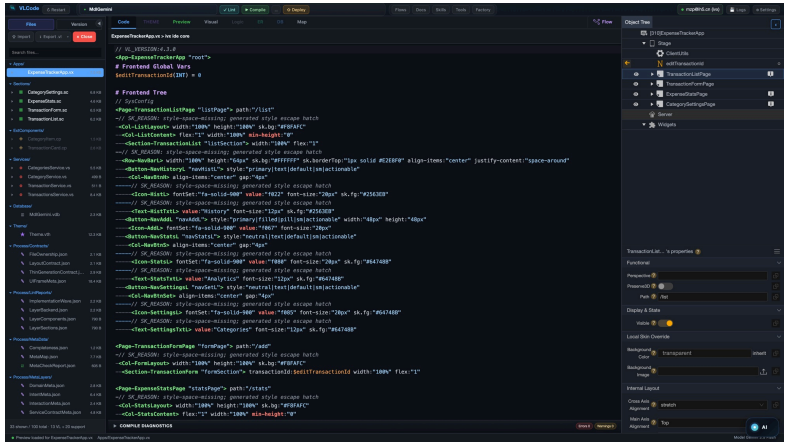
## PITFALL

Don't assume closing the panel or dropping the network equals aborting — graceful pause keeps the run alive, so it is still waiting for input while you think it stopped. To truly end a run, explicitly abort it rather than closing the window.

## CHECKPOINT

You can park a run on a Pause node, close and reopen the panel, then resume and see it continue from the original checkpoint.

# Control A Run In Natural Language (DAG Operator)



The floating AI Assistant panel and chat console

## CORE IDEA

The DAG Operator lets you control a live run in natural language with no LLM round-trip — it maps your words to deterministic control intents and acts instantly. The intents are query_status / query_node / query_var (queries) and control_pause / control_resume / control_stop / control_rerun (controls), plus studio-mode dev_list_nodes / dev_inspect_code / dev_validate. This makes Chat both an observation window and a remote control for the run.

## OPERATIONS

1. In the floating AI Chat panel, issue plain-language commands against the running run.
2. Query intents (status/node/variable) return immediately and consume no LLM call.
3. Control intents (pause/resume/stop/rerun) act directly on the engine.
4. Verify in Detail Log that the control landed (e.g. a paused / resumed event appears).
5. During development, use studio-mode dev_* intents to inspect node code and run structural validation.

## EXAMPLE / EXERCISE

Against a running codegen workflow, say in Chat:

```
Pause the current run
```

The DAG Operator executes control_pause at once, with no model inference. Then `Resume the run` fires control_resume. These are deterministic mappings, not the model 'understanding' and then acting.

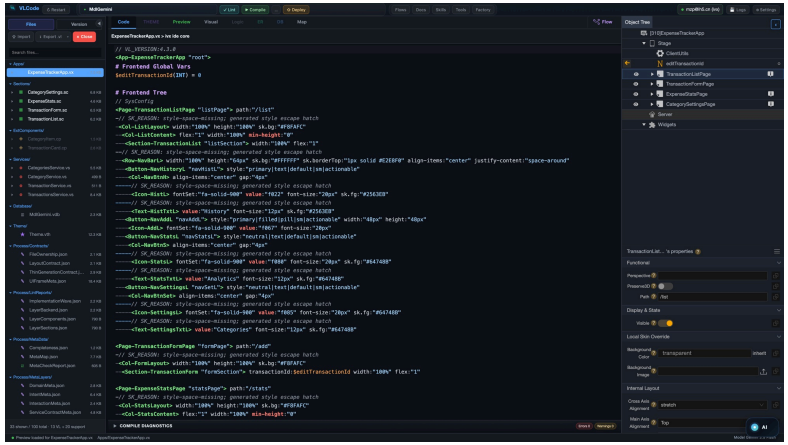
## PITFALL

Treating the DAG Operator like general chat — writing a long, vague paragraph — makes intent matching harder. Use short, explicit imperatives ('pause the current run', 'show variables for N5') to hit a deterministic intent reliably.

## CHECKPOINT

Using only natural language in Chat, you can pause and resume a live run and see the corresponding pause/resume events in Detail Log.

# Query Status, Nodes, And Variables



The floating AI Assistant panel and chat console

## CORE IDEA

The DAG Operator's query intents let you inspect a run's internal state at any time without interrupting it: query_status for overall progress, query_node for a node's execution result, and query_var for a variable's current value. These queries take no LLM round-trip and return immediately, so you can check the run's real state often and cheaply instead of guessing.

## OPERATIONS

1. In AI Chat, issue a plain-language query (status / node / variable).
2. Use query_status to see which step the run is on and whether it's still running.
3. Use query_node with a node id (e.g. N5) to see its output and result.
4. Use query_var with a variable name to read its currently assigned value.
5. Cross-check the query result against the node events in Detail Log for consistency.

## EXAMPLE / EXERCISE

Mid-run, to confirm a plan variable is ready:

```
Show the current value of variable plan
```

It returns the variable's content; then ask:

```
Show the result of node N5
```

The DAG Operator returns N5's output immediately, with no model inference and without pausing the run.

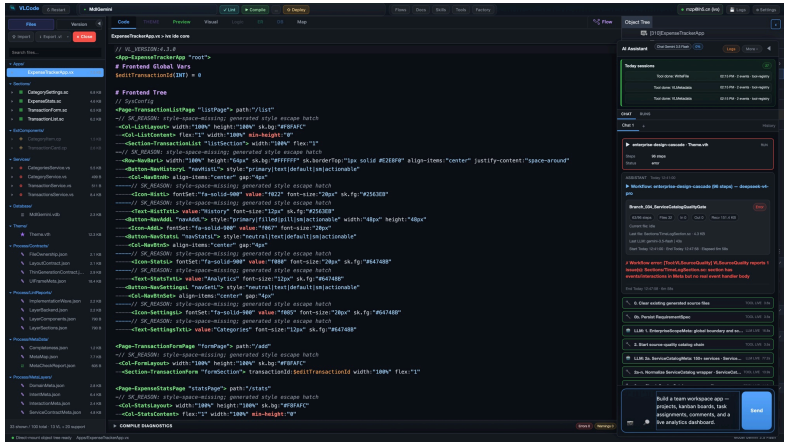
## PITFALL

Vague references ('that node', 'the earlier variable') miss the target. Give a concrete node id or variable name so the query resolves precisely.



## CHECKPOINT

# Rerun From A Checkpoint



Detail Log — the live run ledger and node results

## CORE IDEA

The engine records checkpoints natively, so you don't have to re-run the whole DAG from the top — you can rerun from a specific step (rerun-from-step). This is a huge time saver when debugging: expensive upstream LLM nodes already have results, and you only redo the segment that broke. Rerun is driven by control_rerun, working against the node results and run ledger held in the control plane.

## OPERATIONS

1. In Detail Log, locate which node the run stopped on and where it failed.
2. Confirm the upstream node results you want to keep are preserved in the control plane.
3. Use the DAG Operator's control_rerun (or the UI rerun) to start from that step.
4. Watch Detail Log as the engine skips completed steps and redoes only the chosen segment.
5. Compare the node's result before and after the rerun to confirm the issue changed.

## EXAMPLE / EXERCISE

A 6-file-codegen run fails at the Write node, but the upstream LLM planning already succeeded. After fixing the input, just say:

```
Rerun from node N7
```

control_rerun reuses N7's earlier prerequisites and redoes only the steps from N7 onward, saving a full LLM re-computation.

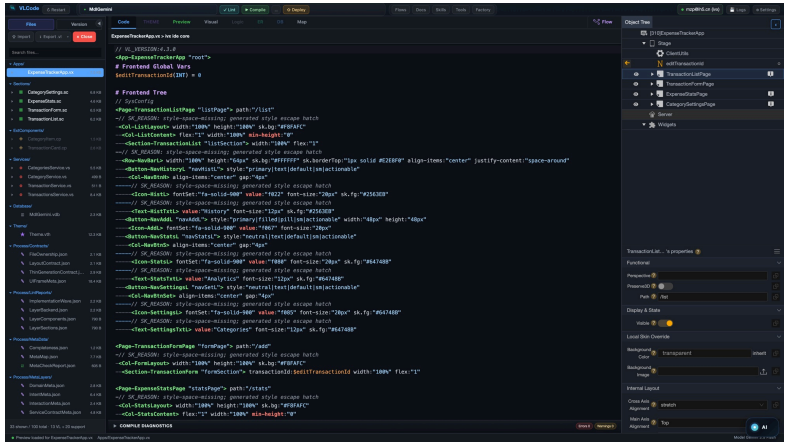
## PITFALL

Fixing upstream logic but still rerunning from the failed node reuses stale upstream results. If your change affects an upstream node, rerun from that earlier affected node, not just from the last step that failed.

## CHECKPOINT

After a failed run you can rerun from a chosen node and see in Detail Log that completed upstream steps were skipped and only the target segment re-executed.

# Register A Conditional Watch



The floating AI Assistant panel and chat console

## CORE IDEA

A conditional watch lets you declare 'if this condition happens, act automatically' so you don't have to babysit a run. You register a watch in natural language — for example 'stop if LLM_430 fails' — bound to an action of stop / pause / notify; when the condition fires it raises a watch-fired event back into Chat. This turns manual monitoring into automatic interception, which long-running workflows especially benefit from.

## OPERATIONS

1. In AI Chat, describe the watch condition and the target node in natural language.
2. Assign the action to take on a hit: stop, pause, or notify.
3. Confirm the watch is registered and active for the current run.
4. Let the run continue; while the condition is unmet the watch waits silently.
5. When the condition fires, receive a watch-fired event in Chat and see the corresponding action in Detail Log.

## EXAMPLE / EXERCISE

Register an interception on a multi-node codegen run:

```
Stop if LLM_430 fails
```

This registers a watch with action stop. The moment node LLM_430 errors, the watch fires, the run is stopped, and a watch-fired notice appears in Chat — no need to watch the screen the whole time.

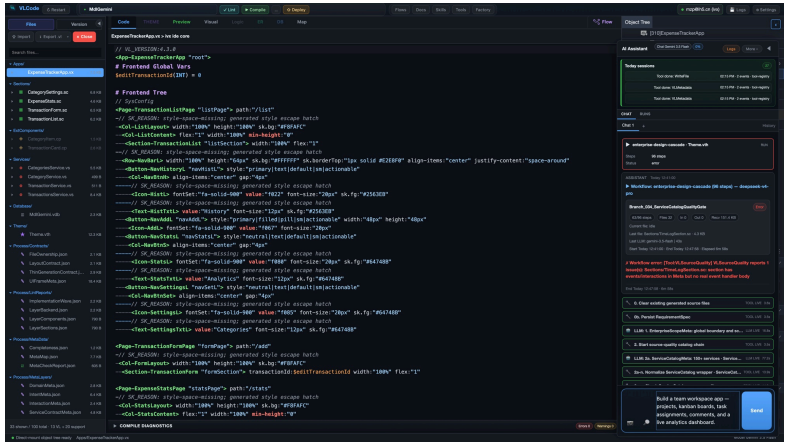
## PITFALL

Confusing notify with stop — a notify-only watch just tells you while the run keeps going. When you actually need to halt, choose stop or pause; don't assume a notify watch made you safe.

## CHECKPOINT

You can register a watch like 'stop if a node fails' and, when that condition is met, receive a watch-fired notice and see the run handled by the chosen action.

# Read Error Stacks And Node Results



Detail Log — the live run ledger and node results

## CORE IDEA

When a run fails, the root cause hides in two places: the failed node's error stack in Detail Log, and the node results plus run ledger persisted in the control plane (.vl-code/control-plane.sqlite). VL Agent follows the root-cause-first principle — errors surface honestly, never swallowed or faked into success — so what you see is the real stack, not a hidden hint. Reading both is how you find the true failure point before fixing it.

## OPERATIONS

1. In Detail Log, find the node event marked error and expand it for the full error stack.
2. Use query_node to pull that node's inputs and outputs and tell a data problem from a logic problem.
3. Cross-reference the control-plane run ledger's timestamps and error stacks to locate the first failing step.
4. Go back to that node's source (VL source or node config) to find the root cause, avoiding broad rewrites.
5. After the fix, rerun from the affected node to verify.

## EXAMPLE / EXERCISE

Detail Log shows:

```
node N4 Tool(vl-compile) error
  VL_VERSION must be the first line of file Sections/Board.sc
```

The root cause is clear: Board.sc's first line isn't VL_VERSION. The fix is to put `// VL_VERSION:4.3.1` on the very first line of that file (nothing before it) and rerun from N4.

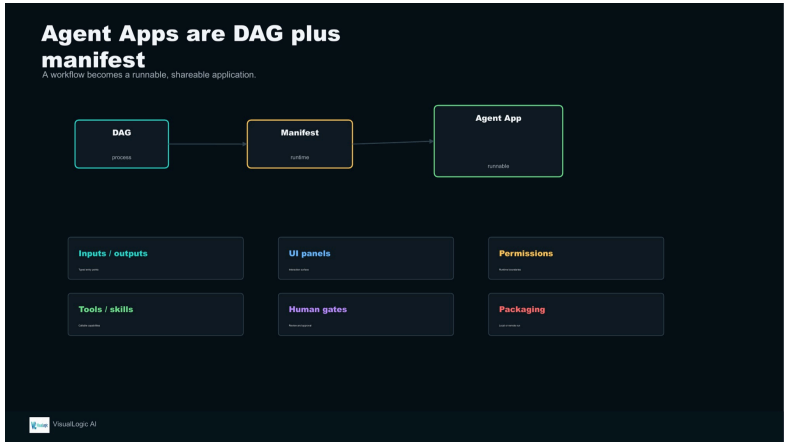
## PITFALL

Acting on the first red line you see often fixes a symptom, not the cause. Walk the run ledger to the *first* failing node, then read its error stack and node result — later errors are usually its downstream fallout.



## CHECKPOINT

# Master Agent: Planner/Evaluator/Scheduler/Arbiter




Master Agent — branch scoring, pruning, and promotion

## CORE IDEA

The Master Agent is VL Agent's runtime scheduling brain, a closed loop of four roles — Planner / Evaluator / Scheduler / Arbiter: the Planner proposes candidate branches, the Evaluator scores them, the Scheduler runs them concurrently, and the Arbiter decides which to keep, prune, or promote. It isn't a single one-shot inference but 'process intelligence' made observable and arbitrable as a loop, with every prune/expand/promote state drawn directly on the DAG.

## OPERATIONS

1. Pick a workflow that uses the Master Agent (e.g. code-evolution-studio's compete mode).
2. Run it and watch the Planner expand several candidate branches on the DAG.
3. Watch the Evaluator score each branch (scores show up on node states).
4. Watch the Scheduler advance candidates concurrently within the concurrency cap.
5. Watch the Arbiter decide: pruned / expanded / promoted states render on the DAG.

## EXAMPLE / EXERCISE

Run code-evolution-studio-main in compete mode against one goal: the Master Agent spawns several candidate implementations, the Evaluator scores them, and the Arbiter marks the top score as promoted and the rest as pruned. You see it on the DAG:

```
branch B2  score 0.86  -> promoted
branch B1  score 0.41  -> pruned
```

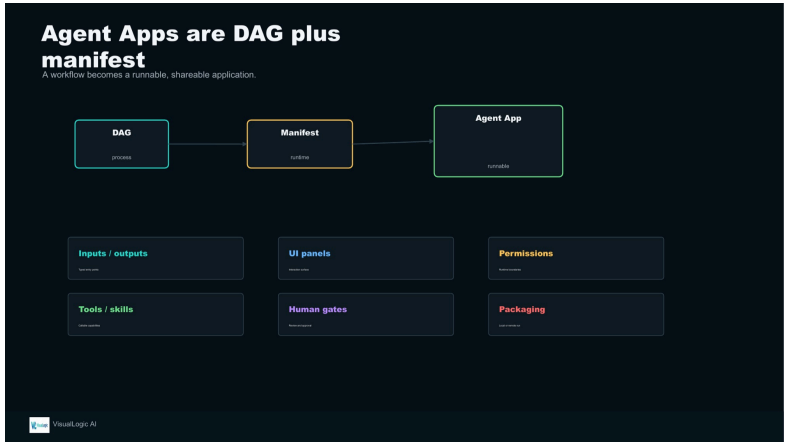
## PITFALL

Treating the Master Agent as a 'smarter single LLM call' and expecting the optimum in one shot. Its value is the multi-branch competition plus scored arbitration — give it real concurrency and candidate space instead of running a single line.

## CHECKPOINT

You can run a Master Agent workflow and tell apart the Planner's candidates, the Evaluator's scores, and the Arbiter's prune/promote outcomes on the DAG.

# Branch Scoring, Pruning, And Mainline Promotion




Master Agent — branch scoring, pruning, and promotion

## CORE IDEA

Inside the Master Agent loop, candidate branches don't all coexist to the end — they are continuously scored, pruned, and the best performer is promoted to the mainline. The Evaluator's scores drive the Arbiter's choices: low-score branches are pruned, the high-score branch is promoted as the mainline, and pruned versions are still preserved for traceback and comparison. This is the mechanism that converges 'explore many possibilities' into 'one deliverable mainline'.

## OPERATIONS

1. Run a Master Agent workflow that spawns multiple candidates.
2. Read each branch's score on the DAG to spot strong versus weak candidates.
3. Watch weak branches get marked pruned.
4. Watch the best branch get promoted to the mainline.
5. When needed, revisit a pruned-but-preserved version to sanity-check the promotion.

## EXAMPLE / EXERCISE

A fleet-style parallel delivery workflow spawns three candidate implementations; after scoring:

```
B1 score 0.52  pruned
B2 score 0.88  promoted (mainline)
B3 score 0.61  pruned (version kept)
```

B2 becomes the mainline; B1/B3 are pruned but their versions are kept, so you can always revisit their artifacts for comparison.

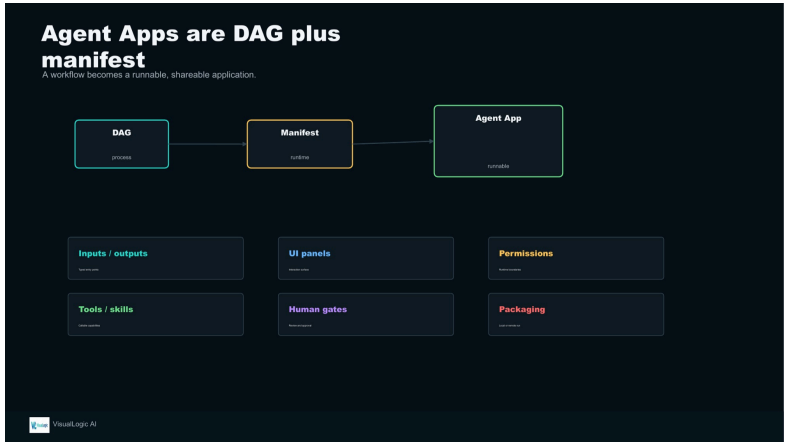
## PITFALL

Assuming pruned means deleted and gone for good. Pruned branch versions are preserved — don't re-run from scratch just because a branch was pruned; revisiting the kept version is often faster.

## CHECKPOINT

You can explain why one branch was pruned and another promoted to mainline in a run, and you can recover a pruned-but-preserved version.

# Goal-Convergence Demos




Master Agent — branch scoring, pruning, and promotion

## CORE IDEA

Goal-convergence demos show how the Master Agent keeps closing in on a declared goal: by repeatedly proposing candidates, scoring, pruning, and promoting, the run converges from divergent exploration onto a mainline that satisfies the goal. Their point is to make 'process intelligence' visible — you watch not a single LLM output but a trajectory where a goal is approached round by round and finally settled, all rendered on the DAG.

## OPERATIONS

1. Pick a goal-convergence Master Agent demo workflow and state the goal explicitly.
2. Run it and watch each round's Planner expand candidates and Evaluator score them.
3. Track how scores move across rounds, confirming overall approach toward the goal.
4. Watch the Arbiter prune weak branches and promote strong ones round by round.
5. Confirm, when the run reaches a terminal state, that the mainline satisfies the declared goal.

## EXAMPLE / EXERCISE

After declaring the goal, start the demo:

```
Goal: converge the campus-ops dashboard's first-screen load logic to an implementation that compiles cleanly and passes Lint
```

The Master Agent iterates over several rounds, scores climb from ~0.4 into the 0.9 range, and the satisfying branch is promoted to mainline — the convergence trajectory is visible on the DAG.

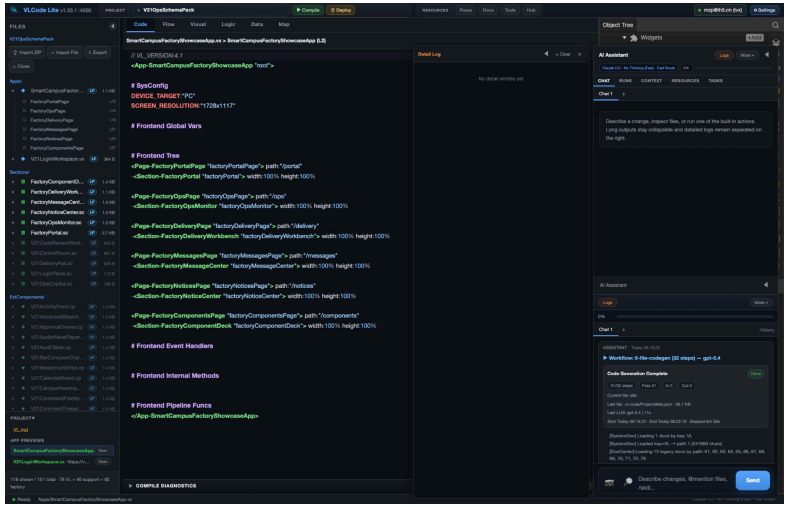
## PITFALL

A vague goal ('make it better') gives the Evaluator nothing to score, so there's no convergence. State a decidable goal (e.g. 'compiles cleanly, passes Lint') so convergence has a direction.

## CHECKPOINT

You can run a convergence demo against a decidable goal and see scores climb round by round on the DAG until the mainline meets the goal.

# Concurrency And Fan-Out Control



Settings — provider, model, and runtime configuration

## CORE IDEA

VL Agent's concurrency is governed by workflow settings: workflowMaxConcurrency (default 5) caps how many parallel tasks advance at once, and workflowFanoutProvider sets which provider the fan-out stage uses. One IDE window = one workspace = one backend, yet multiple chat sessions and multiple workflow runs can run in parallel. Tuning concurrency well both speeds up the Master Agent's multi-branch competition and avoids overrunning a provider's quota.

## OPERATIONS

1. Open the LLM / workflow configuration area in Settings.
2. Set workflowMaxConcurrency to control how many parallel tasks advance at once (default 5).
3. Use workflowFanoutProvider to choose the provider for the fan-out / multi-branch stage.
4. If you need to mix providers, confirm the allowMixedLlmProviders setting.
5. Run a Master Agent workflow and verify in Detail Log that the parallelism matches expectations.

## EXAMPLE / EXERCISE

To speed up a fleet parallel delivery while controlling cost, set concurrency to 8 and route fan-out to a low-cost provider:

```
workflowMaxConcurrency = 8
workflowFanoutProvider = kimi-api
```

Multi-branch competition then fans out on Kimi (one of the cost-sensitive defaults), while the main advance still uses your chosen chat provider.

## PITFALL

Cranking concurrency very high assuming it must be faster — it can trigger provider rate limiting and end up slower or erroring. Start at the default 5, watch throughput in Detail Log, raise it gradually, and confirm the fan-out provider's quota is sufficient.

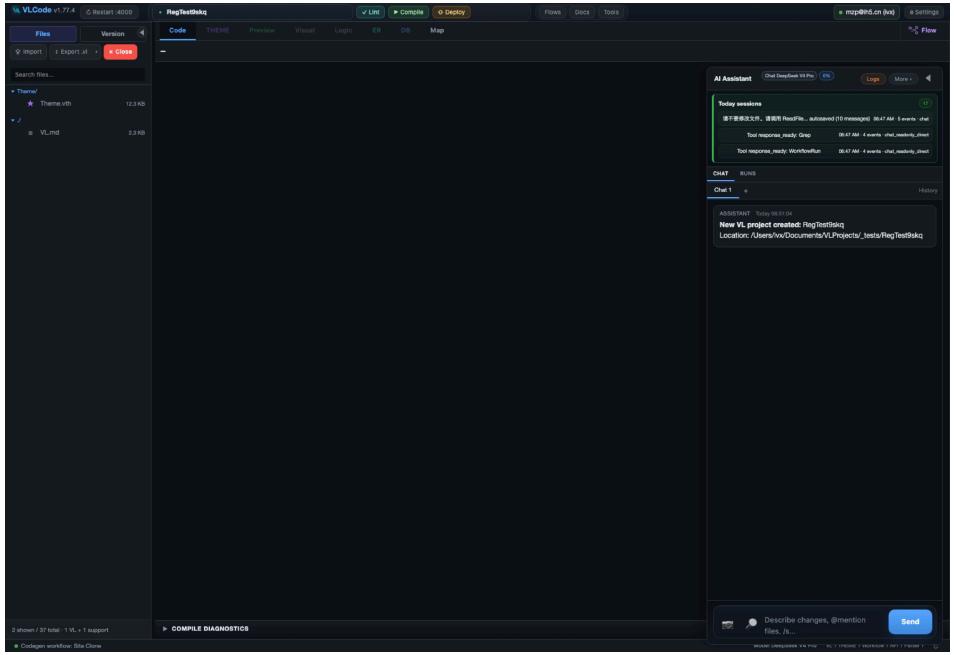
## CHECKPOINT

You can adjust workflowMaxConcurrency and workflowFanoutProvider in Settings and see the run's parallelism change accordingly in Detail Log.

CHAPTER

# Part VIII: .vflow, Resource Center, And Shipping To VLCode

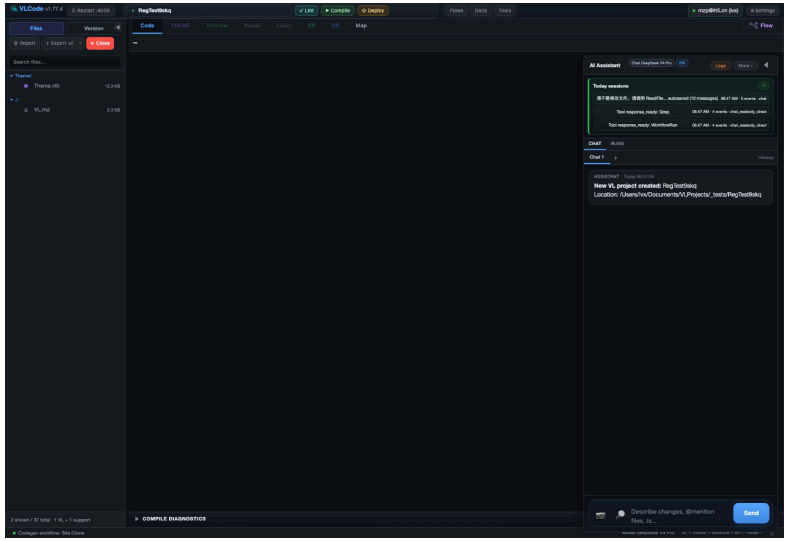
- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.




Flow Bundle Mode — editing a .vflow directly

PART VIII: .VFLOW, RESOURCE CENTER, AND SHIPPING TO VLCODE

# The .vflow Bundle Format



Flow Bundle Mode — editing a .vflow directly

## CORE IDEA

In VL Agent the .vflow is the single app file of Agent App Studio spec v2.2.0: one .vflow IS the complete app pack — the entry workflow (workflow JSON) plus a top-level app block, with NO separate .app.json. The bundle can also carry a manifest, optional subflows, docs and prompts, and embedded VL surfaces plus config docs, so it is self-contained and can be moved to another machine or handed to a VLCode developer as-is. Whether a workflow is hand-authored or produced by generate-agent-app-quick / agent-app-scaffold-main, the result settles into the same .vflow shape.

## OPERATIONS

1. Switch the center canvas to Flow Bundle Mode, which opens, edits, and saves a .vflow directly
2. Inspect the bundle structure: workflow JSON (nodes + edges), the app block, and optional subflows/docs/prompts
3. Open the Manifest Inspector (left slide-in) and confirm the manifest is DERIVED at load time from the app block plus Pause/Review steps, not hand-filled
4. Use the right-side FLOW TREE to see subflows and unit hierarchy inside the bundle
5. Confirm this one file carries the whole app — no companion .app.json is needed

## EXAMPLE / EXERCISE

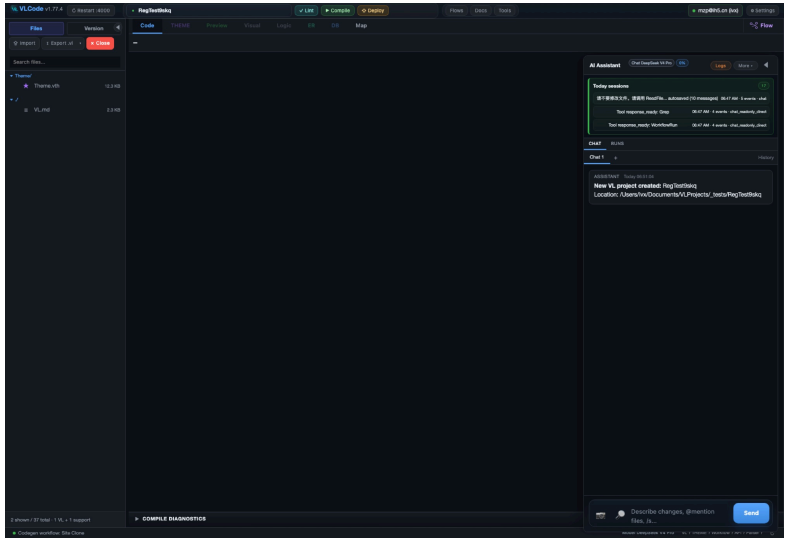
The top-level skeleton of a saved approval Agent App .vflow looks roughly like:

```
{
  "id": "approval-flow-runtime",
  "spec": "4.1",
  "app": { "title": "Expense Approval", "entry": "main" },
  "nodes": [
    { "id": "LLM_010", "type": "LLM" },
    { "id": "REVIEW_020", "type": "Review" }
  ],
  "edges": [ { "from": "LLM_010", "to": "REVIEW_020" } ]
}
```



The app block declares the entry workflow, and the Review node is derived into an approval surface in the manifest.

# Save And Save-As A .vflow



Flow Bundle Mode — editing a .vflow directly

## CORE IDEA

In Agent App Studio the authoring sequence is: compose in Units + DAG (or draw the DAG directly) -> one-click Run to test -> Pause/Resume to debug -> declare the app block manifest -> Save as .vflow. Flow Bundle Mode lets you open/edit/save a .vflow directly, so you can freeze the current canvas into a portable bundle at any time, or Save-As a generated Agent App to fork a variant. Declare the app block before saving so the manifest can be derived correctly at load time.

## OPERATIONS

1. Finish node composition in the Workflow Editor and use the top one-click Run to confirm the run state settles cleanly to done
2. Use Pause / Resume to walk through human gates and branches until stable
3. Switch to Flow Bundle Mode and declare the top-level app block (title, entry workflow)
4. Save to freeze the current workflow + app block + optional subflows/docs/prompts into one .vflow
5. To fork a variant, use Save-As with a new English PascalCase file name, leaving the original .vflow untouched

## EXAMPLE / EXERCISE

To fork a CustomerCRM intake app into a new variant: in Flow Bundle Mode click Save-As and name it `CustomerCrmIntakeV2.vflow` (English PascalCase); the original `CustomerCrmIntake.vflow` stays unchanged, giving you a branch that can evolve independently.

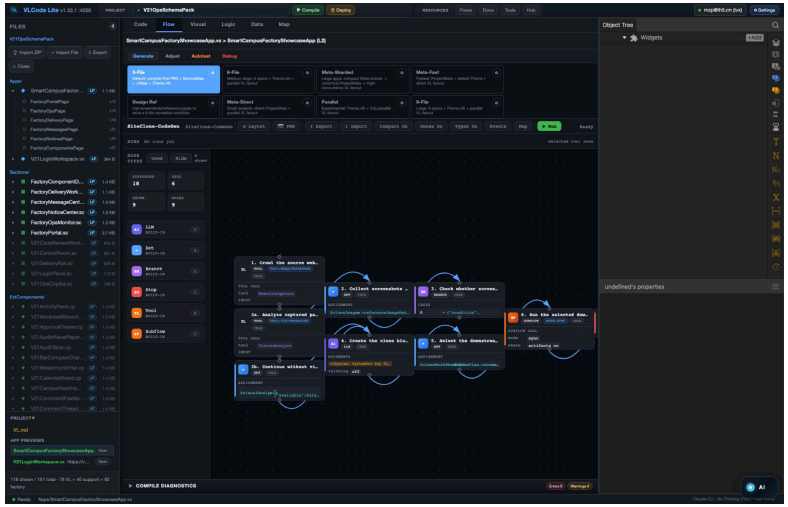
## PITFALL

Saving before declaring the app block yields a bundle with only a workflow and no app entry, so no manifest can be derived at load. Always declare the app block in Flow Bundle Mode first. Also keep file names English PascalCase — no Chinese characters or spaces.

## CHECKPOINT

You can Save the current canvas into a .vflow that carries an app block, and Save-As a renamed variant without breaking the original file.

# Register To The Local Resource Center



Resource Center — registering and syncing a .vflow

## CORE IDEA

Saving a .vflow is not enough — for others (especially VLCode developers) to browse and reuse it, you must register it to the local Resource Center. Registration writes the bundle to .vl-code/registry/workflows/<id>.json under the workspace, which is the first stop of the whole 'generate in VL Agent -> Resource Center -> VLCode' pipeline. Only registered workflows show up in the workflow picker and the Resource Center list, ready for cloud sync and cross-product distribution.

## OPERATIONS

1. Confirm the .vflow is saved in Flow Bundle Mode and its app block is declared
2. Register it to the local Resource Center, writing .vl-code/registry/workflows/<id>.json
3. Confirm it now appears in the left file tree / workflow picker as a selectable workflow
4. If it carries reusable UI, its ViewKit and app resources land under .vl-code/registry/viewkits and apps
5. Verify via CLI: `node bin/vlflow.js list` should list the newly registered workflow

## EXAMPLE / EXERCISE

After registering a reusable workflow produced by workflow-factory, the local registry gains a record:

```
.vl-code/registry/workflows/customer-crm-intake.json
```

Run `node bin/vlflow.js list` to see it in the workflow inventory before it goes to cloud sync.

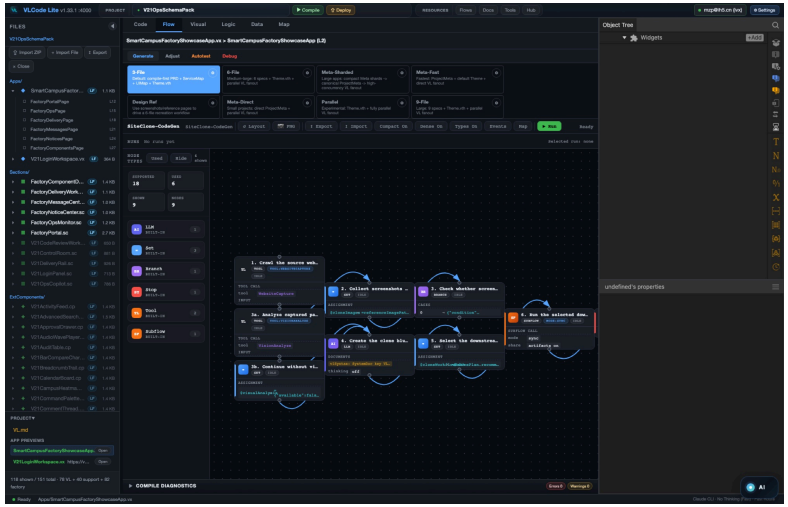
## PITFALL

Saving a .vflow to some folder but never registering it to the Resource Center is the most common broken link — such a file never reaches .vl-code/registry/workflows, never gets cloud-synced, and the VLCode side never sees it. Always register explicitly after saving.

## CHECKPOINT

Your .vflow exists at .vl-code/registry/workflows/<id>.json and is listed by `node bin/vlflow.js list`.

# The Local Registry Layout



Resource Center — registering and syncing a .vflow

## CORE IDEA

Under each workspace, .vl-code/registry is the local on-disk shape of the Resource Center, split into three subdirectories: workflows (workflow bundles), apps (Agent Apps), and viewkits (reusable UI component libraries). It is distinct from the sibling control-plane.sqlite (run ledger / node results / review resources / artifact lineage) — the registry stores reusable asset DEFINITIONS, while control-plane stores the live facts of a particular run. Knowing this layout tells you exactly where generated or registered things land and where VLCode pulls from.

## OPERATIONS

1. Open .vl-code/registry under the workspace root and confirm the workflows / apps / viewkits subdirectories
2. See each registered workflow as <id>.json under workflows/
3. See ViewKit registrations like human-task, table-generic, form-generic, approve-decision under viewkits/
4. Distinguish it from .vl-code/control-plane.sqlite — that is run reality, not asset registration
5. Note that switching workspace reloads both the registry and the control-plane for that workspace

## EXAMPLE / EXERCISE

After registering several assets, a workspace's registry looks like:

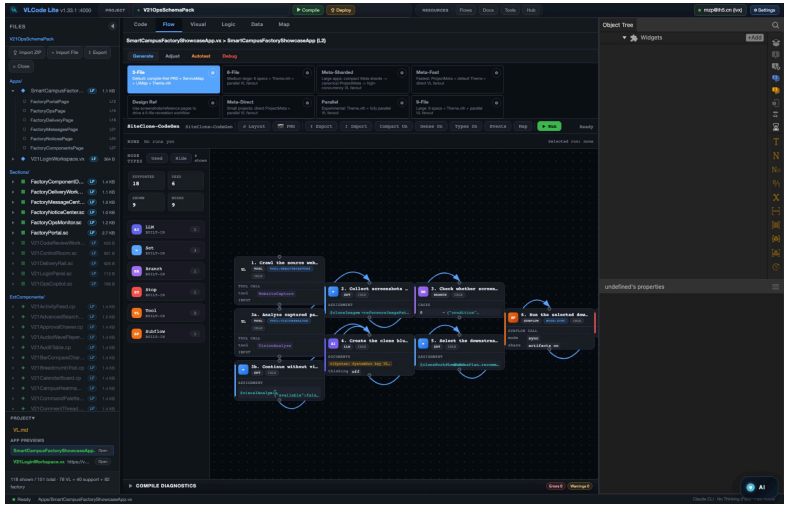
```
.vl-code/registry/
  workflows/   approval-flow-runtime.json
  apps/        expense-approval.json
  viewkits/    approve-decision.json
```

while .vl-code/control-plane.sqlite separately holds each run's ledger and node results.

## PITFALL

Conflating the registry with control-plane.sqlite is a common mistake: deleting control-plane only loses run history, while deleting the registry loses asset definitions. Don't touch the registry directory to 'clear run records.'

# Resource Center Cloud Sync



Resource Center — registering and syncing a .vflow

## CORE IDEA

Local registration only places the .vflow into this workspace's .vl-code/registry — to make it visible across machines and teams you need Resource Center cloud sync. Cloud sync is the bridge in the 'generate in VL Agent -> Resource Center -> VLCode' chain: workflows/Agent Apps from the local registry flow up to the cloud Resource Center, and a VLCode developer browses and imports them down using the SAME unified account (visuallogic.ai, same-site download). That is how one .vflow travels from the author's machine to any VLCode team.

## OPERATIONS

1. Ensure the target .vflow is written to local .vl-code/registry/workflows/<id>.json
2. Trigger cloud sync through the Resource Center to push local registry assets upward
3. Confirm you are signed into the unified visuallogic.ai account (VL Agent and VLCode are same-site, same-account)
4. On the VLCode side, open the Resource Center with the same account and confirm the just-synced workflow appears
5. If the Resource Center is slow, the bundled seed prompts/docs let you keep working offline and reconcile once sync recovers

## EXAMPLE / EXERCISE

To put a reusable workflow in the cloud for a VLCode team: register `customer-crm-intake.json` locally, then cloud-sync via the Resource Center; a VLCode developer signed into the same visuallogic.ai account opens the Resource Center, sees `customer-crm-intake` in the list, and imports it.

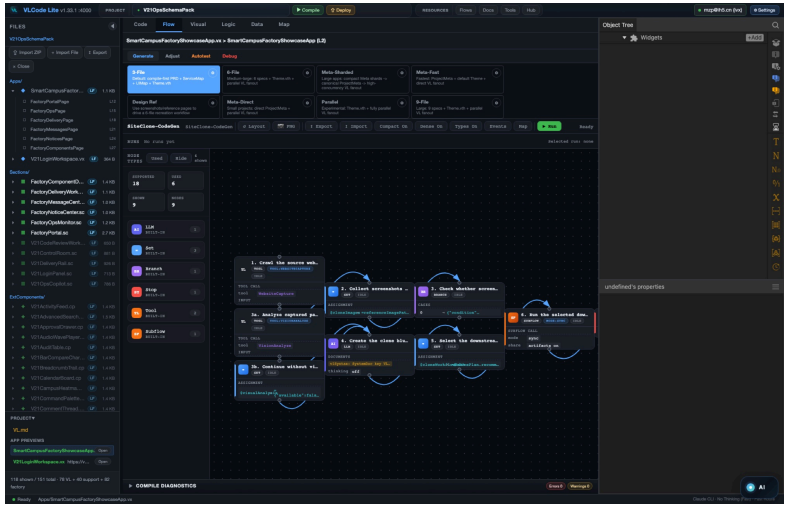
## PITFALL

Different accounts on the two sides is the most common reason an asset 'doesn't show up' — VL Agent and VLCode must use the same unified visuallogic.ai account, or cloud-synced assets never appear on the other side. Verify the signed-in account before syncing.

## CHECKPOINT

A locally registered workflow, after cloud sync, appears in the Resource Center list on the VLCode side under the same visuallogic.ai account.

# Ship A .vflow To VLCode



Resource Center — registering and syncing a .vflow

## CORE IDEA

This is where the two-product relationship between VL Agent and VLCode becomes concrete: an Agent App's entry workflow, and the reusable workflows produced by workflow-factory-main, are all .vflow bundles. The full shipping path is 'author in AAS -> one-click Run to test -> Pause/Resume to debug -> declare the app block manifest -> Save as .vflow -> register to the local Resource Center -> cloud sync -> a VLCode developer browses / imports / customizes / publishes to teams.' So 'VL Agent generates Agent Apps, including the workflows that support VLCode' is not a slogan but this real generate -> Resource Center -> VLCode pipeline.

## OPERATIONS

1. In VL Agent, run workflow-factory-main to generate a reusable workflow from intent (the output is a .vflow)
2. One-click Run to test, debug with Pause/Resume, declare the app block, then Save as .vflow
3. Register it to the local Resource Center (.vl-code/registry/workflows/<id>.json)
4. Cloud-sync via the Resource Center so it is visible to the VLCode team
5. A VLCode developer imports the SAME .vflow to drive enterprise app generation

## EXAMPLE / EXERCISE

To ship a VLCode team a workflow for generating CRM modules: in VL Agent run `workflow-factory-main` with the prompt "Generate a reusable CustomerCRM module generation workflow", producing `customer-crm-codegen.vflow`; after local register + cloud sync, a VLCode developer imports the same .vflow from the Resource Center and uses it to drive enterprise app generation for that module.

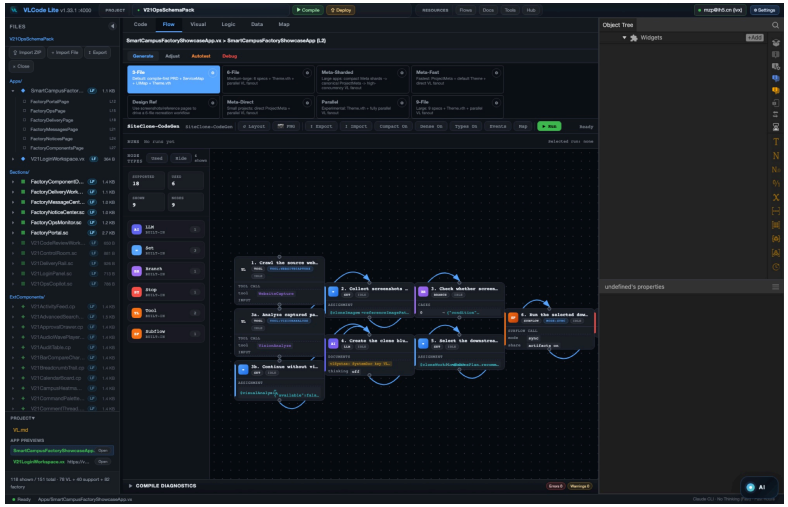
## PITFALL

Mistaking the deliverable as the generated VL source to copy into VLCode — what ships is the .vflow workflow itself, which the VLCode developer imports and runs inside their own project. Getting the object wrong degrades a reusable pipeline into a one-off code transfer.

## CHECKPOINT

You can describe the full generate -> Resource Center -> VLCode chain from a workflow-factory-main .vflow to a VLCode developer importing the same .vflow.

# Import And Customize In VLCode



Resource Center — registering and syncing a .vflow

## CORE IDEA

A .vflow on the VLCode side is not a read-only black box — VLCode's Flow / .vflow capability lets a developer drag it into the Flow panel, categorize, run, and customize it. On import you add it via drag-drop or '+ Upload .vflow' and choose a category (Generate / Enterprise / Clone / MetaData / Adjust / Autotest / Debug / Quality / Custom); run states settle reliably to done/error/aborted. So a workflow shipped from VL Agent gets reused, adjusted, and re-published to teams inside VLCode, closing the final leg of the generate -> Resource Center -> VLCode pipeline.

## OPERATIONS

1. In VLCode switch to the Flow panel and import the Resource-Center .vflow via drag-drop or '+ Upload .vflow'
2. Pick a category on import (e.g. Generate or Enterprise) for filing and management
3. Run it once from the Flow panel and watch the run state settle to done/error/aborted
4. Customize for the team (swap Provider/model, adjust nodes, embed local project context)
5. Once satisfied with reuse, publish the result to teams from VLCode

## EXAMPLE / EXERCISE

A VLCode developer receives `customer-crm-codegen.vflow`: in the Flow panel they click '+ Upload .vflow', choose category `Generate`, run it once to generate the CRM module inside their own `~/Documents/VLProjects/CustomerCRM/` project, then fine-tune nodes for the team theme.

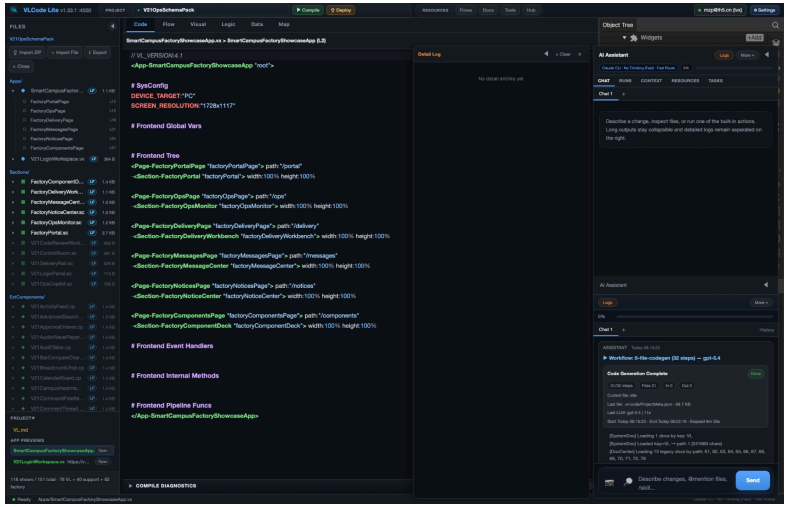
## PITFALL

Picking the wrong category on import files the workflow under the wrong group in the Flow panel and makes it hard to find later; choose the correct category (Generate / Enterprise / Clone, etc.) at the moment of import per its real purpose.

## CHECKPOINT

You can import a .vflow in VLCode's Flow panel via '+ Upload .vflow', pick the right category, and run it to done.

# SysDoc And The Live-Fetch Guard



Settings — provider, model, and runtime configuration

## CORE IDEA

SysDoc is the platform's canonical spec set (VL syntax, ThemeDefault, WorkflowSpec, PlatformAPIs, AgentAppDevelopmentSpec, PortConventionGuide, ComponentFactory, MetaSpec). VL Agent runs a live-fetch guard before generation / compile / runtime to ensure it works against the latest specs rather than a stale cache. This matters especially for the .vflow pipeline: generating Agent Apps rides AgentAppDevelopmentSpec, drawing the DAG and nodes rides WorkflowSpec, and outbound interfaces ride PlatformAPIs — the guard keeps your bundle consistent with the current platform contract so it won't fail on the VLCode side from spec drift.

## OPERATIONS

1. Before generating an Agent App, confirm the live-fetch guard has pulled the latest AgentAppDevelopmentSpec and WorkflowSpec
2. Refresh PlatformAPIs for outbound platform / Resource Center calls
3. Refresh MetaSpec for metadata / project shape, and ThemeDefault for theming
4. Refresh ComponentFactory when using the component factory (generate-agent-app-with-components)
5. Check PortConventionGuide first for port / local-service issues to avoid using the wrong port range

## EXAMPLE / EXERCISE

Before running `generate-agent-app-cascade`, the guard live-fetches `AgentAppDevelopmentSpec`, `WorkflowSpec`, and `PlatformAPIs`; if you trust an old local cache and skip this, the generated manifest may not match the current v2.2.0 contract.

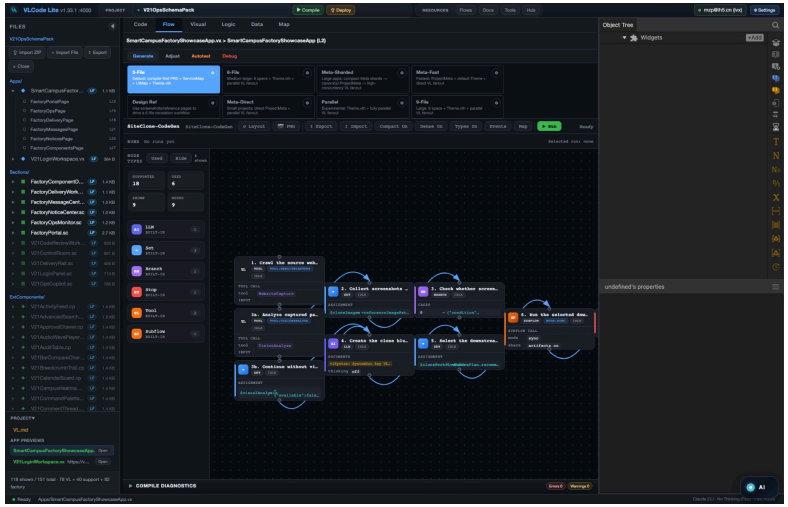
## PITFALL

Disabling or bypassing the live-fetch guard to save time and eating a stale cache is the root of spec drift — generation looks fine but the bundle errors on the VLCode side from contract mismatch. Let the guard pull the latest SysDoc before every generate/compile/run.

## CHECKPOINT

You can name the SysDocs the live-fetch guard should pull when generating an Agent App and drawing a DAG (at least AgentAppDevelopmentSpec and WorkflowSpec).

# The GitHub Publication Path



Resource Center — registering and syncing a .vflow

## CORE IDEA

Resource Center cloud sync is the distribution channel inside the VL Agent -> VLCode ecosystem, letting teams reuse a .vflow under the unified visuallogic.ai account; publishing examples or reusable workflows externally goes through a separate GitHub path. The two paths target different audiences: team reuse via Resource Center cloud sync, community release via a GitHub repo. Understanding this split keeps you from pushing an internal-only bundle to a public repo, or conversely leaving a public example stuck in only the local registry.

## OPERATIONS

1. Distinguish the audience: team reuse via Resource Center cloud sync, public release via a GitHub repo
2. Strip any secrets before publishing — keys live only in Settings (masked) or the SecretRoot vault, never in committable files
3. Keep .vflow file names English PascalCase for cross-repo and cross-platform reference
4. Before publishing an example workflow, confirm it doesn't depend on private control-plane data in the local workspace (.vl-code/control-plane.sqlite)
5. Confirm the bundle you publish is self-contained (workflow + optional embedded VL surfaces + config docs) and references no private resource outside the workspace

## EXAMPLE / EXERCISE

To publish a demo approval workflow to the community: export a clean, self-contained `ApprovalFlowRuntime.vflow` with no keys at all, then publish it to GitHub; the team-internal version keeps flowing through Resource Center cloud sync, with no overlap.

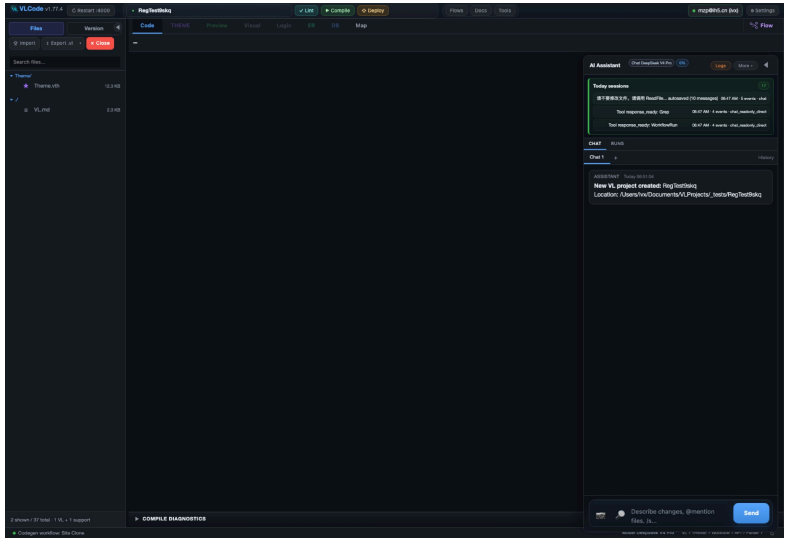
## PITFALL

Publishing a secret-laden bundle externally is a serious mistake — verify the bundle contains no API key, cookie, or token before publishing; secrets stay only in Settings (masked) or the SecretRoot vault, never in committable files.

## CHECKPOINT

You can name the two distribution paths (Resource Center cloud sync for teams, GitHub for external release) and confirm an external bundle carries no secrets.

# Versions, Contracts, And Portability



Flow Bundle Mode — editing a .vflow directly

## CORE IDEA

A .vflow can move safely between machines and between VL Agent and VLCode because of a few explicit versions and contracts: Agent App Studio spec v2.2.0 (single-.vflow model), Workflow Spec 4.1 (Workflow-Engine 4.13.1), and the event schema vl.workflow.run-event.v4. The bundle is self-contained (workflow + optional embedded VL surfaces + config docs) AND honors these contracts, which is what makes it portable. Understanding the version contracts lets you judge whether a .vflow will run as-is on the other side, and which fields to watch when the engine is upgraded.

## OPERATIONS

1. Confirm the .vflow's declared spec (Workflow Spec 4.1) is compatible with the target engine (4.13.1)
2. Confirm the app block follows AAS v2.2.0's single-file model — manifest derived at load, not from an external file
3. Confirm run events follow vl.workflow.run-event.v4 so the ledger stays consistent across sides
4. Keep the bundle self-contained: pack in the embedded VL surfaces / config docs it depends on
5. Run `node bin/vlflow.js validate <id>` for a contract check before migrating

## EXAMPLE / EXERCISE

Validate the contract before moving a .vflow to another machine:

```
node bin/vlflow.js validate approval-flow-runtime
```

It checks structure against Workflow Spec 4.1; once it passes, because the bundle is self-contained and the event schema is v4, the other side runs it as-is and produces a consistent run ledger.

## PITFALL

Depending on a workspace resource that isn't packed into the bundle (a local file, an unregistered ViewKit) breaks portability — it fails on the other side from a missing resource. Validate before migrating and ensure dependencies are embedded into the self-contained .vflow.

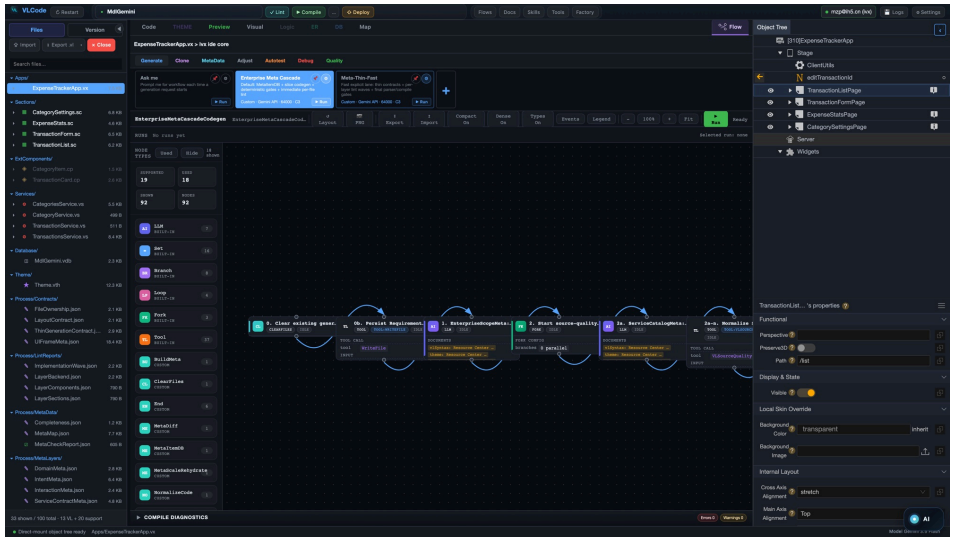


## CHECKPOINT

CHAPTER

# Part IX: Complete Case Studies

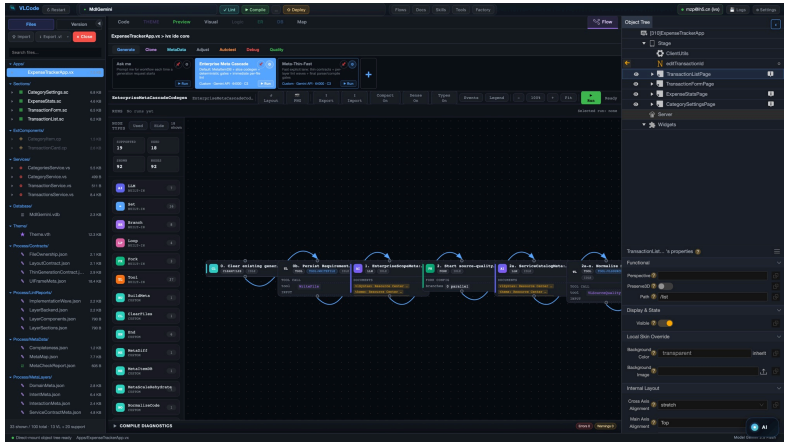
- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



Workflow Editor — nodes and directed edges on the DAG canvas

PART IX: COMPLETE CASE STUDIES

# Case 1: Your First Workflow



Workflow Editor — nodes and directed edges on the DAG canvas

## CORE IDEA

Your first workflow is the smallest exercise for getting comfortable with the DAG canvas: place a few nodes in the Workflow Editor, connect them into one directed chain, run a single LLM call, and write the result to a file. It teaches the basic units of Workflow Spec 4.1 — nodes, directed edges, variable passing — laying the groundwork for later using a Flow to generate a complete Agent App.

## OPERATIONS

1. Click new-workflow in the left file tree / workflow picker to open a fresh DAG canvas in the Workflow Editor (the Flow mode tab opens it).
2. Drag a Set node from the node drawer and assign an initial variable, e.g. topic = "TeamPM progress this week".
3. Drag in an LLM node, write its prompt as a template that references {{topic}}, then drag a directed edge Set → LLM.
4. Drag in a Write node (the engine StepType is Write, not WriteFile) to land the LLM output to a file, then connect LLM → Write.
5. Click the one-click Run and follow the Detail Log for the Set#1 → LLM#2 → Write#3 start/done events until the run state settles to done.
6. Check the ASSETS rail to confirm weekly-summary.md was written with correct content — that file is this three-node chain's artifact.

## EXAMPLE / EXERCISE

A minimal "draft a weekly report" chain, assembled and run in this order:

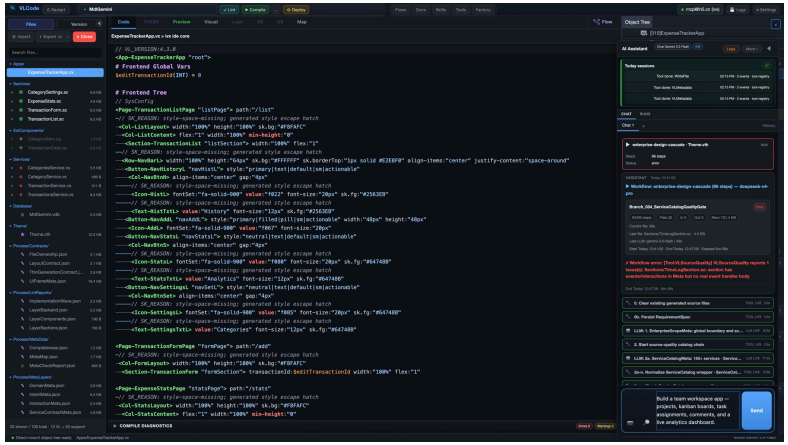
```
# 1. Drag Set, assign the seed value
Set(topic = "TeamPM progress this week")
# 2. Drag LLM, prompt references the variable, edge Set → LLM
  → LLM(prompt = "Write a three-sentence weekly summary based on {{topic}}")
# 3. Drag Write, land to disk, edge LLM → Write
  → Write(path = "weekly-summary.md", content = {{LLM.output}})
```

Three nodes, two edges. After clicking Run the Detail Log shows:

```
Set#1    done
LLM#2    done (provider=deepseek-api)
```

PART IX: COMPLETE CASE STUDIES

# Case 1: Run And Inspect



Detail Log — the live run ledger and node results

## CORE IDEA

Running and inspecting is the step that turns "it ran" into "I understand what happened." VL Agent puts the readable conclusion in the floating AI Chat and the full run ledger (node progress, file writes, tool I/O, validation, errors) in the Detail Log — two surfaces with separate jobs, never duplicated. Reading both is how you decide whether a run truly succeeded.

## OPERATIONS

1. After clicking Run, follow the Detail Log (right) for per-node start/done events and the files it writes.
2. Switch to the floating AI Chat panel to read the final readable summary and any human-decision cards.
3. Expand a node in the Detail Log to inspect its tool I/O and node result.
4. Confirm the run state settles to one of the terminal states done / error / aborted, not stuck mid-flight.
5. Check the ASSETS rail to verify the generated file exists and its content is correct.

## EXAMPLE / EXERCISE

After running the Case 1 weekly-report chain, the Detail Log should show an event sequence like:

```
Set#1   done
LLM#2   done (provider=deepseek-api)
Write#3 done → weekly-summary.md
run: done
```

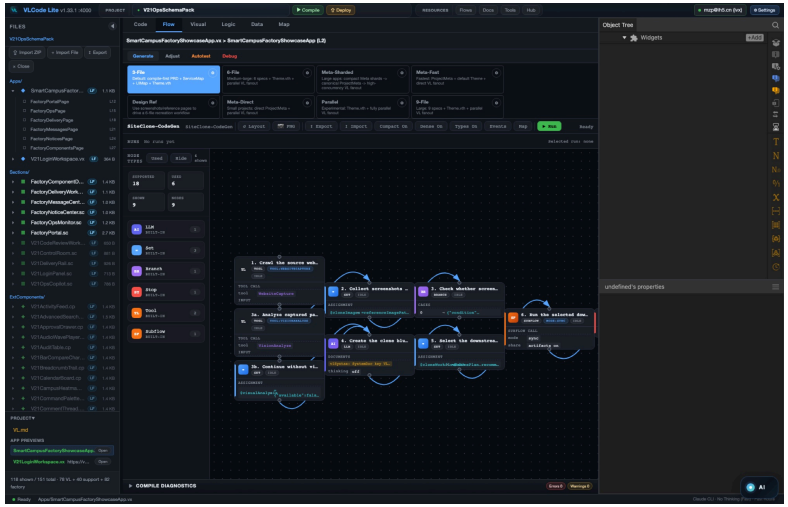
while the AI Chat shows just one summary line, "Weekly report generated, written to weekly-summary.md," without repeating every node.

## PITFALL

Don't assume success from the AI Chat "done" alone — the summary can lag behind a real error. Always confirm in the Detail Log that the run actually settled to done, and verify file content in ASSETS to avoid a false "looks complete" impression.

## CHECKPOINT

# Case 2: Generate An Approval Agent App From One Line



Flow Mode — workflow picker, codegen, autotest

## CORE IDEA

This is VL Agent's signature power: from one natural-language intent, a single Flow generates a runnable Agent App for you — you don't hand-build the DAG, you describe what you want and a workflow produces a manifest plus an entry workflow (a .vflow). For the fast lane use generate-agent-app-quick (no gates; one LLM pass turns intent straight into a runnable app and deterministically normalizes the manifest); when you want approval gates use generate-agent-app-guided (human-in-the-loop, turning intent into a runtime-first app with its gates built in).

## OPERATIONS

1. Open the floating AI Chat, or pick a generation lane in Flow Mode's workflow picker: for an approval gate choose generate-agent-app-guided (human-in-the-loop, gated); use generate-agent-app-quick only for a gate-free fast prototype.
2. Write the intent in one line: who submits, which fields, who reviews, what approve/reject/request_changes each do, which table approval writes to, and who is notified.
3. Let the Flow generate: the guided lane pops cards at key nodes for you to confirm planned surfaces, then produces a top-level app block plus an entry workflow (one .vflow) and normalizes the manifest deterministically (manifestFromFlow, no separate .app.json).
4. In the Manifest Inspector, check planned vs actual surfaces — that the Pause collection form (name/days/reason) and the Review approval gate (approve|reject|request_changes) were both recognized.
5. Click the one-click Run on the generated app to confirm it truly runs and pauses at the Pause node awaiting input, not just that the files exist.

## EXAMPLE / EXERCISE

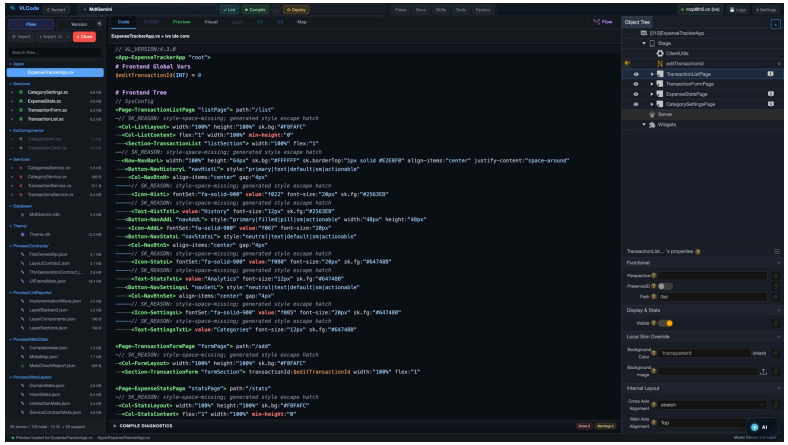
A one-line intent triggers the guided (gated) lane:

```
# 1. Flow Mode workflow picker → choose generate-agent-app-guided
/run generate-agent-app-guided
Intent: Build a "Leave Approval" Agent App — an employee submits leave (name/days/reason),
a manager Reviews (approve|reject|request_changes); on approve write the ApprovedLeave table
and notify the requester; on reject/request_changes send it back.
```



The generated output (VL Agent packs the app block + entry workflow into one .vflow):

# Case 2: From Intent To A Running App



A Pause/Review interaction card waiting for a human

## CORE IDEA

This topic brings the generated app to life: the approval Agent App from the previous section moves through the install → release → runtime lifecycle, with Pause/Review interaction cards letting a human gate at the node layer. A Pause node pops a human input form (text/textarea/number/select/checkbox); a Review node pops an approval card (approve/reject/request_changes) — so the whole chain runs from intent all the way to an app that is actually running and waiting on your decision.

## OPERATIONS

1. Click the one-click Run on the generated Agent App to enter its App chat console.
2. When the run pauses at a Pause node, fill the leave-request fields in the pop-up input form and submit.
3. When the run reaches the Review node, click approve, reject, or request_changes on the approval card.
4. Use /status in the App chat console to see which node it is paused at, and /resume to let it continue.
5. Open App Home — the workflow-first landing page — to see the DAG + manifest + core data + artifacts + run ledger.

## EXAMPLE / EXERCISE

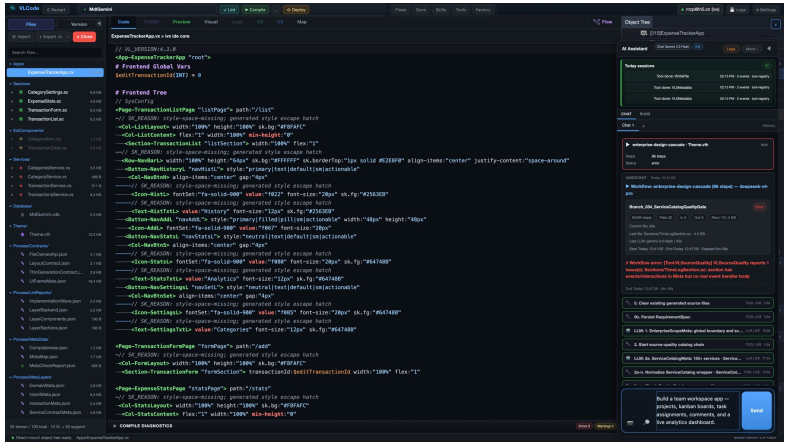
From intent to running, one interaction in the App chat console:

```
/start
> Pause: please fill in the leave request
  requester=Li Lei  days=2  reason=family matter
/status
> Run paused at Review#2 (manager approval), awaiting decision
(click approve on the card)
/status
> Run continues: VisualLogic writes Approved → done
```

## PITFALL

Don't keep clicking Run to "force it forward" while a Pause/Review card is still pending — the run gracefully pauses waiting for your input, and re-triggering only creates confusion. Use /status to see where it stopped, then /resume or click the card to advance.

# Case 2: Audit And Control Plane



Detail Log — the live run ledger and node results

## CORE IDEA

Audit and the control plane are the foundation that makes an approval Agent App trustworthy. Each workspace has a .vl-code/control-plane.sqlite recording the run ledger (history, timestamps, error stacks), node results, review resources (human decisions and audit), artifact lineage, and actor tasks. Every approve/reject leaves a trace here — which is exactly what separates a generated approval Agent App from a plain script.

## OPERATIONS

1. After running the approval Agent App, open the Detail Log to view the full run ledger and each Review's decision result.
2. Understand that the control plane lives at .vl-code/control-plane.sqlite in the current workspace, isolated per workspace and separate from the IDE workspace (instance-scoped).
3. In the Manifest Inspector, inspect resource lineage to trace where decisions and artifacts came from.
4. Note that switching workspaces reloads the control-plane DB, so the audit record follows the workspace.
5. When you need a per-port DB, launch with --isolated for an isolated control plane.

## EXAMPLE / EXERCISE

After an approval run, a Review resource in the control plane records roughly:

```
run_id    : run-2026-06-13-001
node      : Review#2 (manager)
decision  : approve
actor     : human
ts        : 2026-06-13T10:42:11
```

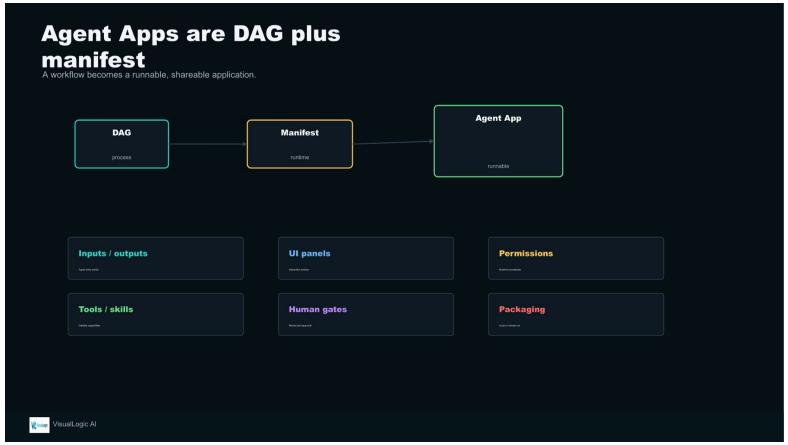
This row, with its timestamp and the run ledger, persists in .vl-code/control-plane.sqlite as an audit record.

## PITFALL

Don't rely on the AI Chat transcript for audit — chat is a transient presentation; the audit truth lives in control-plane.sqlite. And don't expect to see the previous workspace's decisions after switching, since they are isolated per workspace.

PART IX: COMPLETE CASE STUDIES

# Case 3: Fleet Parallel Delivery




Master Agent — branch scoring, pruning, and promotion

## CORE IDEA

Fleet parallel delivery uses the Master Agent runtime and Actor nodes to fan a batch of similar tasks out in parallel — for example generating several modules or working several tickets at once. The Master Agent's Planner/Evaluator/Scheduler/Arbiter loop schedules candidate branches, scores and prunes them, and promotes the mainline, while the actor-demo series (pm-intake/triage, frontend/backend module, qa-review, release-board) demonstrates exactly this fleet collaboration.

## OPERATIONS

1. Start from an actor-demo seed and locate the Actor nodes (distributed actor-task dispatch) on the DAG.
2. Use an Iteration or Channel node to fan a batch of inputs out into multiple parallel actor tasks (the engine StepType is Iteration, not Iterate; a homogeneous actor population can also be driven by a Swarm node).
3. In Settings, set workflowMaxConcurrency (default 5, range 1-20) to control the degree of parallelism.
4. Click Run and watch multiple branches advance simultaneously on the DAG, with nodes showing pruned/expanded/promoted states.
5. Inspect each actor task's node result in the Detail Log, confirm they all settle to a terminal state, and note each actor task is ledgered in the control plane at .vl-code/control-plane.sqlite.

## EXAMPLE / EXERCISE

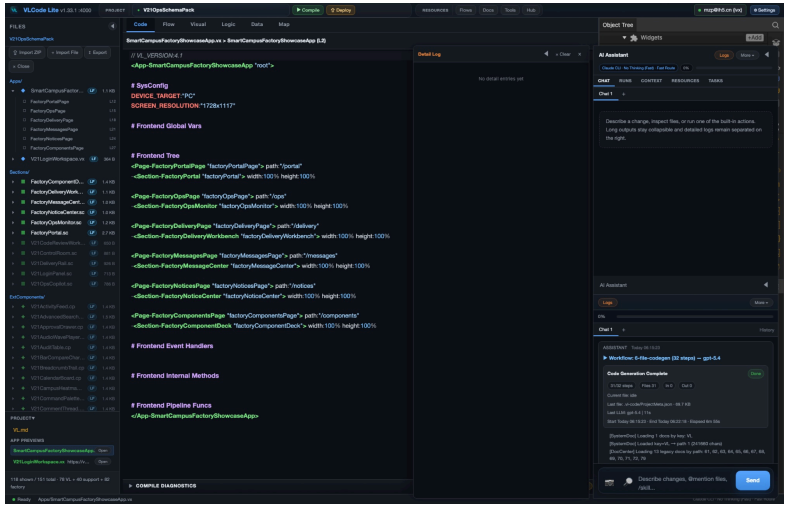
Skeleton for generating four modules in parallel:

```
Set(modules = ["intake","triage","qa","release"])
  → Iteration(over modules)           # engine StepType: Iteration
      → Actor(module-builder: dispatch one actor task per module)
  → Score(rank each branch's output)
  → Arbiter promotes the best branch to the mainline
```

With workflowMaxConcurrency=5, all four actor tasks can run at once. The Detail Log appends them in parallel:

```
Actor#3a done (intake)   Actor#3b done (triage)
Actor#3c done (qa)       Actor#3d done (release)
Score#4 done → promote triage to mainline
run: done
```

# Case 3: Route To Different Providers



Settings — provider, model, and runtime configuration

## CORE IDEA

Routing to different providers lets you align cost and capability with the task: reserve the expensive strong model for critical reasoning, and run batch fan-out on a cheap, fast one. VL Agent supports eight providers — anthropic-api / openai-api / deepseek-api / kimi-api / glm-api / gemini-api / minimax-api / qwen-api — with cost-sensitive defaults of DeepSeek V4 Pro (chat) + Kimi 2.5 (workflow + fanout), and you can override provider/model on an individual LLM node.

## OPERATIONS

1. In Settings → LLM, set provider/model for chat and workflow separately; defaults are DeepSeek V4 Pro + Kimi 2.5.
2. Set workflowFanoutProvider to route fleet fan-out sub-tasks to the cheap, fast model.
3. In a critical LLM node's detail drawer, override the provider/model to use a strong model just there.
4. If you must mix vendors, enable allowMixedLlmProviders (the default advice is to keep the same company unless deliberately mixing).
5. After a run, check the actual provider on each LLM node in the Detail Log to confirm routing took effect.

## EXAMPLE / EXERCISE

Routing providers in a fleet delivery:

```
Settings → LLM
  chatModel        = deepseek-api / DeepSeek V4 Pro
  workflowModel    = kimi-api / Kimi 2.5
  workflowFanoutProvider = kimi-api      # fan-out sub-tasks go cheap and fast
Node override:
  Review-Summarizer node → provider = anthropic-api (strong model for the critical summary)
```

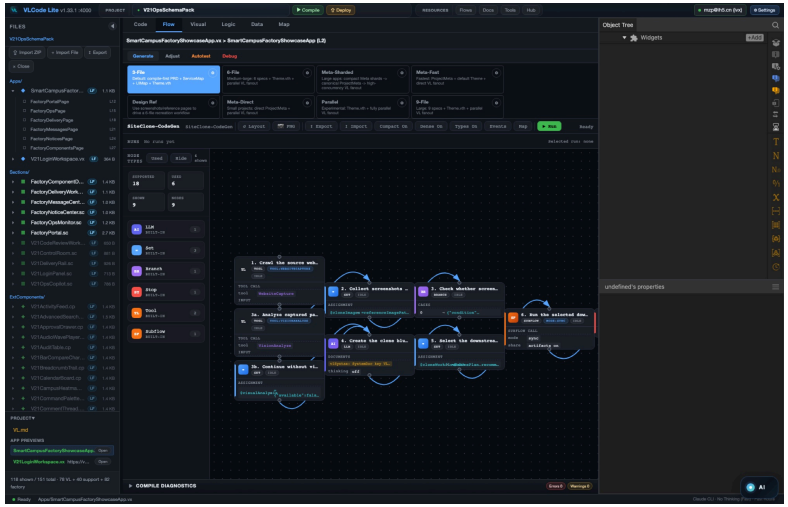
The Detail Log shows `LLM#7 done (provider=anthropic-api)` while the other fan-out nodes show kimi-api.

## PITFALL

Don't blindly put the most expensive model on every node — a fleet fan-out has many nodes, and all-strong sends cost soaring. And don't casually enable allowMixedLlmProviders to

PART IX: COMPLETE CASE STUDIES

# Case 4: Website Clone Studio



Flow Mode — workflow picker, codegen, autotest

## CORE IDEA

The website clone studio shows how to reverse-engineer a real site into a runnable Agent App / workflow artifact. On the VLCode side there is siteclonemax-explore (authenticated deep crawl + network capture) and siteclonemax-pipeline (reverse-engineer a site into real VL); in VL Agent you can orchestrate this crawl-parse-generate chain in Flow Mode and save the result as a reusable .vflow.

## OPERATIONS

1. In Flow Mode's workflow picker, choose siteclonemax-explore (authenticated deep crawl + network capture) and open the DAG canvas to see the crawl chain.
2. Then run siteclonemax-pipeline to reverse-engineer the captured results into real VL (pages + data model).
3. Use Tool nodes (the agent-os tool layer: HTTP / file I/O) for the deep crawl and network capture, and LLM nodes to parse them into structured pages.
4. Use Pause/Review nodes to manually check the crawl scope and generated output at key points, avoiding over-capture; land VL artifacts with a Write node (StepType is Write).
5. In Flow Bundle Mode, save the whole chain as a .vflow and register it to the local Resource Center under .vl-code/registry/workflows/<id>.json for reuse.

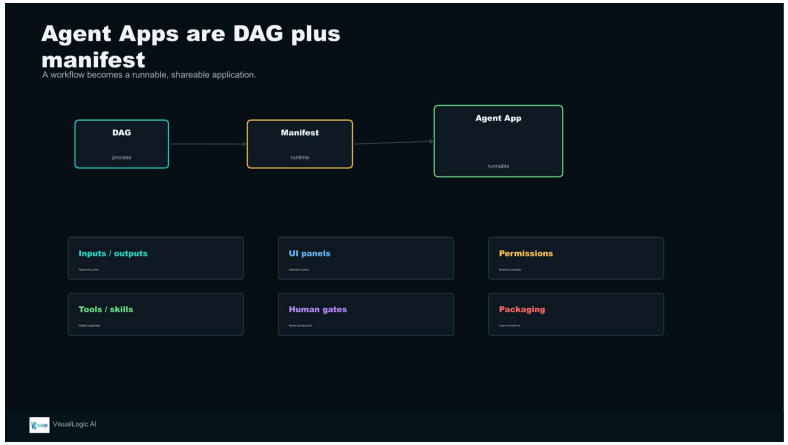
## EXAMPLE / EXERCISE

Skeleton and trigger of a site-clone chain:

```
# 1. Flow Mode workflow picker → siteclonemax-explore
/run siteclonemax-explore
Intent: Clone a public product landing page and reverse-engineer it into VL pages + a data model.
DAG:
  Tool(HTTP deep crawl) → Tool(network capture) → LLM(parse pages/endpoints)
    → Review(confirm crawl scope) → Write(generate VL artifact)
# 2. After scope is confirmed, reverse-engineer into real VL
/run siteclonemax-pipeline
→ produces VL pages (.sc) + data model (.vdb), written to ASSETS
# 3. Flow Bundle Mode → Save as site-clone.vflow → register .vl-code/registry/workflows/
```

PART IX: COMPLETE CASE STUDIES

# Case 5: Goal-Convergence Console




Master Agent — branch scoring, pruning, and promotion

## CORE IDEA

The goal-convergence console demonstrates how the Master Agent iterates toward a goal until it converges. The Planner/Evaluator/Scheduler/Arbiter loop expands candidate branches, prunes by score, preserves versions, and promotes the best to the mainline; the DAG shows pruned/expanded/promoted states directly. It turns the "explore many, pick the best, converge" form of process intelligence into a visible, controllable, replayable run.

## OPERATIONS

1. Pick a goal-convergence demo workflow and find the complexity nodes (Score / Grow / Topology) on the DAG canvas — all native engine-4.13.1 StepTypes.
2. In a Set node, state the goal and the convergence criterion clearly (e.g. a score threshold or iteration cap maxIter).
3. Click Run and watch the Master Agent's Planner/Evaluator/Scheduler/Arbiter loop expand multiple candidate branches and score them with Score nodes.
4. On the DAG, watch branches get pruned, expanded, and promoted, with the mainline moving forward each iteration.
5. Use the DAG Operator in Chat to query the current best branch and variables in natural language (no LLM round-trip), and control_pause to step in if needed; each iteration and promotion is ledgered in the control plane at .vl-code/control-plane.sqlite.

## EXAMPLE / EXERCISE

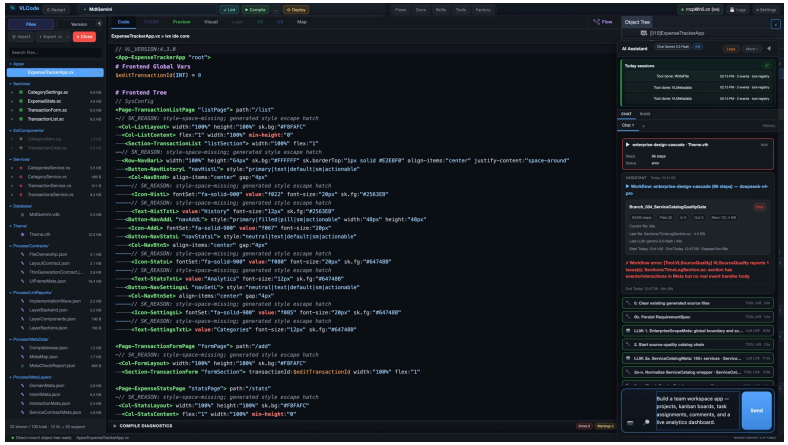
Goal-convergence skeleton and natural-language queries:

```
Set(goal="headline CTR ≥ 8%", maxIter=5)
  → Grow(expand candidate headline branches)
  → Score(rate each candidate)
  → Arbiter(promote the top score to mainline, else iterate again)
```

As it runs, branches prune/promote on the DAG and the Detail Log records each round:

```
iter 1: Score 4 candidates → best=6.2% (below target)
iter 2: Grow 3 more → Score → best=7.4%
iter 3: best=8.3% ≥ 8% → Arbiter promotes to mainline → done
```

# Case 6: Debug A Failed Run



Detail Log — the live run ledger and node results

## CORE IDEA

Debugging a failed run relies on the full traceability in the control plane, not guesswork. Once a run settles to the error terminal state, the Detail Log gives you the run ledger (with error stacks), node results, and tool I/O; apply the root-cause-first principle to pinpoint the exact failing node and see its real error rather than masking it with fallback.

## OPERATIONS

1. Confirm the run has settled to the error terminal state (not stuck) by checking the run status at the top of the Detail Log.
2. Expand the red failing node in the Detail Log and read its error stack and node result.
3. In Chat, use the DAG Operator's query_node for that node's details and query_var for the relevant variables.
4. Compare against the failing node's tool I/O to decide whether the input data, the prompt, or the tool call itself broke.
5. If needed, use studio-mode dev_inspect_code / dev_validate to check the node definition itself.

## EXAMPLE / EXERCISE

Tracing an HTTP Tool node failure:

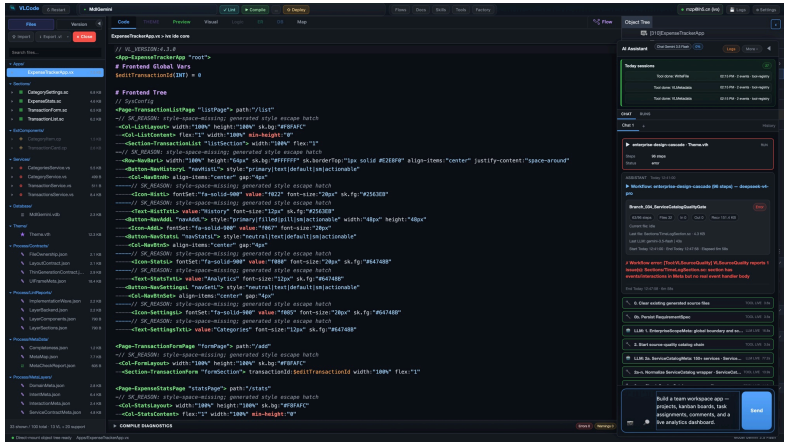
```
Detail Log:
  Tool#4 error → HTTP 401 Unauthorized
  stack: missing auth header in request
Chat:
  query_node Tool#4
  query_var apiToken      # apiToken is empty
```

Root cause: an upstream Set never injected apiToken, so the request lacked an auth header — fix the variable, rather than adding an "ignore error and continue" fallback to the Tool node.

## PITFALL

The worst move is adding error-swallowing or a silent fallback to make the run "look passed" — that masks the real error, lowers code quality, and makes future debugging harder. Errors must surface to the run ledger and the user; fix the root cause, don't route around it.

# Case 6: Rerun From The Failed Node



Detail Log — the live run ledger and node results

## CORE IDEA

Rerunning from the failed node is the practical payoff of VL Agent's checkpoint mechanism: instead of re-running the whole workflow from the top, once you've located and fixed the root cause you continue only from that node (or after it). Native checkpoint / resume / rerun-from-step sharply cuts the debugging cost of long-chain workflows, saving the tokens and time of the already-successful upstream nodes.

## OPERATIONS

1. Fix the failing node's root cause during debugging (e.g. inject the missing variable or correct the prompt).
2. Select that failed node in the Detail Log or on the DAG and trigger rerun-from-step / control_rerun.
3. Confirm the checkpoints of the already-successful upstream nodes are reused, not re-executed, continuing only from the failure point.
4. Watch the run advance from that node, with new events appended to the run ledger.
5. Confirm the run settles back to done and verify the final artifact in ASSETS.

## EXAMPLE / EXERCISE

Rerun from the failed node after fixing apiToken:

```
Chat:
  control_rerun Tool#4        # continue only from Tool#4, don't re-run Set#1..#3
Detail Log:
  Tool#4 done (resumed from checkpoint)
  → LLM#5 done → Write#6 done
  run: done
```

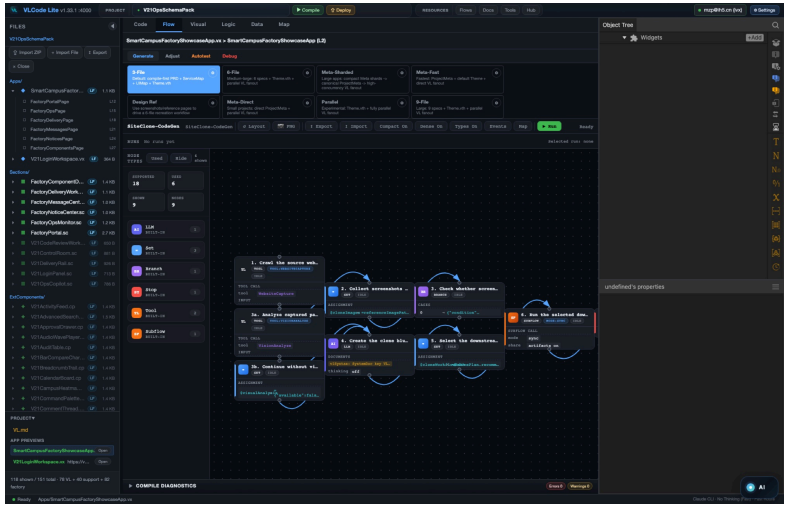
The checkpoints of the three already-successful nodes are reused directly, and the new events are appended to the run ledger in the control plane at .vl-code/control-plane.sqlite.

## PITFALL

Don't keep rerunning from the same node without fixing the root cause — if the same error repeats twice in the same place, the problem isn't a fluke at that step but upstream data or the node definition; go back to the source for the root cause before rerunning.

PART IX: COMPLETE CASE STUDIES

# Case 7: Hand A .vflow To A VLCode Team



Resource Center — registering and syncing a .vflow

## CORE IDEA

This is the last mile that hands VL Agent's output over to VLCode: the workflows you generate or orchestrate in VL Agent (including new reusable workflows produced by workflow-factory-main) are self-contained .vflow bundles; once registered to the Resource Center, a VLCode developer imports the same .vflow to drive enterprise app generation. The path is "generate in VL Agent → Resource Center → VLCode."

## OPERATIONS

1. In Flow Bundle Mode, declare the top-level app block and Save-As the workflow to a .vflow (self-contained bundle: workflow JSON + derived manifest + optional subflows/docs/prompts; it is not a VL source file and never goes through the VL parser).
2. Register it to the local Resource Center at .vl-code/registry/workflows/<id>.json.
3. Use Resource Center cloud sync to push the .vflow to the shared layer the team can see.
4. On the VLCode side, a developer browses/imports the .vflow from the Resource Center, or drops it into the Flow panel and picks one of the 8 import categories: Generate / Enterprise / Clone / MetaData / Test / Adjust / Autofix / Quality (there is no Custom category).
5. The VLCode developer customizes it as the team needs (incremental workflows like add-page / feature-adjust) and publishes it for the team to use.

## EXAMPLE / EXERCISE

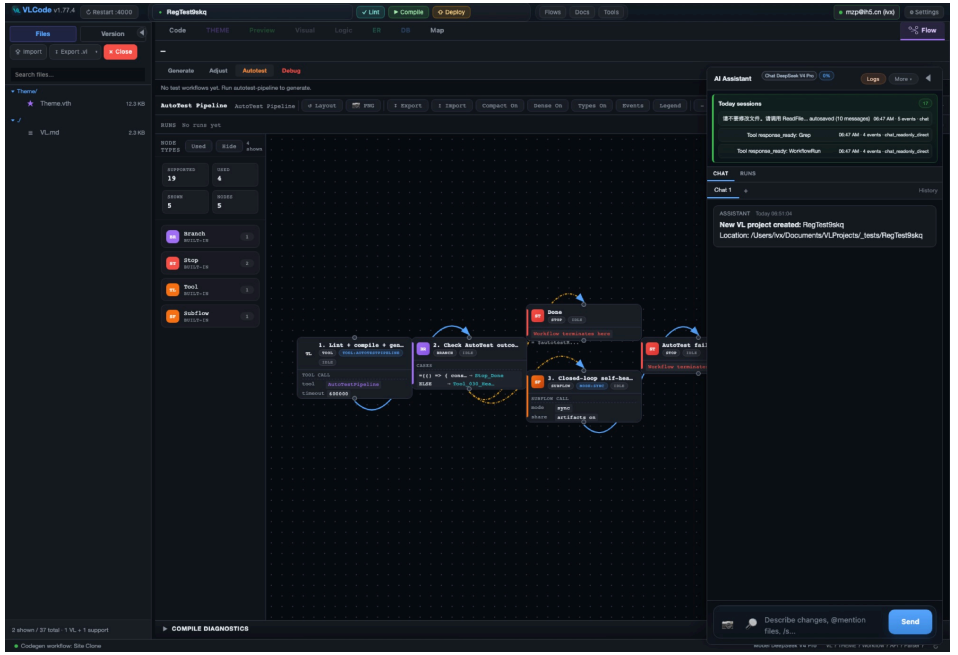
Generate a reusable workflow with workflow-factory and hand it over:

```
# 1. Generate on the VL Agent side
/run workflow-factory-main
Intent: Generate a "Customer CRM data import" workflow for a VLCode team to reuse — input a CSV, output a leads table + an import report.
→ produces customer-crm-import.vflow (self-contained bundle: workflow JSON + app block + derived manifest)
# 2. Register + cloud sync
→ register: .vl-code/registry/workflows/customer-crm-import.json
→ Resource Center cloud sync (push to the team's shared layer)
# 3. Import on the VLCode side
VLCode → Flow panel "+ Upload .vflow" → choose category Generate → import
→ the .vflow now drives CRM app generation; the developer then wires a real backend with add-service
```

CHAPTER

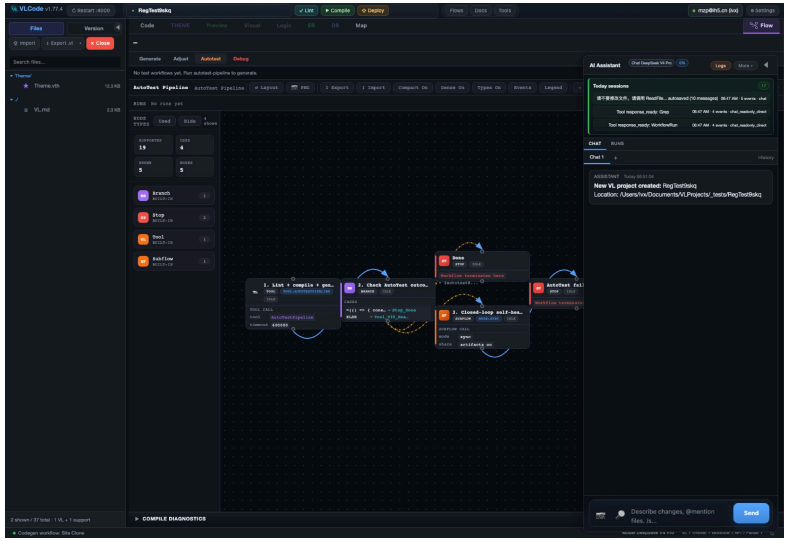
# Part X: Appendices And Quick Reference

- This chapter turns the part into executable lessons.
- Each page explains why it matters, then gives operations, an example, and a common pitfall.



Bottom status bar: provider, model, ports, compile

# Port Cheat Sheet



Bottom status bar: provider, model, ports, compile

## CORE IDEA

Every VL Agent service binds a fixed port and never silently falls back to a random one. Memorizing this table lets you tell at a glance whether a process is the IDE, the kernel, or a running Agent App runtime. On a port clash the correct move is to clear the stale process, not switch ports.

## OPERATIONS

1. Simplest is the desktop app (a .dmg/.exe/AppImage from releases.visuallogic.ai, one-click; it brings up the IDE and gateway internally).
2. For from-source development: `npm start` serves the Web IDE on :3200, range 3200-3209.
3. Gateway / headless runtime: `npm run start:gateway`, listens on :4310, range 4310-4319, for remote / automation.
4. Remember the kernel is fixed at 4300 and the VL parser at 3456.
5. Generated Agent App runtimes land in the 4610-4699 band.
6. Confirm the current listening port and LLM provider in the bottom status bar; before launching, check the port with `lsof -ti:3200`.

## EXAMPLE / EXERCISE

Full VL Agent port table:

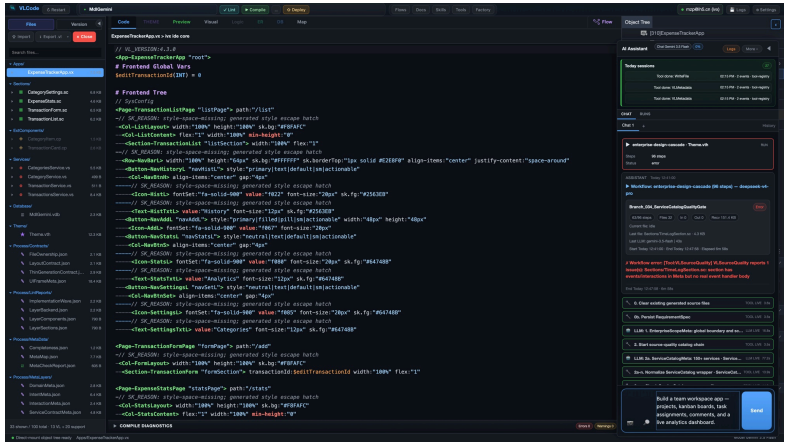
```
VL parser            3456
IDE (Web)            3200–3209  (default 3200)
Kernel               4300
Gateway / headless   4310–4319  (default 4310)
Agent App runtime    4610–4699
```

The desktop app launches in one click; from source, `npm start` serves the Web IDE and `npm run start:gateway` brings up the headless gateway.

## PITFALL

When a port is occupied, do not just bump to 3201 or some other port. Fixed-port discipline means clearing the old process ( `lsof -ti:3200 | xargs kill -9` ) and reusing the original port — otherwise anyone following the convention to find your service hits nothing.

# CLI Command Cheat Sheet



Detail Log — the live run ledger and node results

## CORE IDEA

VL Agent's CLI entry points let you manage workflows and the runtime without opening the IDE. `bin/vlflow.js` covers the full workflow lifecycle; to bring up services prefer the npm scripts — `npm start` for the Web IDE and `npm run start:gateway` for the headless gateway. This matters most for automation, CI, and headless servers.

## OPERATIONS

1. Use `bin/vlflow.js list` to list registered workflows and `show <id>` to view a definition.
2. Use `vlflow validate <id>` for structural validation and `run <id>` to trigger a run.
3. Use `vlflow resume` , `state` , `ledger` , `artifacts` to resume a run, query state, read the run ledger, and fetch artifacts.
4. Use `npm start` to serve the Web IDE (:3200) and `npm run start:gateway` for the headless gateway (:4310).
5. The kernel is fixed at 4300 and is brought up internally by the desktop app / launch scripts — you rarely start it by hand.

## EXAMPLE / EXERCISE

Full workflow lifecycle on the CLI:

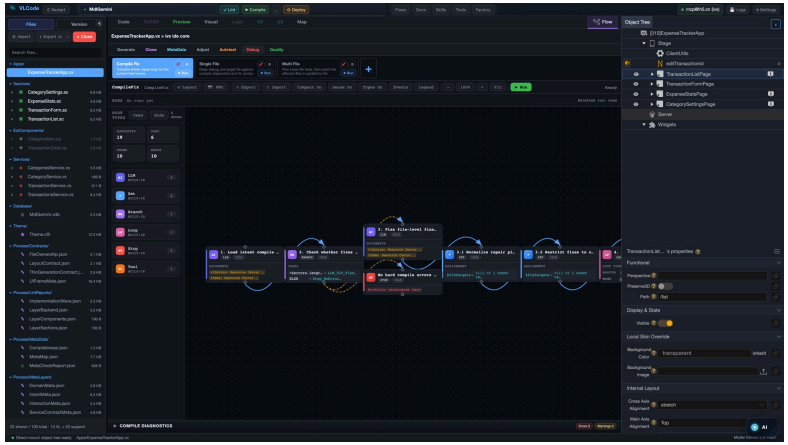
```
node bin/vlflow.js list
node bin/vlflow.js show approval-flow-runtime-main
node bin/vlflow.js validate approval-flow-runtime-main
node bin/vlflow.js run approval-flow-runtime-main
node bin/vlflow.js state <runId>
node bin/vlflow.js ledger <runId>
node bin/vlflow.js artifacts <runId>
```

Bring up services: `npm start` (Web IDE :3200) / `npm run start:gateway` (gateway :4310).

## PITFALL

Do not confuse the local IDE with the gateway: `npm start` (:3200) is the Web IDE for your local use, while `npm run start:gateway` (:4310) is the headless gateway for remote /

# Node Type Cheat Sheet



A node detail drawer open over the DAG

## CORE IDEA

Workflow Spec 4.1 (Workflow-Engine 4.13.1) defines a set of canonical StepTypes (22 in all) — the building blocks you assemble on the DAG canvas. Knowing each node's job lets you read a DAG at a glance: where it calls AI, where it writes files, where a human gates, and where it branches.

## OPERATIONS

1. Use LLM nodes for AI steps; call a backend / endpoint / component with Service / API / Component; the executor layer (file I/O, HTTP, Git, VL compile, component factory) is provided by the agent-os Tool layer.
2. Use Set to assign variables; use Write / Download / Unzip for I/O (the file-writing StepType is Write, not WriteFile).
3. Use Pause for human gating (input form) and Review for a structured decision (approve/reject/request_changes).
4. Branch conditionally with Branch; graphical if/then/switch logic runs as VisualLogic via a customHandler (compiled to runEvent(ctx)); there is no Formula StepType.
5. Use Actor / Swarm for distributed work; Subflow for child flows; Loop / Iteration / Channel / Stop / Noop for control; Score / Grow / Topology / GraphPatch for DAG complexity.

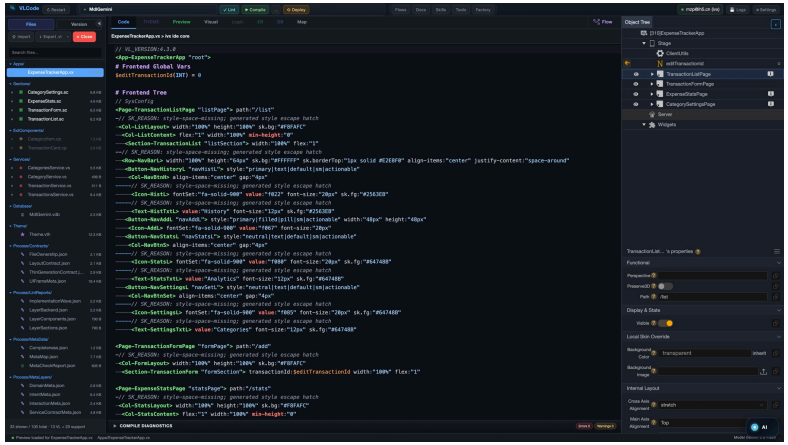
## EXAMPLE / EXERCISE

The 22 canonical StepTypes, grouped by job:

```
AI / reasoning : LLM
Backend / API  : Service, API, Component
Variables      : Set
I/O            : Write, Download, Unzip
Human gates    : Pause (input form), Review (approve/reject/request_changes)
Control flow   : Branch, Loop, Iteration, Stop, Noop
Distributed/sub: Actor, Swarm, Subflow
DAG complexity : Score, Grow, Channel, Topology, GraphPatch
```

Two more are commonly mentioned but are not core StepTypes: Tool (the agent-os tool layer for file I/O / HTTP / Git / VL compile / component factory) and VisualLogic (graphical if/then/switch, run via a customHandler).

# Slash Command Cheat Sheet



The floating AI Assistant panel and chat console

## CORE IDEA

The floating AI Chat panel supports /slash commands, and every generated Agent App ships an App chat console. The two command sets control IDE conversations and the lifecycle of a running app instance respectively. Knowing them lets you drive an entire run without leaving chat.

## OPERATIONS

1. Use /slash commands in the IDE's floating Chat panel for structured directives and streaming conversation.
2. In a generated Agent App's chat console, use `/start` to launch an app instance.
3. Use `/pause` to pause, `/resume` to continue, and `/stop` to stop the current instance.
4. Use `/status` to query the instance's current state.
5. Use `/rerun-step` to rerun from a specific step.

## EXAMPLE / EXERCISE

Agent App chat console command set:

```
/start          launch the app instance
/pause          pause
/resume         resume
/stop           stop
/status         query state
/rerun-step     rerun from a step
```

E.g. when an approval app is waiting on a human, type `/status` in the console to see which Pause node it stopped at, then `/resume` after handling it.

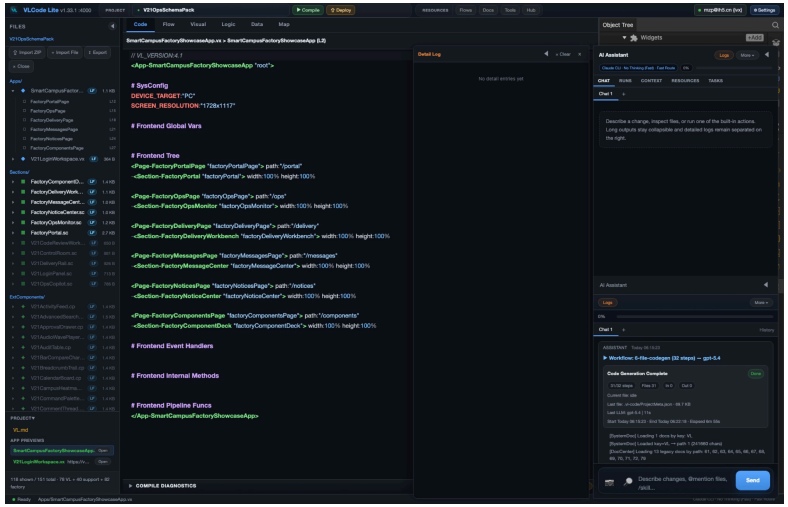
## PITFALL

Don't conflate the App chat console's `/start /pause /resume /stop /status /rerun-step` with the DAG Operator's natural-language intents (query_status, control_pause, etc.): the former are slash commands acting on an App instance's lifecycle, the latter is LLM-round-trip-free run control. Used in the wrong place, the command isn't recognized.



## CHECKPOINT

# Provider And Model Cheat Sheet



Settings — provider, model, and runtime configuration

## CORE IDEA

VL Agent manages providers and models under Settings -> LLM, supporting anthropic-api / openai-api / deepseek-api / kimi-api / glm-api / gemini-api / minimax-api / qwen-api. The cost-sensitive defaults are DeepSeek V4 Pro (chat) + Kimi 2.5 (workflow + fanout). The Chat model and Workflow model can differ, but the provider company should stay consistent by default.

## OPERATIONS

1. Pick a provider under Settings -> LLM and paste a key (or use the SecretRoot vault).
2. Set chatModelPolicy = auto | fast | strong; auto upgrades the model for long/complex prompts or multi-round tool use.
3. Set reasoningLevel = basic | standard | high and enable thinkingEnabled as needed.
4. Set workflowMaxConcurrency (default 5) and workflowFanoutProvider to control fan-out.
5. Enable allowMixedLlmProviders explicitly to mix companies; otherwise keep chat/workflow on the same company.

## EXAMPLE / EXERCISE

Cost-sensitive recommended configuration:

```
Provider (chat)     : deepseek-api  -> DeepSeek V4 Pro
Provider (workflow) : kimi-api      -> Kimi 2.5  (also fanout)
chatModelPolicy     : auto
reasoningLevel      : standard
workflowMaxConcurrency : 5
allowMixedLlmProviders : false
```

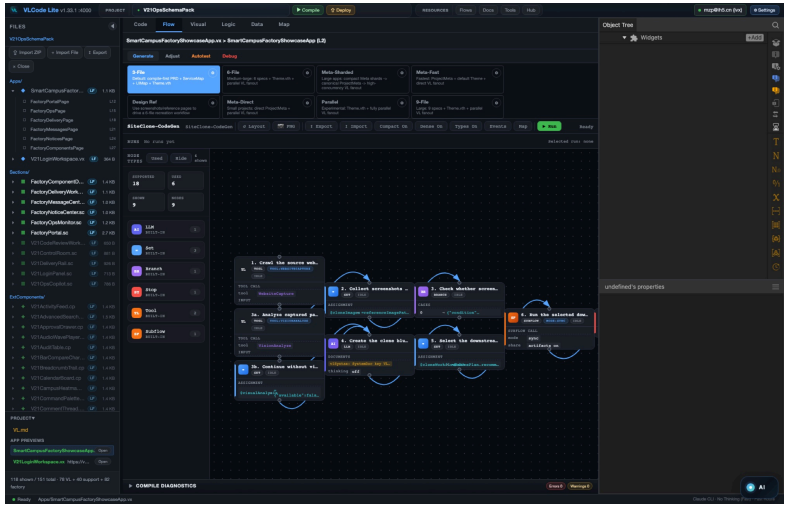
## PITFALL

Don't casually set Chat to Anthropic and Workflow to DeepSeek while leaving allowMixedLlmProviders off — unless you deliberately mix, keeping the same company is simpler and cheaper. The default DeepSeek V4 Pro + Kimi 2.5 pairing is tuned for cost-sensitive work, so don't reflexively switch everything to the priciest model.



## CHECKPOINT

# Agent App Generation Cheat Sheet



Flow Mode — workflow picker, codegen, autotest

## CORE IDEA

This is the cheat sheet for VL Agent's signature power: you describe an intent in one sentence and a Flow generates a runnable Agent App (a manifest + an entry workflow). Different generation lanes trade speed against quality — fast lane, human-in-the-loop, enterprise quality gates, component-equipped, scaffold, staged, the official mother workflow, plus the lane that produces workflows shipped to VLCode.

## OPERATIONS

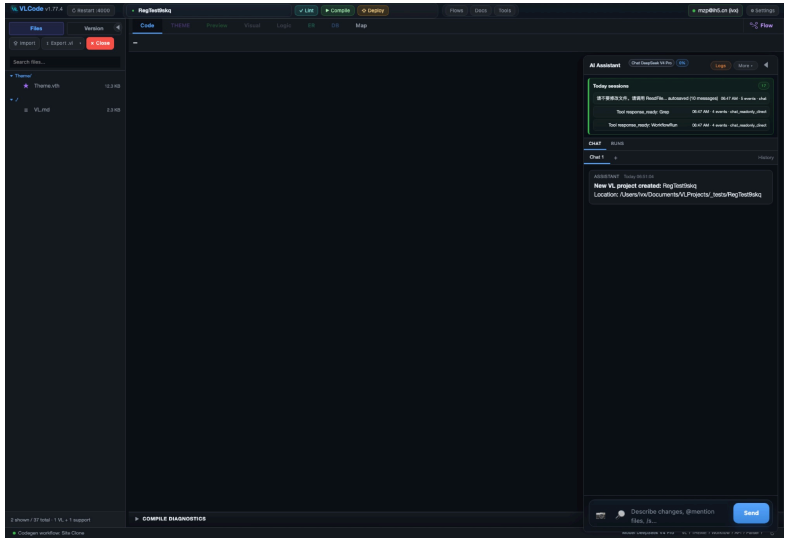
1. Fastest, no gates: use generate-agent-app-quick — one LLM pass turns intent directly into a runnable app and normalizes the manifest.
2. Controlled, human-in-the-loop: use generate-agent-app-guided (the headline 'Flow that generates an App') to turn intent into a runtime-first app with gates.
3. Enterprise quality gates: use generate-agent-app-cascade, a staged planning cascade (solution shape -> product brief -> requirement -> interaction contract -> surface contract -> workflow).
4. Equip UI directly: use generate-agent-app-with-components to filter the local Component Factory for matching components; for a full scaffold from a description use agent-app-scaffold-main; for a VLC-style staged chain use agent-app-3file-main; when unsure use the official mother workflow agent-app-factory-main.
5. To produce reusable workflows shippable to VLCode use workflow-factory-main; to turn a paper into an app use paper-to-agent-app-main.

## EXAMPLE / EXERCISE

Generation-lane cheat sheet:

```
generate-agent-app-quick            one line -> runnable app (no gates, fastest)
generate-agent-app-guided           human-in-the-loop, gated (headline)
generate-agent-app-cascade          enterprise quality gates, staged cascade
generate-agent-app-with-components  generate + Component Factory UI
agent-app-scaffold-main             full scaffold from a description
agent-app-3file-main                VLC-style staged chain
agent-app-factory-main              official 'mother' workflow (intent->plan->validate->gen->scaffold)
workflow-factory-main               generate reusable workflows (-> ship to VLCode)
paper-to-agent-app-main             paper -> Agent App
```

# .vflow Structure Cheat Sheet



Flow Bundle Mode — editing a .vflow directly

## CORE IDEA

.vflow is the single connector between VL Agent and VLCode, and the single-file model for an Agent App. Under AAS spec v2.2.0 an app pack = entry workflow + a top-level app block (no separate .app.json), and the manifest is derived at load time from the app block plus Pause/Review steps. The bundle is self-contained: workflow JSON + manifest + optional subflows/docs/prompts (plus optional embedded VL surfaces).

## OPERATIONS

1. Keep the workflow JSON as the body and add a top-level app block declaring app metadata.
2. Rely on Pause / Review steps to derive the human surfaces in the manifest (no hand-written .app.json).
3. Attach optional subflows, docs, and prompts where needed.
4. Open / edit / save the .vflow directly in Flow Bundle Mode, or Save as .vflow in AAS.
5. After saving, register it to the local Resource Center at `.vl-code/registry/workflows/<id>.json`.

## EXAMPLE / EXERCISE

.vflow bundle structure cheat sheet:

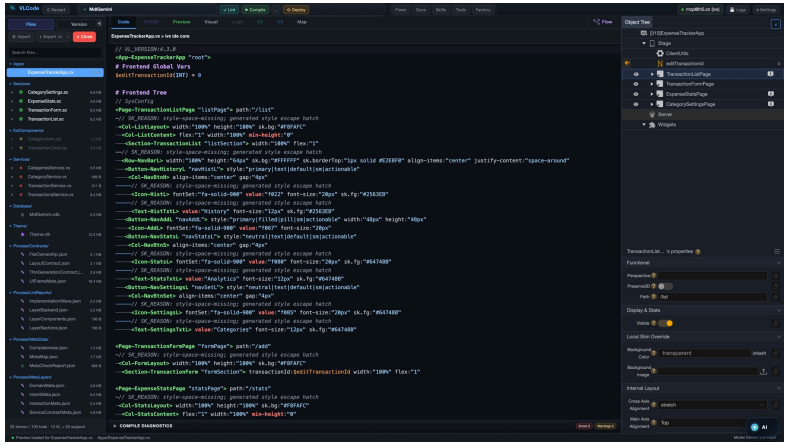
```
.vflow bundle
├─ workflow JSON          (entry workflow, DAG nodes + edges)
├─ app block (top-level)  (app metadata -> manifest derived at load)
├─ manifest (derived)     (planned vs actual surfaces + lifecycle)
└─ optional: subflows / docs / prompts / embedded VL surfaces
```

Registration path: `.vl-code/registry/workflows/<id>.json`.

## PITFALL

Don't look for or create a separate .app.json — AAS v2.2.0 is a single-.vflow file model, and the manifest is derived at load time from the app block plus Pause/Review steps. A hand-written standalone manifest file only conflicts with the derived one.

# DAG Operator Intent Cheat Sheet



The floating AI Assistant panel and chat console

## CORE IDEA

The DAG Operator lets you control a run in natural language without an LLM round-trip — commands map directly to fixed intents, fast and deterministic. It splits into query, control, and studio-dev intents, plus conditional watches. This is how you 'drive' a run without interrupting it.

## OPERATIONS

1. Query with query_status (overall), query_node (a node), query_var (a variable).
2. Control with control_pause, control_resume, control_stop, control_rerun.
3. Develop in studio mode with dev_list_nodes, dev_inspect_code, dev_validate.
4. Register a conditional watch: describe the trigger condition plus an action (stop / pause / notify) in natural language.
5. When a watch fires, the event returns to the Chat panel — no need to stare at Detail Log.

## EXAMPLE / EXERCISE

DAG Operator intent cheat sheet:

```
Query  : query_status | query_node | query_var
Control: control_pause | control_resume | control_stop | control_rerun
Studio : dev_list_nodes | dev_inspect_code | dev_validate
```

Conditional watch (just say it plainly): "if LLM_430 fails, stop" registers a watch with action=stop; when triggered the event appears in Chat.

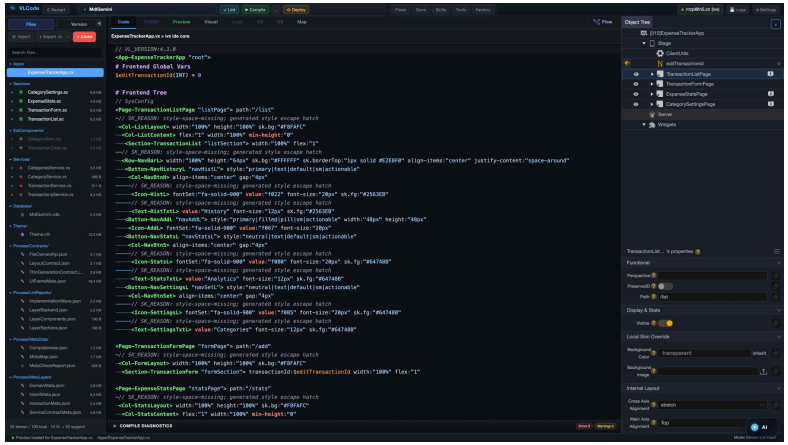
## PITFALL

Don't confuse the DAG Operator's control_pause/control_resume with the App chat console's /pause and /resume: the former are LLM-free run-control intents driven in natural language inside the IDE Chat, the latter are slash commands on an App instance. And don't register an 'infinite-poll' watch — state the trigger condition clearly.

## CHECKPOINT

You can query a node's status in one natural-language sentence and register a 'stop on failure' conditional watch whose event returns to Chat.

# AI Prompt Templates



The floating AI Assistant panel and chat console

## CORE IDEA

Good prompts determine generation quality. VL Agent generates 'intent -> runnable app', so a prompt should spell out the goal, key steps, human gates, and data shape. These are copy-ready templates covering quick generation, guided generation, and producing workflows shipped to VLCode.

## OPERATIONS

1. Quick prototype: one sentence describing the app plus its main flow, handed to generate-agent-app-quick.
2. Need an approval gate: name 'manager approves' and 'a request_changes path' in the description, and use generate-agent-app-guided.
3. Enterprise: list the stages and contracts (requirement, interaction, surface), and use generate-agent-app-cascade.
4. Produce a VLCode-bound workflow: describe the reusable workflow's input/output contract and hand it to workflow-factory-main.
5. Always state the data shape (tables/fields) and success criteria so later review is easy.

## EXAMPLE / EXERCISE

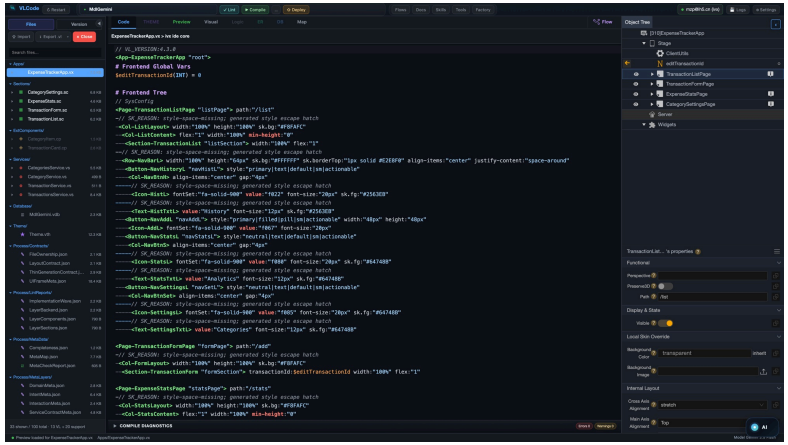
Copy-ready templates:

```
[Quick]   Generate a CustomerCRM intake Agent App: a form collects leads (name, company, email, need) -> write to the leads table -> notify sales.
[Guided]  Generate a procurement-approval Agent App: submit request -> manager Review (approve/reject/request_changes) -> on approve place the order, on
reject return to the applicant.
[Cascade] Generate an enterprise ticketing system: produce the requirement bundle, interaction contract, and surface contract in stages, then compile the
entry workflow with an SLA-escalation branch.
[Workflow Factory] Generate a reusable 'site-clone -> VL generation' workflow: input a URL, output a .vflow, register to the Resource Center for a VLCode
team to import.
```

## PITFALL

Don't write a vague prompt like 'build a CRM' — without the main flow, human gates, and data shape, the generator can only guess and the result drifts. Put the approval node, table fields, and success criteria into the prompt, and quality becomes controllable.

# Things To Avoid



A Pause/Review interaction card waiting for a human

## CORE IDEA

VL Agent has a few disciplines that run through everything; violating them breaks traceability, hides root causes, or creates confusion. This section collapses the cross-chapter traps into a single 'things to avoid' list — a final mental check before release.

## OPERATIONS

1. Don't write fallback / error-swallowing / silent-success code: let errors surface to the caller, Chat, or run ledger, find the root cause first, then change code.
2. Don't silently switch ports: on a clash clear the old process; fixed ports don't fall back.
3. Don't use non-English file names: VL source file names are English PascalCase, always.
4. Don't omit VL_VERSION on the first line, and don't mix indentation (use consecutive hyphens, not spaces).
5. Don't put secrets / cookies / API keys in committable files or prompts; use the SecretRoot vault.
6. Don't ship an approval-requiring enterprise app with the gate-free generate-agent-app-quick; and don't look for a separate .app.json.

## EXAMPLE / EXERCISE

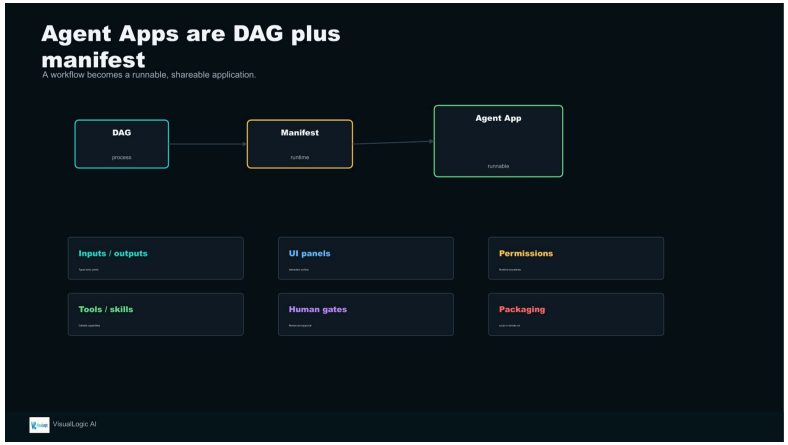
Pre-release 'do not' checklist:

```
✗ fallback / error-swallow / silent fallback / fake success
✗ switching ports on a clash (clear the process instead)
✗ non-English / non-PascalCase file names
✗ missing VL_VERSION first line / space indentation
✗ secrets in committable files or prompts
✗ shipping an approval app via the quick lane
✗ hand-writing a standalone .app.json
```

## PITFALL

The sneakiest trap is 'writing fallback to look done' — it masks the main error, lowers core code quality, and makes future debugging hard. Any failure should be reproduced, the real error captured, the call path inspected, then the root cause fixed.

# Learning Path: Week One




Master Agent — branch scoring, pruning, and promotion

## CORE IDEA

Week one's goal is to go from zero to 'I can generate and run an Agent App.' The path centers on the signature capability: first learn the interface and the DAG, then generate an Agent App from one sentence, then learn to run, pause, and read logs. One small goal per day avoids diving into complex workflows too soon.

## OPERATIONS

1. Day 1-2: Install the desktop app or serve :3200 from source, configure a provider (DeepSeek V4 Pro + Kimi 2.5), and run your first seed workflow end to end.
2. Day 3: Learn the LLM / Tool / Set / Pause / Review nodes on the DAG canvas, hand-build a minimal workflow, and Run it.
3. Day 4: Use generate-agent-app-quick to generate an approval Agent App from one sentence.
4. Day 5: Open the generated app, study the Unit Map / DAG Map and the derived manifest, and run it once with /start.
5. Day 6-7: Practice Pause/Resume, read events in Detail Log, and query status in natural language via the DAG Operator.

## EXAMPLE / EXERCISE

Week-one milestone prompts:

```
Day 4: "Generate an employee leave-approval Agent App: submit -> manager Review (approve/reject/request_changes) -> notify the applicant"
Day 6: In the App console, `/start` to launch; when it stops at a Pause, `/status` to see the node, then `/resume` after handling it
```

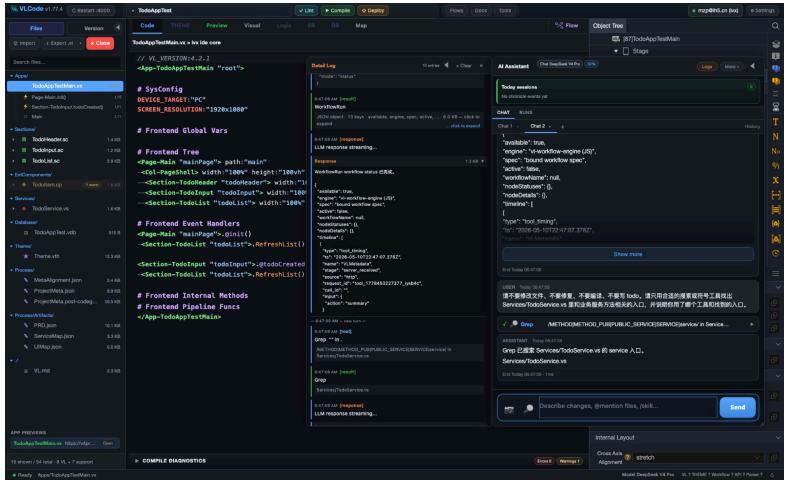
## PITFALL

Don't touch an enterprise quality-gated chain like generate-agent-app-cascade in week one — it has many stages and contracts and is easy to drown in. Build the 'intent -> runnable app' intuition with the quick lane first, then graduate to guided / cascade.

## CHECKPOINT

By the weekend you can generate an Agent App from one sentence with generate-agent-app-quick and run it through with /start + /status + /resume.

# Learning Path: First Month



Agent App Home — the workflow-first landing

## CORE IDEA

The first month's goal is to move from 'I can generate an app' to 'I can produce deliverable, VLCode-shippable assets.' The focus expands to enterprise generation lanes, the Master Agent runtime, and the full .vflow -> Resource Center -> VLCode shipping chain. This is the stage from personal toy to team delivery.

## OPERATIONS

1. Week 2: Master the differences between generation lanes (quick / guided / cascade / with-components / scaffold / 3file / factory) and choose by need.
2. Week 3: Use workflow-factory-main to generate a reusable workflow, Save as .vflow, and register to `.vl-code/registry/workflows/`.
3. Week 3-4: Understand the Master Agent's Planner/Evaluator/Scheduler/Arbiter loop, and watch branch scoring, pruning, and mainline promotion as states on the DAG.
4. Week 4: Walk the full shipping chain — .vflow -> Resource Center cloud sync -> import and customize in VLCode.
5. Throughout: maintain a real app with the agent-app lifecycle (modify / debug / test / review / package) and audit the run ledger via the control plane (control-plane.sqlite).

## EXAMPLE / EXERCISE

First-month delivery milestones:

```
Week 3: generate with workflow-factory-main -> Save as .vflow -> register `.vl-code/registry/workflows/<id>.json`
Week 4: Resource Center cloud sync -> a VLCode developer imports the same .vflow to drive enterprise app generation
Upkeep: agent-app-package-main produces a portable distribution snapshot (install review + release snapshot + rollback metadata),
        while the deliverable is still that .vflow with its app block
```

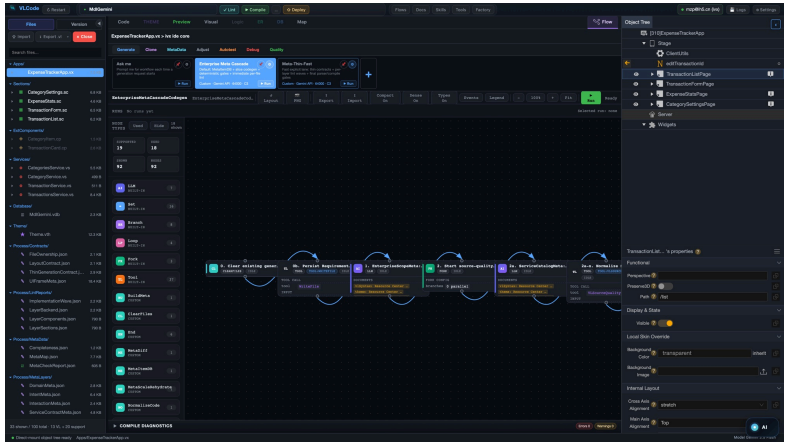
## PITFALL

Don't skip agent-app-review-main and jump straight to package: review checks the manifest/workflow/permissions/runtime/control-plane boundary. Skip it and the .vflow you hand a VLCode team may carry permission or runtime-boundary issues that only blow up after the team imports it.



## CHECKPOINT

# Glossary



VL Agent shell — the DAG canvas with the floating AI Assistant

## CORE IDEA

This glossary collects the core terms across the whole VL Agent tutorial as a quick reference. Mastering them lets you read the interface, the docs, and the precise meaning of the rest of this tutorial — especially Agent App, .vflow, manifest derivation, and control plane, which run through the entire book.

## OPERATIONS

1. VL Agent: local Agent OS + Agent App Studio (formerly AAS) that authors and runs workflows / Agent Apps.
2. Agent App: an app generated from intent = manifest + entry workflow (a .vflow); the signature power is generating it from one sentence.
3. .vflow: a single-file app pack = entry workflow + top-level app block; the connector between VL Agent and VLCode.
4. Manifest: derived at load time from the app block + Pause/Review steps, describing planned vs actual surfaces and lifecycle.
5. Control plane: one `.vl-code/control-plane.sqlite` per workspace, holding the run ledger, node results, review resources, and artifact lineage.

## EXAMPLE / EXERCISE

Core-term cheat sheet:

```
VL Agent          local Agent OS + Agent App Studio
Workflow / DAG    Workflow Spec 4.1, engine 4.13.1, event vl.workflow.run-event.v4
Agent App         manifest + entry workflow (generated from intent)
.vflow            single-file app pack; -> Resource Center -> VLCode
Master Agent      Planner/Evaluator/Scheduler/Arbiter loop
DAG Operator      LLM-free natural-language run control
Control plane     .vl-code/control-plane.sqlite (per workspace)
ViewKit           registry of pre-built reusable UI components
```

## PITFALL

Don't treat 'workflow' and 'Agent App' as synonyms: a workflow is the DAG itself, while an Agent App is the whole app of 'entry workflow + app block + derived manifest.' Conflating them causes misunderstandings when saving a .vflow or deriving the manifest.